

!! DOKUMENTASI UPDATE LOG !!

Note ini berisi update log secara kronologis dari perubahan terkini pada Oden Webix / Dokumentasi Oden.

Hari, Tanggal	Note	Deskripsi update
Jumat, 24-11-2023	<u>oden-dev-3 : UPDATE ODEN ASYNCHRONO US</u>	UPDATE ODEN ASYNCHRONOUS
Kamis, 16-10-2023	<u>5.27 How To Create Subchild Multiview Datagrid (1 Table).</u>	Dokumentasi : Multiview Subchild Datagrid merupakan konsep dimana sebuah view datagrid yang memiliki 1 table, dipecah - pecah ke multiview - multiview yang berbeda - beda.
----- NOVEMBER--- ---	----- -----	----- -----
Rabu, 25-10-2023	<u>2.9.1 How to Create Uploader File in Oden Webix NonBase</u>	Dokumentasi : Upload data ke Bucket NonBase
Senin, 23-10-2023	<u>2.9 How to Create Uploader File in Oden Webix</u>	Dokumentasi : Update File Uploader terbaru
Kamis, 19-10-2023	<u>5.25 How To Add Hak Branch in Oden Webix</u>	Dokumentasi : Hak Branch adalah daftar branch branch yang memiliki akses pada query sql tertentu dan biasanya digunakan pada where clause suatu query sql.
Kamis, 19-10-2023	<u>5.14 How To Create Non- Base Table Block Filter In Oden Webix</u>	Dokumentasi : Mengupdate dengan menambahkan restapi.isWithoutApi()

Selasa, 17-10-2023	5.24 How To Create Custom Footer	Dokumentasi: Pembuatan Custom Footer pada Datagrid untuk menggunakan fungsi (MAX,MIN,SUM,AVG)
Selasa, 17-10-2023	5.23 How To Create Subparent Multiview Form (1 Table)	Dokumentasi : Subparent Multiview merupakan sebuah fitur dimana sebuah form parent memiliki atribut dalam satu tabel database yang letaknya tidak pada form header melainkan pada Multiview dibawahnya. Tujuan : Agar Form pada parent dapat dipisah ke multiview dibawahnya
----- MEI-----	----- -----	----- -----
Kamis, 25-05-2023	5.21 How to Configure Load Data at Start for Multiview	Dokumentasi load data at start untuk CHILD MULTIVIEW Tujuan : untuk efisiensi load data saat pertama kali membuka form. Dengan fitur ini, dapat membuat restapiparam untuk load disaat kita klik multiview tersebut.
Rabu, 24-05-2023	5.2 How To Hide Button in Oden Webix	Update dokumentasi hide button untuk addRowDb
Rabu, 24-05-2023	5.20 How to Configure Load Data at Start for LOV	Dokumentasi load data at start untuk LOV Tujuan : untuk efisiensi load data saat pertama kali membuka form. Dengan fitur ini, dapat membuat lov load disaat kita klik lov (tidak dari awal load form)
Senin, 08-05-2023	5.19 How To Call Form and Call Back in Oden Webix	Update Dokumentasi Call form : Atribut CALL_FORM pada button untuk membuat button menjadi disable saat search atau insert.
Jumat, 05-05-2023	1.1.2 Hasil Generate File	

PY	Update self.limit, attribute baru pada datablocks datagrid. self.limit digunakan untuk override limit otomatis yang diterapkan oden pada datagrid yaitu 500.	
----- APRIL----- -----	-----	----- ----- -----
Jumat, 28-04-2023	<u>5.19 How To Call Form and Call Back in Oden Webix</u>	Update fitur baru : Call form dan Call Back
Senin, 10-04-2023	<u>5.18 How To Display SUM, AVERAGE, MIN, MAX in Datagrid</u>	Update cara untuk mendapatkan nilai operasi pada tampungan javascript
Rabu, 05-04-2023	<u>5.12 How To use checkAllDt, uncheckAllDt, getAllChecklistData on Checkbox in Datagrid</u>	Update note fungsi getAllChecklistData. Untuk mendapatkan data yang terchecklist saja. Pada Datagrid.
----- MARET--- -----	-----	----- ----- -----
Jumat, 31-03-2023	<u>5.18 How To Display SUM, AVERAGE, MIN, MAX in Datagrid</u>	Dokumentasi tampilan hasil perhitungan SUM, MAX, MIN, AVG pada Datagrid di Oden Webix
Jumat, 31-03-2023	<u>2. List Element Oden Webix</u>	Ditambahkan property " HIDE " pada element
Jumat, 17-03-2023	<u>5.17 How To Format Ajax Data in Custom API using formatParamAjax Function Gate</u>	Penambahan dokumentasi formatParamAjax untuk menyesuaikan data yang akan dibawa ke belakang melalui restApiCustom

----- FEBRUARI-----	----- -----	----- -----
Jumat, 17-02-2023	<u>3.1 How To Create Custom Block Validation in Oden Webix</u>	Panduan pembuatan validasi block (object) custom dan fungsi - fungsi Python yang dibutuhkan pada Oden Webix.
Jumat, 17-02-2023	<u>5.15 How To Use GO_BLOCK & GO_ITEM Trigger in Oden Webix</u>	Penambahan dokumentasi fitur trigger GO_BLOCK dan GO_ITEM pada Oden Webix
Kamis, 16-02-2023	<u>5.14 How to Create Non-Base Table Block Filter In Oden Webix</u>	Panduan validasi non-base table filter (membuat block, record, item) dan validasi dengan restApiCustom
Kamis, 16-02-2023	<u>5.13 How to use addDataRecordDt in Oden Webix</u>	Penggunaan fitur addDataRecordDt (menambahkan otomatis record pada datagrid)
Kamis, 16-02-2023	<u>5.7 RestApiCustom in Oden Webix</u>	Penjelasan Cara penggunaan fitur restApiCustom
Selasa, 07-02-2023	<u>5.12 How to use checkAllDt and uncheckAllDt on Checkbox in Datagrid</u>	Penambahan fitur fungsi checkAllDt dan uncheckAllDt pada checkbox di datagrid.

Senin, 06-02-2023	2.3 How To Create Lov in Oden Webix	Penambahan dokumentasi CUSTOM set LOV (window lov berisi kolom yang lebih banyak daripada tampilan LOV)
----- JANUARI-----	----- -----	----- -----
Kamis, 26-01-2023	5.11 How To Use Multi Tab View in Oden Webix	Penambahan fitur view multi tab pada datagrid / detail.

0. Table of Content

Table of Content Oden Webix

1. **Persiapan Penggunaan Framework Oden**

1.1. How To Install and Configure Generator Oden GUI

1.1.1 Hasil Generate File JS

1.1.2 Hasil Generate File PY

1.2 UML Class Diagram Oden (Pohon).

2. **List Element Oden Webix**

2.1 How To Create Textbox in Oden Webix

2.2 How To Create Numeric in Oden Webix

2.2.1 2.2.1 How To Create Numeric Telephone

2.3 How To Create LOV in Oden Webix

2.4 How To Create Radio Button in Oden Webix

2.5 How To Create Combo List in Oden Webix

2.6 How To Create Checkbox in Oden Webix

2.7 How To Create Date in Oden Webix

2.8 How To Create Button in Oden Webix

2.9 How To Create Uploader File in Oden Webix

2.9.1 How to Create Uploader File in Oden Webix NonBase

3. **How to Create Validation in Oden Webix**

3.1 How To Create Custom Block Validation in Oden Webix

4. **Override Method in Python**

[4.1 Pre SELECT Method](#)

[4.2 Post SELECT Method](#)

[4.3 Pre INSERT Method](#)

[4.4 Post INSERT Method](#)

[4.5 Pre UPDATE Method](#)

[4.6 Post UPDATE Method](#)

[4.7 Pre DELETE Method](#)

[4.8 Post DELETE Method](#)

5. **Fitur Oden Framework**

[5.1 How to Change Form Title in Oden Webix](#)

[5.2 How to Hide Button in Oden Webix](#)

[5.3 How to Use Special Character in Oden Webix](#)

[5.4 Form Instances in Oden Webix](#)

[5.4.1 How To Create New Form Instance JS in Oden Webix](#)

[5.4.2 How To Create New Form Instance PY in Oden Webix](#)

[5.5 How to Disable Item in Oden Webix](#)

[5.6 How To Use Pre Item in Oden Webix](#)

[5.7 RestAPI Custom in Oden Webix](#)

[5.8 Session Logged ID in Oden Webix](#)

[5.9 How to Create Automatic Code in Oden Webix](#)

[5.10 Shortcut keyboard in Oden Webix](#)

[5.11 How To Use Multi Tab View in Oden Webix](#)

[5.12 How To use checkAllDt, uncheckAllDt, _getAllChecklistData on Checkbox in Datagrid](#)

[5.13 How To use addDataRecordDt in Oden Webix](#)

[5.14 How To Create Non-Base Table Block Filter In Oden Webix](#)

[5.15 How To Use GO BLOCK & GO ITEM Trigger in Oden Webix](#)

[5.16 How To Custom Pagination Information in Datagrid](#)

[5.17 How To Format Ajax Data in Custom API using formatParamAjax Function Gate](#)

[5.18 How To Display SUM, AVERAGE, MIN, MAX in Datagrid](#)

[5.19 How To Call Form and Call Back in Oden Webix](#)

[5.20 How to Configure Load Data at Start in winConfigForm and winRestApi](#)

[5.21 How to Configure Load Data at Start for Multiview](#)

[5.22 How to Make additional Custom Row in CustomOperatorDatagrid](#)

[5.23 How To Create Subparent Multiview Form \(1 Table\)](#)

[5.24 How To Create Custom Footer](#)

5.25 How To Add Hak Branch in Oden Webix

5.26 How To Make Date Format in LOV

5.27 How To Create Subchild Multiview Datagrid (1 Table).

6. **Utilities**

6.1. Query SELECT

6.2. Query Execute



TABLE OF CONTENT - 2022

1. Persiapan Penggunaan Framework Oden di Compute Engine

Dokumentasi langkah - langkah persiapan sebelum menggunakan framework Oden pada CE (Compute Engine).

1. Buka SFTP tools (WinSCP atau Filezilla)
2. Masukkan **Host** (ip) : 34.101.215.169, **Port** : 22, Jangan lupa gunakan Json **PPK** pada private key file di konfigurasi tool sftp. **Password** adalah (nama)+234
3. Masuk ke folder **oden-dev-3** (cd oden-dev-3) pada WinSCP / Filezila
4. Konfigurasi port pada fungsi app.run di main.py sesuai dengan port yang telah dibagikan,
5. Open session di PuTTY, masukkan password : (nama)+234.

Note : Jika belum mempunyai PuTTY, silahkan download di : [Download PuTTY 64 Bit Windows](#)

6. Jalankan sintaks berikut :

```
python3 main.py
```

8. Tunggu beberapa saat,
9. Buka 34.101.215.169:(PortAnda) di browser.

1.1 How To Install and Configure Generator Oden GUI

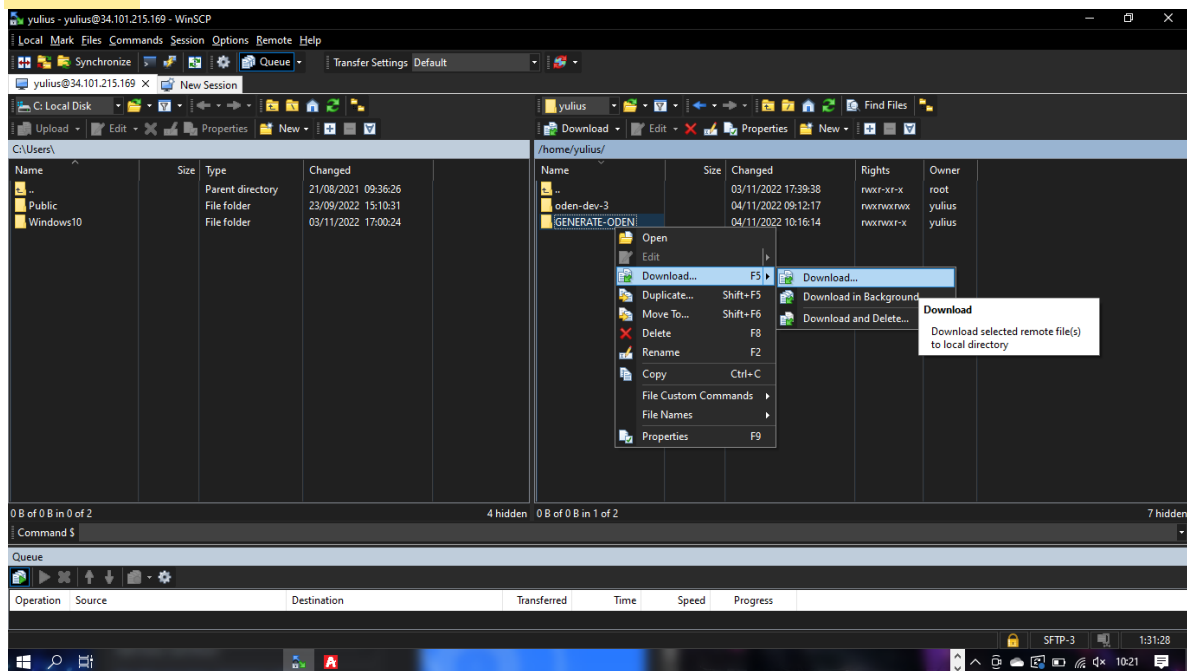
Dokumentasi untuk instalasi dan konfigurasi Generator ODEN GUI

INSTALASI

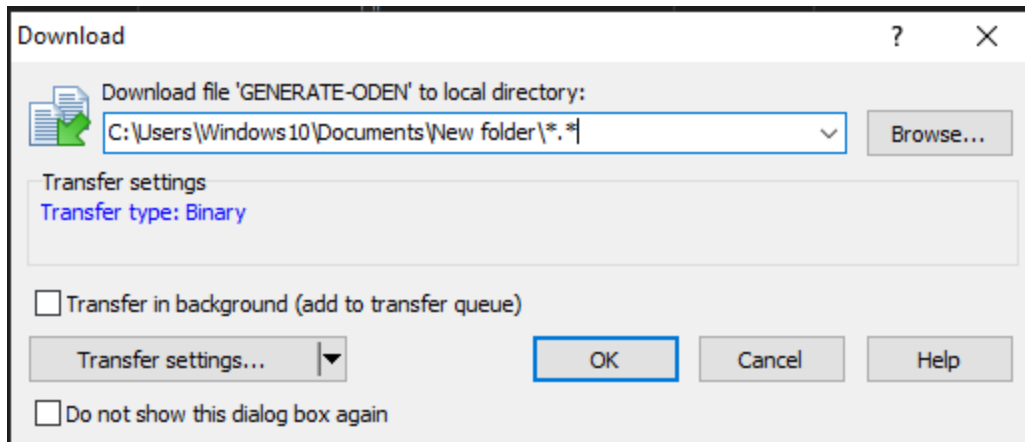
- Setelah berhasil masuk ke WinSCP, masuk Putty dan copy folder **/oden-project/GENERATE-ODEN** kedalam folder anda

```
cp -R /home/oden-project/GENERATE-ODEN /home/[NAMA ANDA]
```

- Jika sudah, download folder **GENERATE-ODEN** dengan cara pilih Folder lalu klik Download



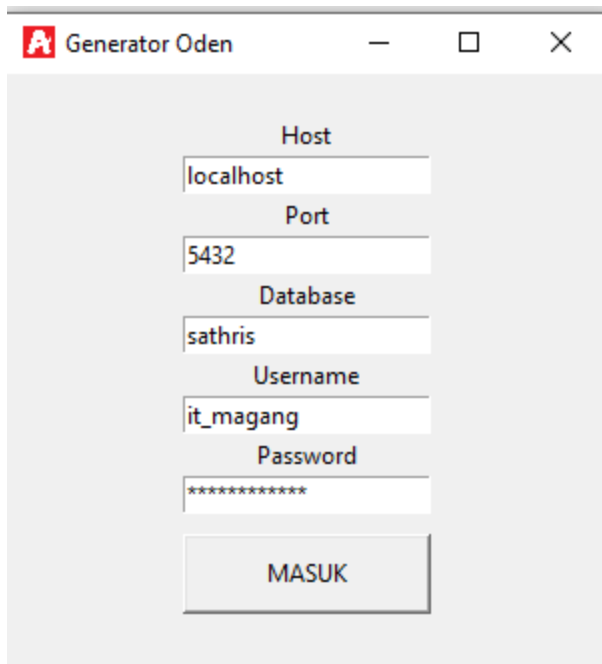
- Lalu akan muncul popup window, pilih lokasi download file dan klik OK



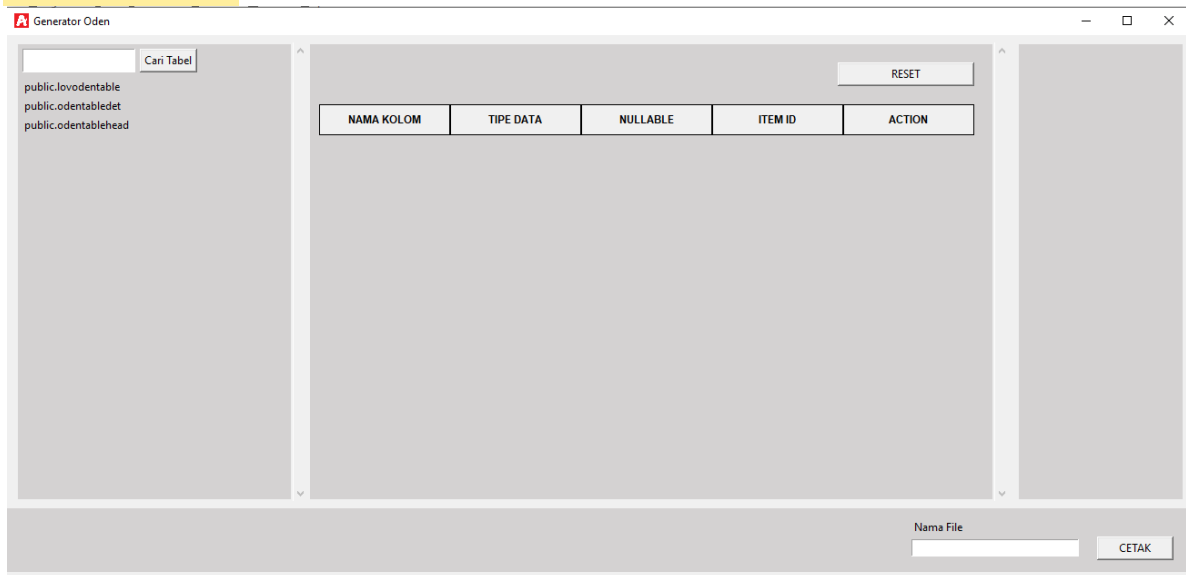
- Buka **Command Prompt** dan arahkan ke dalam direktori dimana **cloud_sql_proxy.exe** berada. dan ketikkan kode dibawah ini

```
cloud_sql_proxy.exe -instances=satbi-dev:asia-east2:satbi-dev-  
db=tcp:5432
```

- Buka lokasi download folder dan jalankan **main.py**. Maka tampilan akan muncul seperti ini

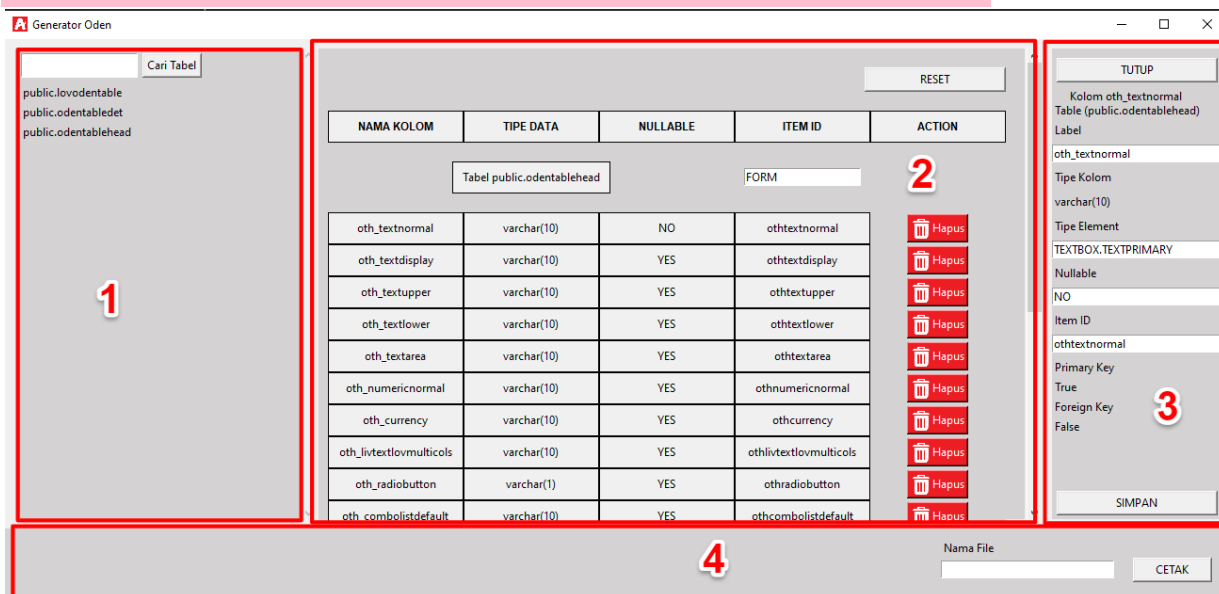


- Sesuaikan isian form dengan data yang ada. Jika sudah klik **MASUK** dan tampilan akan muncul seperti ini



- Proses Instalasi Oden telah selesai

PENJELASAN TAMPILAN DALAM ODEN GUI



Penjelasan tampilan dalam Oden GUI

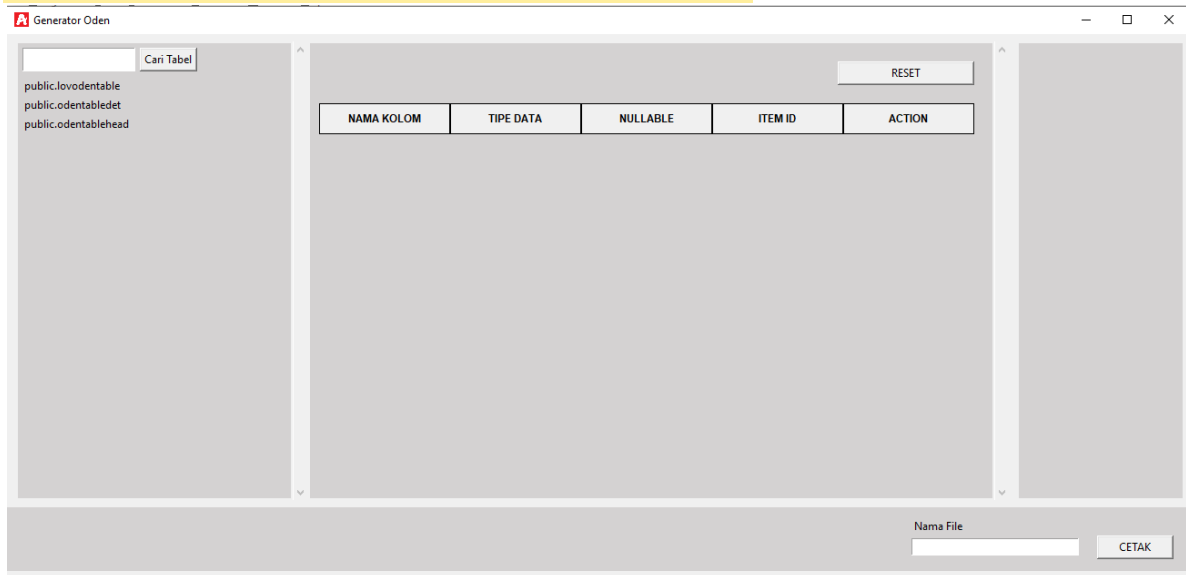
- Nomor **1**. List tabel dalam database
- Nomor **2**. Detail tabel yang dipilih dari list tabel. List tabel ini yang nanti akan di export ke dalam file PY dan file JS
- Nomor **3**. Detail properti dari masing masing baris dalam detail tabel.
- Nomor **4**. Nama file

KONFIGURASI

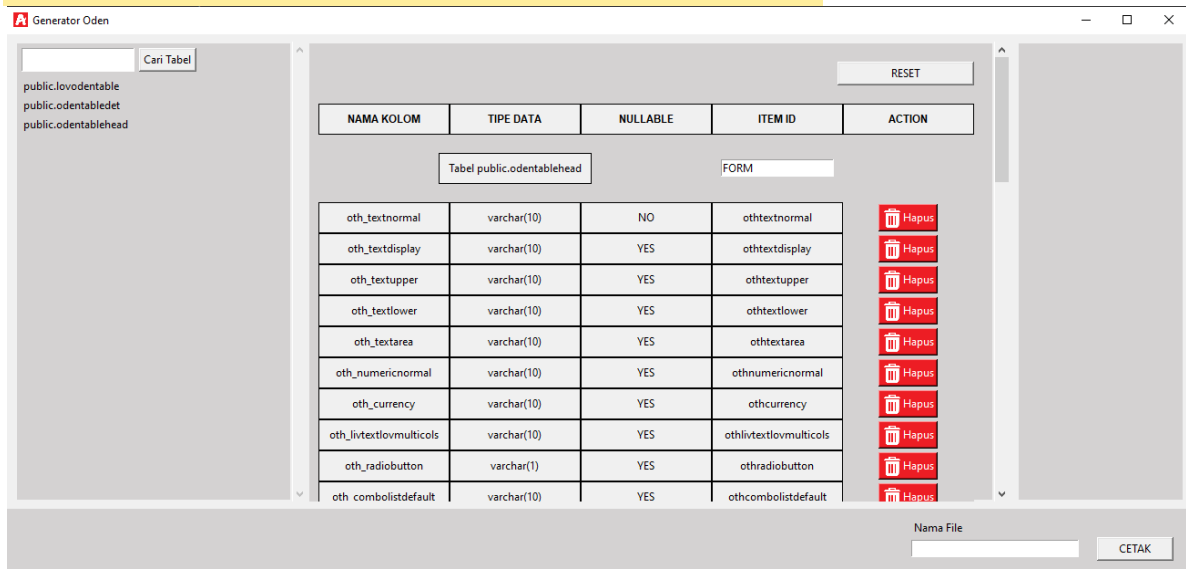
Generator Oden Webix ini digunakan untuk *generate* file Javascript dan file Python.

Berikut langkah-langkahnya:

- Buka Oden Webix hingga muncul tampilan seperti ini



- Drag and drop salah satu tabel maka akan muncul detail tabel



- Jika sudah muncul, maka tekan button **public.odentablehead** untuk memunculkan propertinya.

RESET

NAMA KOLOM	TIPE DATA	NULLABLE	ITEM ID	ACTION
Tabel public.odentablehead				FORM
oth_textnormal	varchar(10)	NO	othtextnormal	Hapus
oth_textdisplay	varchar(10)	YES	othtextdisplay	Hapus
oth_textupper	varchar(10)	YES	othtextupper	Hapus
oth_textlower	varchar(10)	YES	othtextlower	Hapus
oth_textarea	varchar(10)	YES	othtextarea	Hapus
oth_numericnormal	varchar(10)	YES	othnumericnormal	Hapus
oth_currency	varchar(10)	YES	othcurrency	Hapus
oth_livtextlovmulticols	varchar(10)	YES	othlivtextlovmulticols	Hapus
oth_radiobutton	varchar(1)	YES	othradiobutton	Hapus
oth_combolistdefault	varchar(10)	YES	othcombolistdefault	Hapus

TUTUP

Tabel public.odentablehead

Created User

oth_create_user

Updated User

oth_update_user

SIMPAN

- Sesuaikan isian dengan nama kolom yang digunakan untuk trigger kolom update_user dan create_user pada tabel. Jika sudah Klik **SIMPAN**.

- Hapus beberapa kolom yang tidak digunakan saat generate form. Anda juga dapat mengubah properti kolom sesuai keinginan seperti berikut.

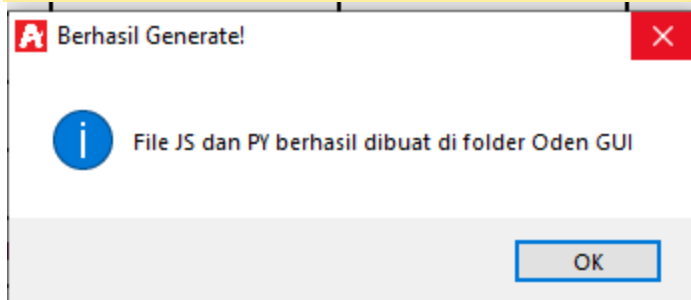
The screenshot shows a configuration window for a column. At the top is a 'TUTUP' button. Below it, the column name 'Kolom oth_textnormal' and table 'Table (public.odentablehead)' are displayed. The 'Label' field contains 'INI LABELLL'. The 'Tipe Kolom' is 'varchar(10)' and the 'Tipe Element' is 'TEXTBOX.TEXTPRIMARY'. The 'Nullable' field is set to 'NO'. The 'Item ID' is 'othtextnormal'. The 'Primary Key' is 'True' and the 'Foreign Key' is 'False'. At the bottom is a 'SIMPAN' button.

- Penjelasan Kolom Properti
 - Label**, Berisi Label dan Placeholder pada kolom Inputan. Dapat diubah sesuai keinginan
 - Tipe Kolom**, Berisi Tipe dari kolom inputan
 - Tipe Element**. Silahkan isi sesuai dengan tipe inputan seperti pada **TABEL LIST ELEMENT**.
 - Nullable**, Berisi NO / YES untuk menandakan apakah kolom tersebut dapat kosong atau tidak.
 - Item ID**, Berisi ID dari Kolom Inputan.
 - Primary Key**, Berisi True / False untuk menandakan apakah kolom tersebut merupakan Primary Key atau bukan
 - Foreign Key**, Berisi True / False untuk menandakan apakah kolom tersebut merupakan Foreign Key atau bukan

- Jika sudah klik **SIMPAN**.
- Selanjutnya masuk ke bagian nama file lalu isi nama file sesuai dengan keinginan.



- Jika sudah klik **CETAK**. Maka selanjutnya akan muncul popup status generate



- Selanjutnya muncul 2 File baru yaitu **file PY** dan **file JS** di dalam folder yang sama dengan **odenGUI.exe**

Name	Date modified	Type	Size
assets	04/11/2022 9:48	File folder	
INI NAMA FILEEE.js	04/11/2022 10:30	JavaScript Source ...	4 KB
INI NAMA FILEEE.py	04/11/2022 10:30	Python File	17 KB
main.exe	05/10/2022 22:31	Application	9.672 KB

- Proses Konfigurasi Oden telah berhasil dilakukan hingga generate file **PY** dan file **JS**.
Silahkan beralih ke modul selanjutnya untuk mempelajari penggunaan Oden lebih lanjut.

[KEMBALI KE TABLE LIST OF CONTENT](#)

1.1.1 Hasil Generate File JS

Dokumentasi penjelasan lengkap file Javascript hasil generate dari Oden Generator.

File JS hasil generate dari Oden Generator :

```

var menuId = 'TutorialOden'
var winFormTitle = 'TutorialOden'
var winConfigForm = {
  'ODENTABLEHEAD': {'ELEMENT': ['othtextnormal', 'othtextdisplay'], 'PRIMARY_KEY': ['othtextnormal'],
    'RELATION_KEY': [], 'WHEN_NEW_FORM': {}, 'BLOCK_TYPE': ['FORM', 'PARENT']},
  'ODENTABLEDET': {'ELEMENT': ['otdtextnormal', 'otdothtextnormal', 'otdtextdisplay'], 'PRIMARY_KEY': ['otdtextnormal'],
    'RELATION_KEY': ['othtextnormal'], 'WHEN_NEW_FORM': {}, 'BLOCK_TYPE': ['DATAGRID', 'CHILD']}}

var winRestApi = function(){
  restapi.restApiParam({"BLOCK_NAME":ODENTABLEDET,"PARAM_ID":["othtextnormal"]});
}

var hideButton = {
  // true = hidden button
  // false = show button
  "next" : false,
  "searchValue" : false,
  "addValue" : false,
  "updateValue" : false,
  "deleteValue" : false,
  "clearBtn" : false
}

let elementODENTABLEHEAD = [
  TEXTBOX.TEXTPRIMARY({ITEM_ID: "othtextnormal", LABEL: "oth_textnormal", LENGTH: 10}),
  TEXTBOX.TEXTNORMAL({ITEM_ID: "othtextdisplay", LABEL: "oth_textdisplay", LENGTH: 10}),
]

let elementODENTABLEDET = [
  TEXTBOX.TEXTPRIMARY({ITEM_ID: "otdtextnormal", LABEL: "otd_textnormal", LENGTH: 10}),
  TEXTBOX.TEXTNORMAL({ITEM_ID: "otdothtextnormal", LABEL: "otd_oth_textnormal", LENGTH: 10}),
  TEXTBOX.TEXTNORMAL({ITEM_ID: "otdtextdisplay", LABEL: "otd_textdisplay", LENGTH: 10}),
]

let rulesODENTABLEHEAD = {
  "othtextnormal":webix.rules.isNotEmpty,
}

let rulesODENTABLEDET = {
  "otdtextnormal":webix.rules.isNotEmpty,
}

let blockODENTABLEHEAD = form.SimpleForm("ODENTABLEHEAD", elementODENTABLEHEAD, rulesODENTABLEHEAD)
let blockODENTABLEDET = datatable.SimpleDatatable("ODENTABLEDET",elementODENTABLEDET,rulesODENTABLEDET,[0,0])

webix.ready(function(){
  restapi.restApiSelect()
})

let content = layout.baseLayout([blockODENTABLEHEAD, blockODENTABLEDET, ]);

function renderLov(){
  return []
}

```

Penjelasan masing - masing variabel dan fungsi dalam file JS tergenerate :

1. **var menuId**

```
var menuId = "TutorialOden"
```

Variabel menuId diatas selaras mengikuti nama file saat user generate file dari Oden Generator.

Note : Perubahan menuId pada file JS harus disertai dengan mengganti menuId pada file PY.

2. var winFormTitle

```
var winFormTitle = "TutorialOden"
```

Variabel winFormTitle mengubah tampilan form. Dokumentasi : [5.1 How To Change Form Title in Oden Webix](#)

3. var winConfigForm

```
var winConfigForm = {  
  "ODENTABLEHEAD": {  
    "ELEMENT": ["othtextnormal", "othtextdisplay", "othtextupper",  
      "othtextlower"],  
    "PRIMARY_KEY": ["othtextnormal"], "RELATION_KEY": [], "WHEN_NEW_FORM":  
    {}, "BLOCK_TYPE": ["FORM", "PARENT"], 'BASE_TABLE' : true  
  },  
  "ODENTABLEDET": {  
    "ELEMENT": ["otdtextnormal", "otdothtextnormal", "otdtextdisplay",  
      "otdtextupper", "otdtextlower", "otdtextarea", "otdnumericnormal",  
      "otdcurrency", "otdlivtextlovmulticols", "otdradiobutton",  
      "otdcombolistdefault", "otdcombolistapi", "otdcombolistapidata",  
      "otdnormalcheckbox", "otdnormalcheckboxinteger", "otdddmmmy", "otdmmmy",  
      "otdyy"],  
    "PRIMARY_KEY": ["otdtextnormal"], "RELATION_KEY": [], "WHEN_NEW_FORM":  
    {}, "BLOCK_TYPE": ["DATAGRID", "CHILD"], 'BASE_TABLE' : true,  
    'REST_API': {'BLOCK_NAME': 'ODENTABLEDET', 'PARAM_ID': ['othtextnormal']},  
    //karena form form  
  },  
}
```

Variabel winConfigForm berisi konfigurasi form berbentuk dictionary per block id.

BLOCK ID adalah key dengan value berbentuk dictionary berisi daftar element, primary key, relation key, when new form, dan block type.

ELEMENT adalah key dengan value berisi list daftar element yang dipilih saat generate, atau jika user menambahkan element baru.

PRIMARY_KEY adalah key dengan value berisi list primary key pada block id tersebut.

RELATION_KEY adalah key dengan value berisi list relation key pada block id tersebut.

WHEN_NEW_FORM adalah key dengan value berupa dictionary berisi item id dengan value static yang akan ditampilkan pada NEW FORM INSTANCES.

Catatan : Penggunaan WHEN_NEW_FORM dapat dilihat pada Dokumentasi [5.4.1 How To Create New Form Instance JS IN ODEN WEBIX](#)

BLOCK_TYPE adalah key dengan value list. Value pertama dalam list mendefinisikan jenis block tersebut (FORM / DATAGRID), value kedua mendefinisikan parent/ child dari tampilan yang akan dibuat.

NOTE: Jika terdapat ITEMID pada list ELEMENT yang dihapus, jangan lupa untuk menghapus ITEMID pada RulesBLOCKID

Contoh, jika menghapus element othtextnormal, hapus juga rules webixNotEmpty pada variabel rulesODENTABLEHEAD seperti gambar di bawah ini:



```
3  var winConfigForm = {
4  'ODENTABLEHEAD': {
5    'ELEMENT': ['othtextnormal', 'othddmmy', 'othddmmy_timestamp', 'othddmmy_clock'],
6    'PRIMARY_KEY': ['othtextnormal'],
7    'RELATION_KEY': [],
8    'WHEN_NEW_FORM': {},
9    'BLOCK_TYPE': ['FORM', 'PARENT'],
10   'BASE_TABLE': 'Y'
11 },
12
105 let rulesODENTABLEHEAD = {
106   "othtextnormal": webixValueIsNotEmpty,
107 }
```

4. var winRestApi

```
var winRestApi = function(){
  restapi.restApiParam({"BLOCK_NAME":ODENTABLEDET,"PARAM_ID":
  ["othtextnormal"]});
}
```

Variabel winRestApi adalah fungsi yang memanggil restapi.restApiParam.

Fungsi ini ditambahkan jika pada tabel tidak ada relasi foreign-key / relasi antar tabel.

UNTUK MEMBUAT CHILD MULTIVIEW LOADING DATA SAAT NAMA MULTIVIEW DI KLIK, SILAHKAN BACA DOKUMENTASI BERIKUT

5.21 How to Configure Load Data at Start for Multiview

5. var hideButton

```
var hideButton = {
  // true = hidden button
  // false = show button
  "next" : false,
  "searchValue" : false,
  "addValue" : false,
  "updateValue" : false,
```

```

"deleteValue" : false,
"clearBtn" : false
}

```

Variabel `hideButton` memungkinkan user untuk menghapus tombol pada form Oden.

Dokumentasi : [5.2 How To Hide Button in Oden Webix](#)

6. `let elementBlockID`

```

let elementODENTABLEHEAD = [
TEXTBOX.TEXTPRIMARY({ITEM_ID: "othtextnormal", LABEL: "Text Primary",
LENGTH: 10}),
TEXTBOX.TEXTNORMAL({ITEM_ID: "othtextdisplay", LABEL: "Text Display",
LENGTH: 10}),
]
let elementODENTABLEDET = [
TEXTBOX.TEXTPRIMARY({ITEM_ID: "otdtextnormal", LABEL: "Text Primary",
LENGTH: 10}),
TEXTBOX.TEXTNORMAL({ITEM_ID: "otdothtextnormal", LABEL: "Text Normal",
LENGTH: 10}),
TEXTBOX.TEXTNORMAL({ITEM_ID: "otdtextdisplay", LABEL: "Text Display",
LENGTH: 10}),
]

```

Variabel `elementBlockId` berisi element-element yang tergenerate dari Generator Oden. Pada variabel `elementBlockId` ini bagian dimana user melakukan konfigurasi elemen serta penambahan elemen jika dibutuhkan. Contohnya, pada `elementODENTABLEDET` :

```

TEXTBOX.TEXTNORMAL({ITEM_ID: "otdtextdisplay", LABEL: "Text Display",
LENGTH: 10}),

```

Diubah menjadi :

```

TEXTBOX.TEXTDISPLAY({ITEM_ID: "otdtextdisplay", LABEL: "Text Display",
LENGTH: 10}),

```

Note : Untuk melihat list element silahkan kunjungi, [2. List Element Oden Webix](#).

7. `let rulesBlockID`

```

let rulesODENTABLEHEAD = {
"othtextnormal":webix.rules.isNotEmpty,
}
let rulesODENTABLEDET = {

```

```
"otdtextnormal":webix.rules.isNotEmpty,
}
```

Variabel rulesBLOCKID berisi itemid beserta fungsi bawaan webix yaitu webix.rules.isNotEmpty.

Fungsi webix.rules.isNotEmpty membuat itemId terdaftar pada variabel ini tidak boleh kosong. Jika variabel kosong, maka webix akan memberikan validasi bahwa field tidak boleh kosong.

8. let blockBlockID

```
let blockODENTABLEHEAD = form.SimpleForm("ODENTABLEHEAD",
elementODENTABLEHEAD, rulesODENTABLEHEAD)
let blockODENTABLEDEET =
datatable.SimpleDatatable("ODENTABLEDEET",elementODENTABLEDEET,rulesODENTA
BLEDET,[0,0])
```

Variabel blockBlockID diatas menentukan jenis form / datatable (untuk datagrid) pada masing - masing block yang ada.

Parameter di dalam value berupa blockId, elementBlockId, dan rulesBlockId.

Fungsi datatable.SimpleDatatable menerima 4 parameter, yaitu: Block_ID, variabel element block, variabel rules block, dan Freeze Column*

****NOTE****

- * Freeze Column datagrid secara default memiliki value [0, 0] artinya tidak ada kolom yang dibekukan pada sisi kanan ataupun kiri, nilai pada list menandakan jumlah kolom yang akan dibekukan pada datagrid.

- * Jumlah Freeze Column yang dapat diterapkan masih terbatas, apabila terdapat kasus khusus yang memerlukan beberapa Freeze Column sekaligus, dapat dikonsultasikan terlebih dahulu dengan Tim Oden.

9. webix.ready(function...

```
webix.ready(function(){
restapi.restApiSelect()
})
```

Fungsi dalam webix.ready **dapat** berisi restApiSelect, restApi untuk Lov, dan restApi untuk combo form dan combo datagrid.

Dokumentasi tentang pembuatan Lov : [2.3 How to Create LOV in Oden Webix](#)

Dokumentasi tentang pembuatan ComboListAPI : [2.5 How to create Combo List in ODEN Webix](#)

CATATAN PENTING :

```
restapi.restApiSelect()
```

Pada fungsi ini, kode `restApiSelect` di atas **harus diletakkan di urutan terakhir / paling bawah** setelah `restApi` lainnya.

Contoh :

```
webix.ready(function() {  
  restapi.restApi({"ITEM_ID":"dtDisplayHeaderView","API_NAME":"lovHead_"+menuId})  
  restapi.restApiSelect('/'+menuId+'_data',{})  
})
```

10. **let content**

```
let content = layout.baseLayout([blockODENTABLEHEAD, blockODENTABLEDET,  
]);
```

Variabel `content` di atas mengembalikan `layout.baseLayout` memiliki parameter list berisi variabel `blockBlockID` seperti pada nomor 8 di atas.

11. **function renderLov**

```
function renderLov() {  
  return []  
}
```

Fungsi `renderLov` diatas mengembalikan `return` sehingga Oden dapat memberikan tampilan window pada Lov.

Dokumentasi pembuatannya dapat dilihat di [2.3 How to Create LOV in Oden Webix](#).

1.1.2 Hasil Generate File PY

Dokumentasi penjelasan lengkap file Python hasil generate dari Oden Generator.

Penjelasan Fungsi dalam file PY yang dihasilkan Oden Generator :

- **Class Form Layout**

```

1 class COdentablehead(FormLayout):
2     """
3     Class ini digunakan untuk class block layout
4     @return: -
5     """
6     def __init__(self, blockId):
7         super().__init__(blockId)
8         self.whereclause = ""
9         self.databaseTable = "dmhr.odentablehead"
10        self.nonVisibleItems = [{"oth_create_user", "createUser"}, {"oth_update_user", "updateUser"}] #ketik manual kolom yang mau ditempatkan sebagai non visible items
11        self.noDbItems = []
12        self.orderBy = ""
13

```

Class form layout standar untuk mengatur atribut - atribut pada halaman form Oden, di mana pada class ini dapat ditentukan nama tabel database yang digunakan (**self.databaseTable**), item - item pada database yang ditetapkan sebagai non-visible (**self.nonVisibleItems**), dan apa saja item pada form yang bukan merupakan item dari database. **self.orderBy**= ""

- **Class Layout untuk datagrid (Datagrid detail)**

```

1 class COdentabledet(DatagridLayout):
2     """
3     Class ini digunakan untuk class block layout
4     @return: -
5     """
6     def __init__(self, blockId):
7         super().__init__(blockId)
8         self.whereclause = "otd_oth_textnormal = %(othtextnormal)s"
9         self.databaseTable = "dmhr.odentabledet"
10        self.nonVisibleItems = [{"otd_create_user", "createUser"}, {"othd_update_user", "updateUser"}] #ketik manual kolom yang mau ditempatkan sebagai non visible items
11        self.noDbItems = []
12        self.orderBy = ""
13

```

NOTE : ADA UPDATE BARU UNTUK INIT (self.limit) PADA DATAGRID:

self.limit = "25"

self.limit akan melakukan override pada limit otomatis yang diterapkan Oden pada datagrid, yaitu 500.

Layout datagrid merupakan layout yang akan ditampilkan pada form berbentuk master detail memiliki atribut - atribut yang hampir sama dengan class layout berjenis form, tetapi user perlu menentukan kolom / item yang menjadi constraint **foreign key** antara tabel master dan tabel detail pada variabel **self.whereclause**.

Pada Datagrid juga dapat ditambahkan variabel **self.orderBy** untuk mengurutkan row datagrid berdasarkan kolom dan urutan yang ditentukan, pada contoh di atas baris datagrid diurutkan menurut kolom *oth_textnormal* dengan urutan ASC.

Contoh :

```
self.orderBy = "oth_textdisplay DESC"
```

Form Master Detail Tanpa Relasi Foreign Key :

1. Jika membuat form berbentuk master detail dan pada tabel tidak ada relasi FOREIGN_KEY (relasi antar tabel), silahkan tambahkan script di bawah ini

```
self.whereclause = "namaKolomForeignKey = %(itemIDPrimaryKeyMaster)s"
```

otd_oth_textnormal adalah nama kolom dari tabel **dmhr.odentabledet** yang digunakan sebagai FOREIGN KEY yang mengacu ke kolom **othtextnormal** dari tabel **dmhr.odentablehead**

Contoh : **self.whereclause = "otd_oth_textnormal = %(othtextnormal)s"**, seperti pada baris 34 gambar script di atas.

2. Jika membuat form berbentuk master detail dan pada tabel tidak ada relasi FOREIGN_KEY (relasi antar tabel), silahkan samakan **itemID** pada class yang mengacu pada **FOREIGN KEY** dengan class yang mengacu pada **PRIMARY KEY**.

Contoh :

Contoh Item PRIMARY	Contoh Item FOREIGN
<pre>class Coth_textnormal(TextPrimary): lengthItem = 10 itemId = "othtextnormal" databaseColumn = "oth_textnormal" databaseItem = True nullable = False primaryKey = True allowSpecialChar = [] def validate(self): """ Fungsi ini digunakan untuk validasi column oth_textnormal @return: dictionary dengan key status = nilai boolean (True = validasi berhasil, False = validasi gagal) key msg = nilai string """ return {"status": True, "msg": ""}</pre>	<pre>class Coth_textnormal(Text lengthItem = 10 itemId = "othtextnormal" databaseColumn = "oth_otd_ databaseItem = True nullable = True primaryKey = False allowSpecialChar = [] def validate(self): """ Fungsi ini digunakan untuk oth_textnormal @return: dictionary dengar nilai boolean (True = vali False = validasi gagal) key msg = nilai string """ return {"status": True, "n</pre>

- c. Tabel master - class primary key : terdapat contoh validasi tiap item berupa comment. Uncomment yang diperlukan dan copy-paste ke tiap class yang merupakan bagian dari tabel master. Dan silahkan tambahkan logic script validate sesuai form.

Contoh :

```
22
23 class Coth_textnormal(TextPrimary):
24     lengthItem = 10
25     itemId = "othtextnormal"
26     databaseColumn = "oth_textnormal"
27     databaseItem = True
28     nullable = False
29     primaryKey = True
30     allowSpecialChar = []
31     def validate(self):
32         """
33         Fungsi ini digunakan untuk validasi column oth_textnormal
34         @return: dictionary dengan key status = nilai boolean (True = validasi berhasil, False = validasi gagal)
35         key msg = nilai string
36         """
37         # othtextnormal = self.item("othtextnormal")
38         # othtextdisplay = self.item("othtextdisplay")
39         # othtextupper = self.item("othtextupper")
40         # othtextlower = self.item("othtextlower")
41         # othtextarea = self.item("othtextarea")
42         # othnumericnormal = self.item("othnumericnormal")
43         # othcurrency = self.item("othcurrency")
44         # othlivtextlovmulticols = self.item("othlivtextlovmulticols")
45         # othradiobutton = self.item("othradiobutton")
46         # othcombolistdefault = self.item("othcombolistdefault")
47         # othcombolistapi = self.item("othcombolistapi")
48         # othcombolistapidata = self.item("othcombolistapidata")
49         # othnormalcheckbox = self.item("othnormalcheckbox")
50         # othnormalcheckboxinteger = self.item("othnormalcheckboxinteger")
51         # othddmmyy = self.item("othddmmyy")
52         # othmmyy = self.item("othmmyy")
53         # othyy = self.item("othyy")
54         return {"status": True, "msg": ""}
55
```

- **Class Base Record Untuk Form Master**

```
13
14 class CRecordOdentablehead(Record):
15     def validate(self):
16         """
17         Fungsi ini digunakan untuk validasi record
18         @return: dictionary dengan key status = nilai boolean (True = validasi berhasil, False = validasi gagal)
19         key msg = nilai string
20         """
21         return {'status': True, 'msg': ''}
22
```

- **Class Base Record Untuk Detail Datagrid**

```
13
14 class CRecordOdentabledet(Record):
15     def validate(self):
16         """
17         Fungsi ini digunakan untuk validasi record
18         @return: dictionary dengan key status = nilai boolean (True = validasi berhasil, False = validasi gagal)
19         key msg = nilai string
20         """
21         return {'status': True, 'msg': ''}
22
```

- **Class Untuk Elemen - Elemen Dalam Form**

```

55
56 class Coth_textdisplay(TextNormal):
57     lengthItem = 10
58     itemId = "othtextdisplay"
59     databaseColumn = "oth_textdisplay"
60     databaseItem = True
61     nullable = True
62     primaryKey = False
63     allowSpecialChar = []
64     def validate(self):
65         """
66         Fungsi ini digunakan untuk validasi column oth_textdisplay
67         @return: dictionary dengan key status = nilai boolean (True = validasi berhasil, False = validasi gagal)
68         key msg = nilai string
69         """
70         return {"status": True, "msg": ""}
71

```

Merupakan class dari masing - masing elemen dalam form, pada contoh di atas merupakan class dari elemen TextPrimary, di mana dalam class tersebut terdapat atribut - atribut yang dapat dikonfigurasi sesuai dengan kebutuhan user.

Contoh : [How to Create Textbox in Oden Webix](#) atau [list elemen](#) lain yang tersedia di Oden Webix.

- **Fungsi Untuk Membentuk Tampilan Berdasarkan Block**

```

311
312 execVcount = """
313 session["loggedId"] = kwargs
314 vContainer = Container()
315 vForm = Form()
316 vContainer.update(vForm)
317 vBlockOdentablehead = COdentablehead("ODENTABLEHEAD")
318 vBlockOdentablehead.setListObj([Coth_textnormal, Coth_textdisplay, Coth_textupper, Coth_textlower, Coth_textarea,
319 Coth_numericnormal, Coth_currency, Coth_livtextlovmulticols, Coth_radiobutton, Coth_combolistdefault, Coth_combolistapi,
320 Coth_combolistapidata, Coth_normalcheckbox, Coth_normalcheckboxinteger, Coth_ddmmyy, Coth_mmyy, Coth_yy, ])
321 vBlockOdentablehead.ObjRecord = CRecordOdentablehead
322
323 vContainer.getForm().dataBlocks = []
324 vContainer.getForm().append(vBlockOdentablehead)
325 """
326

```

- **Fungsi untuk menentukan New Form Instance**

```

326
327 #NEW FORM INSTANCE
328 def newInstance_FORM_ODENTABLEHEAD(**kwargs):
329     newInstance = {'status':True,'data':{},'msg':''}
330     # newInstance = selectHeaderQueryRO(sql)
331     # newInstance = selectDataHeaderQueryRO(sql,param)
332     return newInstance
333

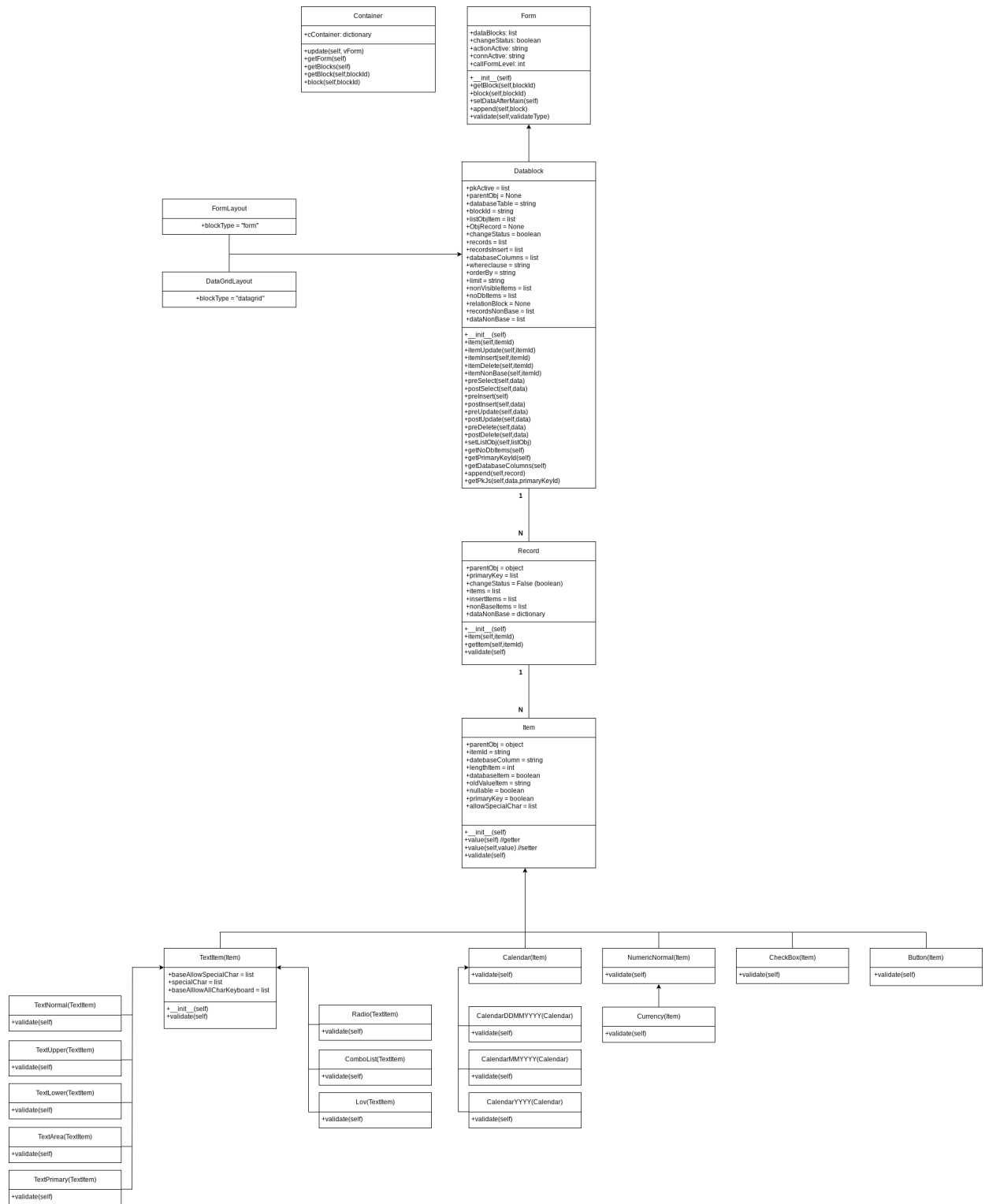
```

Berikut penjelasan secara lengkap dan langkah pembuatan New Form Instance :

1. [New Form Instance JS](#)
2. [New Form Instance PY](#)

1.2 Class Diagram Oden

Update : 8 September 2023



2. List Element Oden Webix

Berikut adalah daftar elemen - elemen yang tersedia di Oden Webix beserta fungsi pada form Javascript dan parameter pada Class Python

ITEM	Item Javascript	Class Python
<u>TEXTBOX</u>	TEXTBOX.TEXTNORMAL(property,trigger)	TextNormal
	TEXTBOX.TEXTDISPLAY(property,trigger)	TextNormal
	TEXTBOX.TEXTUPPER(property,trigger)	TextUpper
	TEXTBOX.TEXTLOWER(property,trigger)	TextLower
	TEXTBOX.TEXTAREA(property,trigger)	TextArea
	TEXTBOX.TEXTPRIMARY(property,trigger)	
<u>NUMERIC</u>	NUMERIC.NUMERICNORMAL(property,trigger)	NumericNormal
	NUMERIC.CURRENCY(property,trigger)	Currency
	NUMERIC.TELEPHONE	NumericNormal
<u>LOV</u>	LOVFIELD.LIVTEXTLOVMULTICOLS(property,trigger)	Lov
<u>RADIO</u>	RADIO.RADIOBUTTON(property,trigger)	ComboList
<u>COMBO</u>	COMBO.COMBOLISTDEFAULT(property,trigger)	ComboList
	COMBO.COMBOLISTAPI(property,trigger)	ComboList
	COMBO.COMBOLISTAPIDATA(property,trigger)	ComboList
<u>CHECKBOX</u> <u>X</u>	CHECKBOX.NORMALCHECKBOX(property,trigger)	CheckBox

<u>DATE</u>	DATE.DDMMYY(property,trigger)	Calendar
	DATE.MMYYY(property,trigger)	Calendar
	DATE.YY(property,trigger)	Calendar
	DATE.CLOCK(property, trigger)	Time
	DATE.TIMESTAMP(property, trigger)	Time
**property	{ITEM_ID:"divisicode",LABEL:"Code",LENGTH:10,HIDE:false}	
**trigger	{PRE_ITEM:namaPreItem,VALIDATE_ITEM:namaTrigger,SEARCH_LIKE:true/false}	

NB: Untuk contoh semua element bisa dilihat pada modul **samplegenerate.py** dan **samplegenerate.js**

[KEMBALI KE TABLE LIST OF CONTENT](#)

2.1 How To Create Textbox in Oden Webix

Dokumentasi untuk membuat element TEXTBOX(TEXTPRIMARY, TEXTNORMAL, TEXTDISPLAY, TEXTUPPER, TEXTLOWER, TEXTAREA) pada Oden Webix.

Text Primary

10

Text Normal

Input User

Text Display

Text Display

Text Upper

INPUT USER

Text Lower

input user

Textarea

Input User

File JS (Folder path : static/application)

1. TEXTBOX.TEXTPRIMARY

Digunakan untuk membentuk element text primary, TEXTPRIMARY tidak dapat di-update. Pada list element di file Javascript, buat sintaks berikut :

```
TEXTBOX.TEXTPRIMARY({ITEM_ID: "itemPrimary", LABEL: "Text Primary",  
LENGTH: 10, IS_PRIMARY_KEY: true}),
```

2. TEXTBOX.TEXTNORMAL

Digunakan untuk membentuk element text normal, TEXTNORMAL dapat di-update. Pada list element di file Javascript, buat sintaks berikut :

```
TEXTBOX.TEXTNORMAL({ITEM_ID: "itemNormal", LABEL: "Text Normal", LENGTH:  
10}),
```

3. TEXTBOX.TEXTDISPLAY

Digunakan untuk membentuk element text display, TEXTDISPLAY bersifat disabled. Pada list element di file Javascript, buat sintaks berikut :

```
TEXTBOX.TEXTDISPLAY({ITEM_ID: "itemDisplay", LABEL: "Text Display",  
LENGTH: 10}),
```

4. TEXTBOX.TEXTUPPER

Digunakan untuk membentuk element text upper, TEXTUPPER mengubah input user menjadi UPPERCASE.

Pada list element di file Javascript, buat sintaks berikut :

```
TEXTBOX.TEXTUPPER({ITEM_ID: "itemUpper", LABEL: "Text Upper", LENGTH: 10}),
```

5. TEXTBOX.TEXTLOWER

Digunakan untuk membentuk element text lower, TEXTLOWER mengubah input user menjadi LOWERCASE.

Pada list element di file Javascript, buat sintaks berikut :

```
TEXTBOX.TEXTLOWER({ITEM_ID: "itemLower", LABEL: "Text Lower", LENGTH: 10}),
```

6. TEXTBOX.TEXTAREA

Digunakan untuk membentuk element textarea, TEXTAREA mengubah size field input menjadi 3 baris.

Pada list element di file Javascript, buat sintaks berikut :

```
TEXTBOX.TEXTAREA({ITEM_ID: "itemTextarea", LABEL: "Textarea", LENGTH: 10}),
```

```
let elementTABLENAME = [
  TEXTBOX.TEXTPRIMARY({ITEM_ID: "othtextnormal", LABEL: "Text Primary", LENGTH: 10, }),
  TEXTBOX.TEXTNORMAL({ITEM_ID: "textnormal", LABEL: "Text Normal", LENGTH: 10}),
  TEXTBOX.TEXTDISPLAY({ITEM_ID: "othtextdisplay", LABEL: "Text Display", LENGTH: 10}),
  TEXTBOX.TEXTUPPER({ITEM_ID: "othtextupper", LABEL: "Text Upper", LENGTH: 10}),
  TEXTBOX.TEXTLOWER({ITEM_ID: "othtextlower", LABEL: "Text Lower", LENGTH: 10}),
  TEXTBOX.TEXTAREA({ITEM_ID: "othtextarea", LABEL: "Textarea", LENGTH: 10}),
]
```

Penjelasan property element textbox :

ITEM_ID	Property ITEM_ID harus terisi pada element textbox. Pastikan itemId terdaftar pada variabel winConfigForm, serta sesuai dengan itemId yang terdapat pada item class di file form PY. Contoh : <pre>ITEM_ID: "itemid"</pre>
LABEL	Property LABEL dapat diubah sesuai kebutuhan tampilan (hanya berpengaruh terhadap tampilan). Contoh : <pre>LABEL: "Header Normal"</pre>
LENGTH	Property LENGTH membatasi panjang karakter input user. Contoh : <pre>LENGTH : 8</pre>
IS_PRIMARY_KEY	Property IS_PRIMARY_KEY dengan value true ditambahkan jika element tersebut adalah primary key.

File PY (Folder path : application/)

Textbox (TEXTPRIMARY, TEXTNORMAL, TEXTDISPLAY, TEXTUPPER, TEXTLOWER, TEXTAREA)

```
class Coth_textnormal(TextPrimary):
    lengthItem = 10
    itemId = "itemId"
    databaseColumn = "oth_textnormal"
    databaseItem = True
    nullable = False
    primaryKey = True
    allowSpecialChar = []
    def validate(self):
        """
```

Fungsi ini digunakan untuk validasi column oth_textnormal

```

@return: dictionary dengan key status = nilai boolean (True = validasi
berhasil, False = validasi gagal)
key msg = nilai string
"""
# othtextnormal = self.item("othtextnormal")
# othtextdisplay = self.item("othtextdisplay")
# othtextupper = self.item("othtextupper")
# othtextlower = self.item("othtextlower")
# othtextarea = self.item("othtextarea")
return {"status": True, "msg": ""}

```

Penjelasan class item diatas :

```

class Coth_textnormal(TextPrimary)

```

1. TEXTPRIMARY

Parameter (**TextPrimary**) pada class item **PRIMARY** menyesuaikan jenis element yang ada pada javascript,

Contoh :

Jika element primary adalah **text normal** maka class python diubah menjadi :

```

class Coth_textnormal(TextNormal)

```

Jika element primary adalah **LOV** (List of View) maka class python diubah menjadi :

```

class Coth_textnormal(Lov)

```

Jika element primary adalah **Date** maka class python diubah menjadi :

```

class Coth_textnormal(Calendar)

```

2. TEXTNORMAL

Parameter (TextNormal) untuk element TEXTNORMAL.

```

class Coth_textnormal(TextNormal)

```

2. TEXTDISPLAY

Parameter (TextNormal) untuk element TEXTDISPLAY.

```

class Coth_textdisplay(TextNormal)

```

2. TEXTUPPER

Parameter (TextNormal) untuk element TEXTUPPER **diubah** menjadi (**TextUpper**).

```
class Coth_textupper(TextUpper)
```

2. TEXTLOWER

Parameter (TextNormal) untuk element TEXTLOWER **diubah** menjadi (**TextLower**).

```
class Coth_textupper(TextLower)
```

2. TEXTAREA

Parameter (TextNormal) untuk element TEXTAREA **diubah** menjadi (**TextArea**).

```
class Coth_textarea(TextArea)
```

Penjelasan atribut dari class item pada file PY :

lengthItem	Atribut lengthItem berfungsi untuk membatasi panjang karakter yang diterima oleh backend (Python). Value dari atribut lengthItem disesuaikan dengan property LENGTH yang ada pada bagian JS. <pre>lengthItem = 10</pre>
itemId	Atribut itemId memiliki value yang sama dengan ITEM_ID pada form JS. <pre>itemId = "itemId"</pre>
databaseColumn	Atribut databaseColumn harus menyesuaikan nama kolom pada database. <pre>databaseColumn = "oth_textnormal"</pre>
databaseItem	Atribut databaseItem dengan value True menyatakan bahwa item tersebut berada dalam database. <pre>databaseItem = True</pre>
nullable	Atribut nullable dengan value True menyatakan bahwa field dari class tersebut dapat di-save / di-update dengan keadaan kosong. Sebaliknya, value False menyatakan bahwa field dari class tersebut harus diisi dan akan memberikan validasi jika user menekan Save / Update tanpa mengisi field tersebut. <pre>nullable = False</pre>
primaryKey	Atribut primaryKey dengan value True menyatakan bahwa item tersebut adalah primary key. <pre>primaryKey = True</pre>
allowSpecialChar	Atribut allowSpecialChar memungkinkan user untuk mengisi field dengan special character yang di listkan pada atribut ini. Contoh : <pre>allowSpecialChar = ['.', '(', '+']</pre>

[KEMBALI KE TABLE LIST OF CONTENT](#)

2.2 How To Create Numeric in Oden Webix

Dokumentasi untuk membuat element numeric pada Oden Webix.

Numeric Normal

Numeric Normal

Currency

Currency

Float

Float

Pada Oden Webix, terdapat 3 jenis numeric element (**NUMERIC.NUMERICNORMAL**, **NUMERIC.CURRENCY**, dan **NUMERIC.FLOAT**).

NUMERIC element mewajibkan user mengisi field dengan format numeric (number).

Element **NUMERIC.CURRENCY** merubah angka numeric menjadi format mata uang Rupiah (Rp 0,00).

File JS (Folder path : static/application/)

1. **NUMERIC.NUMERICNORMAL**

Pada list element di javascript, buat sintaks berikut :

```
NUMERIC.NUMERICNORMAL({ITEM_ID: "itemidnumeric", LABEL: "Numeric Normal", LENGTH: 10}),
```

2. **NUMERIC.CURRENCY**

Pada list element di javascript, buat sintaks berikut :

```
NUMERIC.CURRENCY({ITEM_ID: "itemidcurrency", LABEL: "Currency", LENGTH: 10}),
```

3. **NUMERIC.FLOAT**

Pada list element di javascript, buat sintaks berikut :

```
NUMERIC.FLOAT({ITEM_ID: "itemidfloat", LABEL: "Float", LENGTH: 10}),
```

```
72 let elementODENTABLEHEAD = [  
73   TEXTBOX.TEXTPRIMARY({ITEM_ID: 'othtextnormal', LABEL: 'othtextnormal', LENGTH: 10}),  
74   NUMERIC.NUMERICNORMAL({ITEM_ID: "itemidnumeric", LABEL: "Numeric Normal", LENGTH: 10}),  
75   NUMERIC.CURRENCY({ITEM_ID: "itemidcurrency", LABEL: "Currency", LENGTH: 10}),  
76   NUMERIC.FLOAT({ITEM_ID: "itemidfloat", LABEL: "Float", LENGTH: 10}),  
77 ]
```

Penjelasan property element textbox :

ITEM_ID	Property ITEM_ID harus terisi pada element numeric. Pastikan itemId terdaftar pada variabel winConfigForm, serta sesuai dengan itemId pada item class di file form PY. Contoh : <pre>ITEM_ID: "itemidnumeric"</pre>
LABEL	Property LABEL dapat diubah sesuai kebutuhan tampilan (hanya berpengaruh terhadap tampilan). Contoh : <pre>LABEL: "Numeric Normal"</pre>
LENGTH	Property LENGTH membatasi panjang karakter input user. Contoh : <pre>LENGTH : 10</pre>
IS_PRIMARY_KEY	Property IS_PRIMARY_KEY dengan value true ditambahkan jika element tersebut adalah primary key.

File PY (Folder path : application/)

1. Numeric Normal

```
class Coth_numericnormal(NumericNormal):  
lengthItem = 10  
itemId = "othnumericnormal"  
databaseColumn = "oth_numericnormal"  
databaseItem = True  
nullable = True
```

```

primaryKey = False
allowSpecialChar = []
def validate(self):
    """
    Fungsi ini digunakan untuk validasi column oth_numericnormal
    @return: dictionary dengan key status = nilai boolean (True = validasi
    berhasil, False = validasi gagal)
    key msg = nilai string
    """
    return {"status": True, "msg": ""}

```

Pada Numeric Normal, parameter (TextNormal) diubah menjadi (**NumericNormal**)

2. Currency

```

class Coth_currency(Currency):
    lengthItem = 10
    itemId = "othcurrency"
    databaseColumn = "oth_currency"
    databaseItem = True
    nullable = True
    primaryKey = False
    allowSpecialChar = []
    def validate(self):
        """
        Fungsi ini digunakan untuk validasi column oth_currency
        @return: dictionary dengan key status = nilai boolean (True = validasi
        berhasil, False = validasi gagal)
        key msg = nilai string
        """
        return {"status": True, "msg": ""}

```

Pada Currency, parameter (TextNormal) diubah menjadi (**Currency**)

3. Float

```

class Coth_numericfloat(NumericNormal):
    lengthItem = 10
    itemId = "othnumericfloat"
    databaseColumn = "oth_numericfloat"
    databaseItem = True
    nullable = True

```

```
primaryKey = False
allowSpecialChar = ["."]
def validate(self):
    """
    Fungsi ini digunakan untuk validasi column oth_numericnormal
    @return: dictionary dengan key status = nilai boolean (True = validasi
    berhasil, False = validasi gagal)
    key msg = nilai string
    """
    return {"status": True, "msg": ""}
```

Pada Currency, parameter (TextNormal) diubah menjadi (**NumericNormal**)

Penjelasan atribut dari class item pada file PY :

Atribut **lengthItem** berfungsi untuk membatasi panjang karakter yang diterima oleh backend (Python). Value dari atribut lengthItem disesuaikan dengan property LENGTH yang ada pada bagian JS.

```
lengthItem = 10
```

Atribut **itemId** memiliki value yang sama dengan ITEM_ID pada form JS.

```
itemId = "itemId"
```

Atribut **databaseColumn** harus menyesuaikan nama kolom pada database.

```
databaseColumn = "oth_textnormal"
```

Atribut **databaseItem** dengan value **True** menyatakan bahwa item tersebut berada dalam database.

```
databaseItem = True
```

Atribut **nullable** dengan value **True** menyatakan bahwa field dari class tersebut dapat di-save / di-update dengan keadaan kosong. Sebaliknya, value **False** menyatakan bahwa field dari class tersebut harus diisi dan akan memberikan validasi jika user menekan Save / Update tanpa mengisi field tersebut.

```
nullable = False
```

Atribut **primaryKey** dengan value **True** menyatakan bahwa item tersebut adalah primary key.

```
primaryKey = True
```

Atribut **allowSpecialChar** tidak diubah pada element numeric.

Contoh :

```
allowSpecialChar = []
```

[KEMBALI KE TABLE LIST OF CONTENT](#)

2.2.1 How To Create Numeric Telephone

The screenshot shows a web application interface with a sidebar on the left containing a 'List Menu' and a main form area on the right. The form is titled 'FORM' and contains several input fields: 'NIM' (with value 'ADIT'), 'Nama' (with value 'Nama'), 'Hobby' (with value 'Hobby'), 'Jenis Kelamin' (with radio buttons for 'Laki-Laki' and 'Perempuan'), 'Email' (with value 'Email'), 'Asal' (with value 'Asal'), and 'Tempat Tinggal' (with value 'Tempat Tinggal'). The 'Nomor Telepon' field is highlighted with a red box and contains the value '6285163739313'. The form also has a toolbar with buttons for 'prev', 'next', 'search', 'add', 'update', 'delete', and 'reset'.

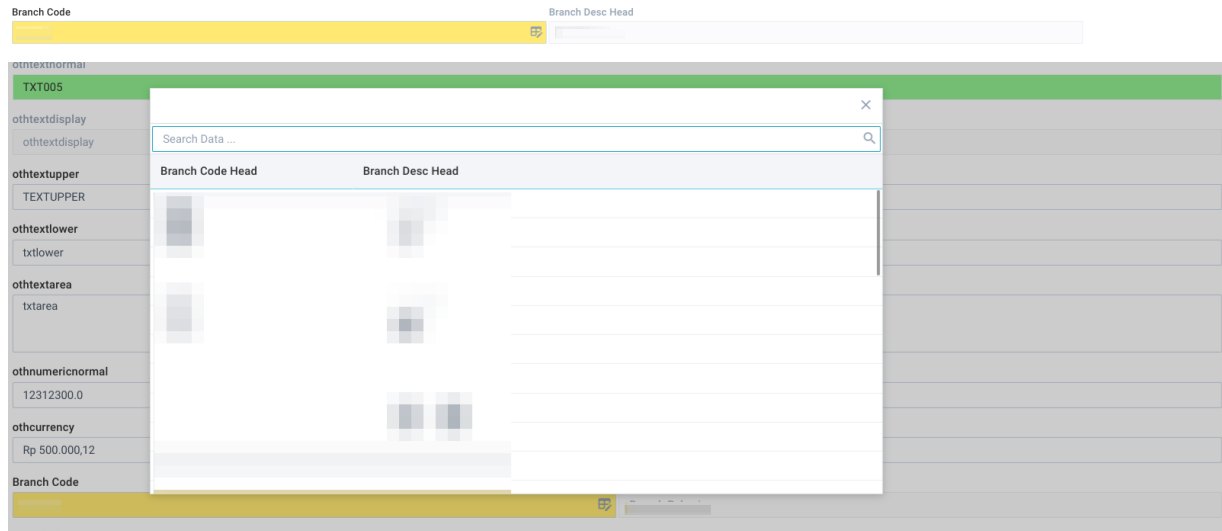
File.JS

```
NUMERIC.TELEPHONE({ITEM_ID: "othnumericnormal", LABEL: "Nomor Telepon"},  
{VALIDATE_ITEM: validateTelephone}),
```

buatlah code seperti contoh tersebut untuk menggunakan NUMERIC.TELEPHONE

2.3 How To Create LOV in Oden Webix

Tampilan Element LOV



FILE JS

Pada list element javascript, buat baris kode seperti di bawah ini:

```
LOVFIELD.TEXTLOVMULTICOLS({ITEM_ID:["item1","item2",item-N],LABEL:
["label1","label2",label-
N],WINDOW_ID:"wdLovOden",TOTAL_COLS:N,LENGTH:0,UPPER:true},
{VALIDATE_ITEM:validateLov})
```

- Sisipkan baris kode diatas pada list element javascript:

```
let element[BLOCKID] = [ LOVFIELD.TEXTLOVMULTICOLS({ITEM_ID:
["item1","item2",item-N],LABEL:["label1","label2",label-
N],WINDOW_ID:"wdLovOden",TOTAL_COLS:N,LENGTH:0,UPPER:true}),
]
```

Penjelasan properti pada element LOV:

ITEM_ID	Berisi list item id yang akan digunakan dalam pembuatan lov. User dapat mengatur banyaknya item sesuai dengan kebutuhan. Contoh : <pre>ITEM_ID: ["id1", "id2"]</pre>
LABEL	Berisi list label item yang digunakan user. Jumlah label dalam list bersesuaian dengan list pada item id. Gunakan huruf lowercase pada penulisan label item lov. Contoh : <pre>LABEL: ["label1", "label2"]</pre>
WINDO W_ID	Berisi id dari window popup yang akan muncul saat lov di trigger. Gunakan prefix wd sebelum menulis nama id window lov. Contoh : <pre>WINDOW_ID: "wdLovOden"</pre>
TOTAL_C OLS	Berisi jumlah kolom lov yang akan dibuat. Contoh : <pre>TOTAL_COLS: 2</pre>
LENGTH	Berisi panjang kolom (jumlah karakter) yang akan digunakan pada lov, attribute length pada lov hanya merujuk pada elemen di kolom pertama. Contoh: <pre>LENGTH: 10</pre>
UPPER	Property UPPER memungkinkan LOV untuk diinput ke format UPPERCASE / huruf kapital. Value dari key UPPER ada dua yaitu: <pre>UPPER: true UPPER: false</pre>

Pada fungsi renderLov, tambahkan baris kode seperti kode di bawah ini:

```
function renderLov() {
```



```

return [
lov.lovField("LovOden", ["othlivtextlovmulticols", "displayitemhead"],
["Display Code", "Display Desc"], 2),
]
}

```

- Parameter pertama, "**LovOden**" adalah id datatable yang akan digunakan pada lov. Samakan idnya dengan **WINDOW_ID** element namun tanpa prefix **wd**.
- Parameter kedua, **["id1", "id2"]** adalah id yang akan digunakan sebagai id coloumn datatble lov. Samakan dengan id pada **ITEM_ID** element
- Parameter ketiga, **["label1", "label2"]** adalah label yang akan digunakan sebagai label coloumn datatable lov. Samakan dengan label pada **LABEL** element
- Parameter keempat, **2** adalah jumlah kolom yang akan di generate pada datatable LOV

Pada webix ready function, tambahkan satu baris kode untuk memanggil API lov

```

var winConfigForm = {
  "NAMABLOCK": {
    .
    .
    .

    "LOV_API": {
      "othlivtextlovmulticols": {
        "ITEM_ID": "othlivtextlovmulticols",
        "TABLE_ID": "dtLovOden",
        "PARAM_ID": []
      }
    }

    .
    .
    .
  }

}

```

Panggil API lov pada fungsi webix ready, dan tempatkan sebelum pemanggilan **ApiSelect**

- Parameter pertama **"ITEM_ID"** memiliki value id dari element lov.
- Parameter kedua **TABLE_ID** memiliki value id dari window lov menggunakan prefix **dt** (**dtidtable**).

Note : Untuk fungsi validasi Lov pada file JS dapat dilihat pada catatan : [**How to Create Validation in Oden Webix**](#)

FILE PY

Ubah parameter pada class yang dipakai sebagai code lov (Lov)

```
class CnamaclassItem1(Lov):
    lengthItem = 10
    itemId = "item1"
    databaseColumn = "label1"
    databaseItem = True
```

Tambahkan satu class pada file PY yang akan digunakan untuk menampung data dari deskripsi lov.

```
class CnamaclassItem2(TextItem):
    itemId = "item2"
    databaseItem = False
```

Jumlah class yang ditambahkan bersesuaian dengan jumlah kolom deskripsi yang akan dibuat pada lov.

- Attribute `databaseItem` = False **bisa diganti True** jika kolom tersebut ingin digunakan pada transaksi insert/update

Masukan class yang sudah dibuat pada list object item block

```
vBlockOdentablehead.setListObj([..., CnamaclassItem2, ...])
```

Buat satu api yang digunakan untuk mengambil data lov

```
def lov_itemId_menuId():
    ajaxData = json.loads(request.args.get("data")) #wajib
    sqlData = """ select columnDB1 as lovDisplayCode, columnDB2 as
    lovDisplayDesc from schema.table where ( colomDB1 like %(search)s or
    colomDB2 like %(search)s ) offset %(offset)s limit 50
    """
    return selectDataHeaderQueryRO(sqlData, ajaxData)
```

- nama fungsi yang digunakan harus menggunakan prefix `lov_itemId_menuId`
- **itemId** : id dari element code LOV
- **menuId** : ID Menu

- Alias yang digunakan untuk get data pada sql statement adalah **LABEL** element lov dengan menggunakan prefix **lov (lovnamalabel)**.
- Pada LOV wajib menggunakan where clause untuk menjalankan fungsi searching
- **(colomDB1 like %(search)s or colomDB2 like %(search)s . . .) =>** menggunakan buka kurung dan tutup kurung untuk where clause searching.
- Jumlah kolom pada where clause harus sama dengan jumlah kolom yang di tampilkan pada window LOV atau sesuai dengan kebutuhan.
- **%(search)s** adalah parameter **default**.
- Pada SQL wajib menggunakan/menambahkan **offset %(offset)s => %(offset)s** sebagai default.
- Pada SQL wajib menggunakan/menambahkan **limit 50** atau kondisional pada query.

CATATAN PENTING:

Pada kolom lov kode ataupun display deskripsi dengan tipe data INTEGER, DATE Wajib melakukan casting pada query like pada where clause. MENJADI :

```
( cast(column_DB_Integer as varchar) like %(search)s or
column_DB_Not_Integer like %(search)s)
```

Tambahkan fungsi pada postSelect untuk menampilkan deskripsi dari lov saat pertama kali load data

```
def postSelect(self,data):
sql = """select columnDbDesc as itemiddesc from schema.table
where columnDbCode = %(itemidcode)s"""
param = ["itemidcode"]
setDesc = ["itemiddesc"]
data =
getDataDisplayItem({"DATA":data,"BLOCK_TYPE":self.blockType,"SQL":sql,"PARAM":param, "SET_DESC":"setDesc"})
return {"status":data["status"],"data":data["data"]}
```

NOTE :

Jika param atau deskripsi lebih dari 1, silahkan tambah dengan format list, contoh :

Terdapat sql dengan parameter lebih dari 1

```
sql = """select item_desc1 as itemiddesc1, item_desc2 as itemiddesc2
from schema.dbName where item_code1 = %(itemidparam1)s and item_code2= %(
itemidparam2)s"""
```

Maka pada variabel param ditambahkan satu itemid sesuai where clause, dengan format list.

```
param = ["itemidparam1","itemidparam2"]
```

Variabel setDesc juga ditambahkan sesuai kolom deskripsi yang diperlukan, jika 2 maka :

```
setDesc = ["itemiddesc1","itemiddesc2"]
```

KUSTOMISASI ELEMEN LOV

• Pembuatan Validasi LOV

File JS

Single Line Validation LOV

****penamaan id deskripsi LOV menggunakan lower case semua**

User pada umumnya menggunakan fungsi ini untuk mengambil deskripsi dari suatu Code yang diinput.

```
let validateotdlivtextlovmulticols = function(){ return  
validateLov({"LOV_CODE":"othlivtextlovmulticols","DISPLAY_ID":  
["displayitemhead"],"API_NAME":"validate..." +menuId, "PARAM_ID":  
["PARAM1","PARAM2"]}) }
```

Penjelasan code diatas :

- parameter pertama **"LOV_CODE"** memiliki value **"otdlivtextlovmulticols"**, value tersebut adalah id dari element LOV code.
- parameter kedua **"DISPLAY_ID"** memiliki value **["displayitemhead",...]**, value tersebut berbentuk list yang bisa diisi lebih dari 1 key yang akan dijadikan display item.
- parameter ketiga **"API_NAME"** memiliki value **"validate..." + menuId**, value tersebut disamakan dengan nama fungsi yang akan digunakan sebagai validate pada file python.
- parameter keempat **"PARAM_ID"** memiliki value berupa list **["PARAM1", ..]**, value tersebut merupakan parameter tambahan pada LOV

File PY

****penamaan id deskripsi LOV menggunakan lower case semua**

User pada umumnya menggunakan fungsi ini untuk mengambil deskripsi dari suatu Code yang diinput.

```
def validate..._MENU001(**kwargs):  
    ajaxData = json.loads(request.args.get("data"))  
    sqlDataDepartemen2 = """select ps_nama as lovDisplayDesc  
    from dmhr.ps_karyawan  
    where ps_status_aktif = 'Y'  
    and ps_nik = %(othlivtextlovmulticols)s """
```

```
return selectDataQueryRO(sqlDataDepartemen2, ajaxData)
```

Penjelasan code diatas :

- Penamaan Fungsi validate..._menulid
- ajaxData merupakan variable yang digunakan untuk mendapatkan data yang dikirimkan.
- sql merupakan bagian yang digunakan untuk menuliskan query validate yang dibutuhkan
- selectDataQueryRO(sql,ajaxData) merupakan fungsi default yg bisa digunakan user untuk menjalankan query dengan parameter

PENGUNAAN FUNGSI SELECT:

```
selectDataQueryRO(sqlDataDepartemen2, ajaxData)
```

- **Mengirim parameter id pada Lov**

Kode PrelItemLov

```
let preItemLov = function(){  
  restapi.restApiLov({"ITEM_ID":"othlov","TABLE_ID":"dtDisplayHeaderView",  
    "PARAM_ID":["PARAM1","PARAM2"]})  
  return true  
}
```

Note :

1. **Jangan lupa tambahkan {PRE_ITEM: preItemLov} pada element**
 2. **Hapus restapi.restApiLov pada webix.ready function**
- **Menambahkan parameter di LOV Datagrid**

Untuk menambahkan parameter dari BLOCK lain di WHERE clause fungsi LOV,
ex:

```
670  
671  ✓ sql = ""  
672  SELECT  
673  |   column_det1  
674  FROM  
675  |   tabel_det  
676  WHERE  
677  |   ( ps_nik like %(search)s or ps_nama like %(search)s )  
678  |   column_det2 = %(column_head2)s  
679  offset %(offset)s  
680  limit 50  
681  ""
```

Pindahkan "restapi.restApiLov" dari webix.ready ke winRestApi, ex:

```
var winRestApi = function(){
restapi.restApiParam({"BLOCK_NAME":"ODEN_DET","PARAM_ID":["column_det1",
"column_det2", "column_det3"]}));
restapi.restApiLov({"ITEM_ID":"othlov","TABLE_ID":"dtLovOden",
"PARAM_ID":["column_head2"]}));}
```

UNTUK MEMBUAT LOV LOADING DATA SAAT ICON LOV DI KLIK, SILAHKAN BACA DOKUMENTASI BERIKUT

5.20 How to Configure Load Data at Start for LOV

- **How to CUSTOM SET LOV**

Custom set lov dibutuhkan jika lov pada element lebih sedikit daripada kolom pada WINDOW LOV.

Contoh :

Lihat gambar dibawah ini, pada bagian yang di kotak dan arrow merah, Lov disajikan dalam bentuk 2 column, namun pada saat membuka window lov, user menampilkan column sebanyak 6 kolom.

Untuk melakukan set data dengan klik data pada window lov ke item element yang lebih sedikit dari window, tambahkan hal berikut.

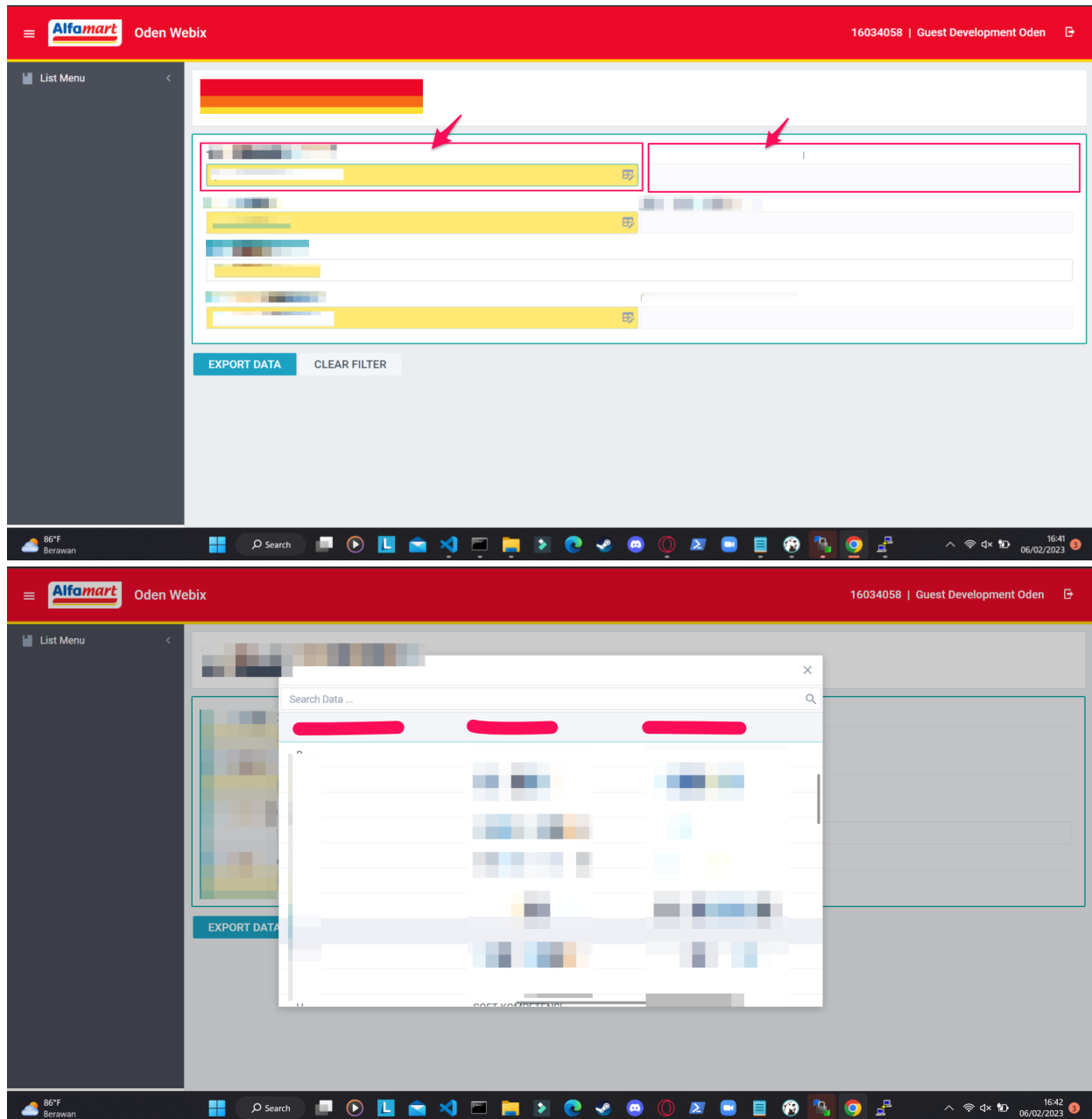
Contoh:

```
lov.lovField("TrainingProgramLOV",["itemId1", "itemId2"], ['Label1',
'Label2', 'Label3', 'Label4', 'Label5', 'Label6'],6,
{"itemId1":"lovlabel1","itemId2":"lovlabel2"}),
```

NOTE!! :

penulisan value dari custom set lov (lov+label) harus ditulis dalam huruf kecil (lowercase).

Untuk contoh di atas, window lov berisi 6 kolom dan label, namun saat set data lov dari window mengisi 2 item pada element.



KEMBALI KE TABLE LIST OF CONTENT

2.4 How To Create Radio Button in ODEN Webix

Dokumentasi untuk membuat element Radio Button pada Oden Webix

INI RADIO BUTTON

☐ Option 1 ☐ Option 2

FILE JS

Lokasi File = "**{LOKASI_PROJECT}**\static\application"

Pada File Javascript, ubah sintaks kode dari

```
TEXTBOX.TEXTNORMAL({  
  ITEM_ID: "othradiobutton",  
  LABEL: "oth_radiobutton",  
  LENGTH: 1,  
}),
```

menjadi seperti ini:

```
RADIO.RADIOBUTTON({  
  ITEM_ID: "othradiobutton",  
  LABEL: "INI RADIO BUTTON",  
  OPTIONS: [  
    { id: 1, value: "Option 1" },  
    { id: 2, value: "Option 2" },  
  ],  
  IS_PRIMARY_KEY: true,  
}),
```

Penjelasan Properti pada element Radio Button sebagai berikut:

ITEM_ID	Berisi item id yang akan digunakan dalam pembuatan RadioButton dengan tipe data String. Pastikan ITEM_ID harus ada dalam array ELEMENT pada variable winConfigForm <pre> 3 var winConfigForm = {'ODENTABLEHEAD': 4 {'ELEMENT': [5 'othtextnormal', 6 'othtextdisplay', 7 'othtextupper', 8 'othtextlower', 9 'othtextarea', 10 'othnumericnormal', 11 'othcurrency', 12 'othlivtextlovmulticols', 13 'othradiobutton', </pre> Contoh penggunaan: <pre>ITEM_ID: "othradiobutton"</pre>
LABEL	Berisi label yang akan digunakan dalam pembuatan RadioButton dengan tipe String. Label ini akan muncul diatas options Contoh: <pre>LABEL: "INI RADIO BUTTON"</pre>
OPTIONS	Berisi list options yang akan digunakan dalam pembuatan RadioButton dengan tipe data Objek yang berisi key and value. Untuk key harus ada 2 yaitu id dan value. id digunakan untuk inisialisasi masing masing radio buttonnya dan value digunakan untuk label / text yang muncul di setiap radio buttonnya. Contoh: <pre> { id: 1, value: "Option 1" }, { id: 2, value: "Option 2" }, </pre>

FILE PY

Lokasi Folder= "{**LOKASI_PROJECT**}\application"

Pada file Python , tampilan kode adalah sebagai berikut:

```
class Coth_radiobutton(ComboList):
    lengthItem = 1
    itemId = "othradiobutton"
    databaseColumn = "oth_radiobutton"
    databaseItem = True
    nullable = True
    primaryKey = False
    def validate(self):
        """
        Fungsi ini digunakan untuk validasi column oth_radiobutton
        @return: dictionary dengan key status = nilai boolean (True = validasi
        berhasil, False = validasi gagal)
        key msg = nilai string
        """
        return {"status": True, "msg": ""}
```

Penjelasan properti pada element Radio Button

Parameter base (Base.**ComboList**) untuk element RADIOBUTTON.

```
class Coth_radiobutton(ComboList):
```

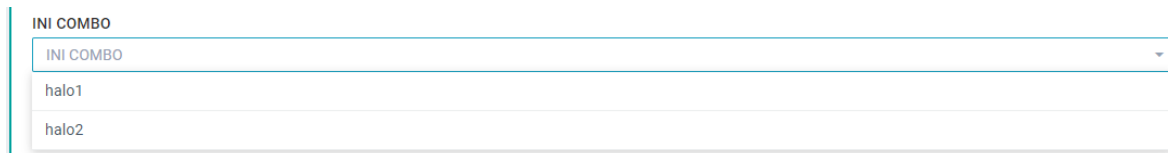
lengthItem	Berisi panjang karakter yang dapat diterima oleh sistem dengan tipe data Integer. contoh: <pre>lengthItem = 1</pre>
itemId	Berisi id item yang harus sama dengan ITEM_ID yang ada pada file JS dengan tipe data String Contoh: FILE JS <pre>ITEM_ID: "othradiobutton"</pre> FILE PY <pre>itemId = "othradiobutton"</pre>
databaseColumn	Berisi nama kolom pada database dengan tipe data String Contoh: <pre>databaseColumn = "oth_radiobutton"</pre>
databaseItem	Berisi ketersediaan kolom pada database dengan tipe Boolean (True / False). True menyatakan bahwa kolom tersebut tersedia pada database <pre>databaseItem = True</pre>
nullable	Berisi apakah data tersebut boleh kosong atau tidak. True menyatakan bahwa data tersebut dapat bernilai kosong / null <pre>nullable = True</pre>
primaryKey	Berisi apakah data tersebut merupakan primary key atau tidak. True menyatakan bahwa item tersebut adalah primary key. <pre>primaryKey = True</pre>

CATATAN : Pada datagrid, tidak ada radio button. Gunakan combo list default.

[KEMBALI KE LIST OF CONTENT](#)

2.5 How To Create Combo List in ODEN Webix

Dokumentasi pembuatan element Combo List pada Oden Webix



Dalam Oden, terdapat 3 jenis Combo List yaitu sebagai berikut

- *Combo List Default*
- *Combo List API*
- *Combo List API Data*

FILE JS

Lokasi File = "{**LOKASI_PROJECT**}\static\application"

Combo List Default

Ubah kode berikut pada file JS

```
TEXTBOX.TEXTNORMAL({  
  ITEM_ID: "othcombolistdefault",  
  LABEL: "oth_combolistdefault",  
  LENGTH: 10,  
}),
```

Menjadi seperti ini

```
COMBO.COMBOLISTDEFAULT({  
  ITEM_ID: "othcombolistdefault",  
  LABEL: "INI COMBO",  
  OPTIONS: [  
    { id: 1, value: "halo1" },  
    { id: 2, value: "halo2" },  
  ],  
  IS_PRIMARY_KEY: true  
}),
```

Penjelasan properti pada element Combo List Default

ITEM_ID	Berisi item id yang akan digunakan dalam pembuatan Combo List Default dengan tipe data String. Pastikan ITEM_ID harus ada dalam array ELEMENT pada variable winConfigForm <pre> 3 var winConfigForm = {'ODENTABLEHEAD': 4 {'ELEMENT': [5 'othtextnormal', 6 'othtextdisplay', 7 'othtextupper', 8 'othtextlower', 9 'othtextarea', 10 'othnumericnormal', 11 'othcurrency', 12 'othlivtexttlovmulticols', 13 'othradiobutton', 14 'othcombolistdefault', </pre> Contoh penggunaan: <pre>ITEM_ID: "othcombolistdefault"</pre>
LABEL	Berisi label yang akan digunakan dalam pembuatan ComboListDefault dengan tipe String. Contoh penggunaan: <pre>LABEL: "othcombolistdefault"</pre>
OPTIONS	Berisi list options yang akan digunakan dalam pembuatan ComboListDefault dengan tipe data Objek yang berisi key and value. Untuk key harus ada 2 yaitu id dan value. id digunakan untuk inisialisasi masing masing radio buttonnya dan value digunakan untuk label / text yang muncul di setiap radio buttonnya. Contoh penggunaan: <pre> { id: 1, value: "halo1" }, { id: 2, value: "halo2" }, </pre>
IS_PRI-	

MARY_ KEY	Berisi nilai primary key yang akan digunakan dalam pembuatan ComboList dengan tipe data Boolean (true / false). Contoh penggunaan: <pre>IS_PRIMARY_KEY: true,</pre>
----------------------	---

Combo List API

Ubah kode berikut pada file JS

```
TEXTBOX.TEXTNORMAL({
  ITEM_ID: "othcombolistapi",
  LABEL: "oth_combolistapi",
  LENGTH: 10,
}),
```

Menjadi seperti ini

```
COMBO.COMBOLISTAPI({
  ITEM_ID: "othcombolistapi",
  LABEL: "othcombolistapi",
  IDPARAM: "othcombolistapidata",
}),
```

Penjelasan properti element Combo List API

ITEM_ID	Berisi item id yang akan digunakan dalam pembuatan Combo List API dengan tipe data String. Pastikan ITEM_ID harus ada dalam array ELEMENT pada variable winConfigForm <pre> 3 var winConfigForm = {'ODENTABLEHEAD': 4 {'ELEMENT': [5 'othtextnormal', 6 'othtextdisplay', 7 'othtextupper', 8 'othtextlower', 9 'othtextarea', 10 'othnumericnormal', 11 'othcurrency', 12 'othlivtextlovmulticols', 13 'othradiobutton', 14 'othcombolistdefault', 15 'othcombolistapi', </pre> Contoh penggunaan: ITEM_ID: "othcombolistapi"
LABEL	Berisi label yang akan digunakan dalam pembuatan Combo List API dengan tipe String. Contoh penggunaan: LABEL: "othcombolistapi"
IDPARAM	Berisi Parameter ID yang digunakan untuk menampung data yang didapat dari API. Contoh penggunaan: IDPARAM: "othcombolistapidata",

Pada webix ready function tambahkan satu baris kode yang terletak sebelum restApiSelect untuk memanggil API Combo List

```

webix.ready(function () {
// Rest API Combo List Form dan Data Grid

```



```
restapi.restApiCombo({"ITEM_ID":"othcombolistapi","API_NAME":"combolist_
"+menuId, 'PARAM_ID':['othcombolistapi']})
restapi.restApiSelect()
});
```

Penjelasan Parameter:

- **Parameter pertama** yaitu ("othcombolistapi") merupakan **ITEM_ID** dari **COMBOLISTAPI**. samakan dengan **ITEM_ID** pada **COMBOLISTAPI**.
- **Parameter Kedua** yaitu ("API_NAME":"combolist_"+menuId) merupakan **Nama Fungsi API pada Python**. Samakan urlnya dengan fungsi yang akan dibuat pada **file Python**

Combo List API Data

Ubah kode berikut pada file JS

```
TEXTBOX.TEXTNORMAL({
ITEM_ID: "othcombolistapidata",
LABEL: "oth_combolistapidata",
LENGTH: 10,
}),
```

Menjadi seperti ini

```
COMBO.COMBOLISTAPIDATA(
{ ITEM_ID: "othcombolistapidata", LABEL: "othcombolistapidata" },
{ PRE_ITEM: preItemCombo }
),
```

Penjelasan properti element Combo List API Data

ITEM_ID	Berisi item id yang akan digunakan dalam pembuatan Combo List API Data dengan tipe data String. Pastikan ITEM_ID harus ada dalam array ELEMENT pada variable winConfigForm <pre> 3 var winConfigForm = {'ODENTABLEHEAD': 4 {'ELEMENT': [5 'othtextnormal', 6 'othtextdisplay', 7 'othtextupper', 8 'othtextlower', 9 'othtextarea', 10 'othnumericnormal', 11 'othcurrency', 12 'othlivtextlovmulticols', 13 'othradiobutton', 14 'othcombolistdefault', 15 'othcombolistapi', 16 'othcombolistapidata', 17 'othnormalcheckbox' </pre> Contoh penggunaan: ITEM_ID: "othcombolistapidata"
LABEL	Berisi label yang akan digunakan dalam pembuatan Combo List API dengan tipe String. Contoh penggunaan: LABEL: "othcombolistapidata"
PRE_ITEM	Berisi nama fungsi pemanggilan data dari COMBOLISTAPI dengan tipe data String. Contoh penggunaan: <pre>{ PRE_ITEM: preItemCombo }</pre>

Contoh fungsi **preItemCombo** yang didapat dari properti **PRE_ITEM**:

```

let preItemCombo = function(){
let code = $$("othcombolistapi").getValue();
if(code == ""){

```

```

RMSGBOX("Pilih Branch Terlebih Dahulu",GO_ITEM("othcombolistapi"))
}else{
restapi.restApiDataCombo({"ITEM_ID":"othcombolistapi","API_NAME":"combol
istdata_"+menuId, 'PARAM_ID':['othcombolistapi':code]})
}
}

```

Penjelasan fungsi **preItemCombo**

RMSGBOX	Merupakan fungsi untuk memunculkan popup box dan memiliki 2 parameter yaitu: • Parameter Pertama berisi pesan yang ingin ditampilkan pada popup box • Parameter Kedua berisi fungsi untuk mengarah focus dengan parameter ID input
----------------	--

Parameter base (**ComboList**) untuk element **ComboList** .

```
class Coth_combolistdefault(ComboList):
```

Penjelasan properti fungsi **preItemCombo**:

branchCode	Berisi Kode Cabang yang didapat dari value dari id Combo List API.
-------------------	--

Penjelasan Fungsi **preItemCombo**:

- branchCode mendapatkan data dari value dari **othcombolistapi**
- Jika branchCode bernilai kosong, maka akan memunculkan popup dari fungsi **RMSGBOX**
- Jika branchCode terdapat nilai, maka akan memanggil restapi.restApiDataCombo dengan 3 parameter yaitu:
 - **Parameter pertama** yaitu ID **OTHCOMBOLISTAPI DATA**. Samakan dengan **ITEM_ID** pada **Combo List API Data**
 - **Parameter kedua** yaitu API_NAME berisi **url api combo list data**.
 - **Parameter ketiga** yaitu berisi PARAM_ID berisi Variabel object dengan key **othcombolistapi** dan value nya **Kode Cabang / Branch Code**.

FILE PY

COMBO LIST DEFAULT , COMBO LIST API DAN COMBO LIST API DATA

Tampilan kode dalam file PY element **Combo List Default , Combo List API dan Combo List API Data** adalah sebagai berikut:

```
class Coth_combolistdefault(ComboList):
    lengthItem = 10
    itemId = "othcombolistdefault"
    databaseColumn = "oth_combolistdefault"
    databaseItem = True
    nullable = True
    primaryKey = False
    allowSpecialChar = ['.', '(', '+']
    def validate(self):
        """
        Fungsi ini digunakan untuk validasi column oth_combolistdefault
        @return: dictionary dengan key status = nilai boolean (True = validasi
        berhasil, False = validasi gagal)
        key msg = nilai string
        """
        return {"status": True, "msg": ""}
```

Penjelasan properti dalam element Combo List Default PY

lengthItem	Berisi panjang karakter yang dapat diterima oleh sistem dengan tipe data Integer. contoh: <pre>lengthItem = 1</pre>
itemId	Berisi id item yang harus sama dengan ITEM_ID yang ada pada file JS dengan tipe data String Contoh: FILE JS <pre>ITEM_ID: "othcombolistdefault"</pre> FILE PY <pre>itemId = "othcombolistdefault"</pre>
databaseColumn	Berisi nama kolom pada database dengan tipe data String Contoh: <pre>databaseColumn = "oth_combolistdefault"</pre>
databaseItem	Berisi ketersediaan kolom pada database dengan tipe Boolean (True / False). True menyatakan bahwa kolom tersebut tersedia pada database contoh: <pre>databaseItem = True</pre>
nullable	Berisi apakah data tersebut boleh kosong atau tidak. True menyatakan bahwa data tersebut dapat bernilai kosong / null. contoh: <pre>nullable = True</pre>
primaryKey	Berisi apakah data tersebut merupakan primary key atau tidak. True menyatakan bahwa item tersebut adalah primary key. contoh: <pre>primaryKey = False</pre>

allows pecialC har	Berisi beberapa special karakter yang dapat di terima user dalam bentuk List. Contoh : <pre>allowSpecialChar = ['.', '(', '+']</pre>
-----------------------------------	--

COMBO LIST API

Tambahkan kode berikut pada bagian file PY untuk pengambilan data Combo List dari API

```
def combolist_MENUUID(**kwargs):
    sqlBranch = """
    select mbr_code as id, mbr_desc as value
    from dmhr.ms_branch
    where mbr_status_aktif = 'Y' and mbr_msc_code = '01' """
    result = selectHeaderQueryRO(sqlBranch)
    return result
```

Penjelasan Kode

```
def combolist_MENUUID(**kwargs):
```

- **Nama File** merupakan value dari API_NAME pada File JS

File JS :

```
restapi.restApiCombo({"ITEM_ID":"othcombolistapidata","API_NAME":"combolist_"+menuId, 'PARAM_ID':['othcombolistapi']})
```

- **Parameter pertama**, dengan key **ITEM_ID** berisi **ITEM_ID** dari element ComboListData
- **Parameter kedua**, dengan key **API_NAME** berisi **NAMA FUNGSI API** pada file Python
- **Parameter ketiga**, dengan key **PARAM_ID** berisi **LIST ITEM ID** dari element *Combo List API*

```
def combolist_MENUUID(**kwargs):
    sqlBranch = """
    select mbr_code as id, mbr_desc as value
    from dmhr.ms_branch
    where mbr_status_aktif = 'Y' and mbr_msc_code = '01' """
    result = selectHeaderQueryRO(sqlBranch)
    return result
```

Pada fungsi ini terdapat 2 variable yaitu

sqlBranch	berisi <i>select statement</i> untuk mendapatkan data dari database
result	memanggil fungsi selectHeaderQueryRO pada library utilities dengan parameter yaitu sqlBranch

Nilai kembalian dari fungsi ini adalah **result**

COMBO LIST API DATA

Tambahkan kode berikut pada bagian file PY untuk pengambilan data Combo List Data dari API

```
def combolistdata_FORM(**kwargs):  
    ajaxData = json.loads(request.args.get('data'))  
    sqlToko = """select mst_kd_store as id , mst_nama_store as value from  
dmhr.ms_stores ms where mst_status_aktif = 'Y' and mst_kd_branch = %  
(othcombolistapi)s """  
    return selectDataHeaderQueryRO(sqlToko,ajaxData)
```

Penjelasan Kode

```
def combolistdata_MENUID(**kwargs):
```

- **Nama File** merupakan value dari API_NAME pada File JS

File JS :

```
restapi.restApiDataCombo({"ITEM_ID":"othcombolistapidata","API_NAME":"co  
mbolistdata_"+menuId, 'PARAM_ID':['othcombolistapi']})
```

```
def combolistdata_FORM(**kwargs):  
    ajaxData = json.loads(request.args.get('data'))  
    sqlToko = """select mst_kd_store as id , mst_nama_store as value from  
dmhr.ms_stores ms where mst_status_aktif = 'Y' and mst_kd_branch = %  
(othcombolistapi)s """  
    return selectDataHeaderQueryRO(sqlToko,ajaxData)
```

Pada fungsi ini terdapat 2 variable yaitu

ajaxData	berisi request yang dapat di terima oleh server
sqlToko	berisi <i>select statement</i> untuk mendapatkan data dari database

Nilai kembalian dari fungsi ini adalah **selectDataHeaderQueryRO(sqlToko,ajaxData)**

KEMBALI KE TABLE LIST OF CONTENT

2.6 How To Create Checkbox in Oden Webix

Dokumentasi pembuatan element Checkbox pada Oden Webix

Normal Checkbox



oth_normalcheckboxinteger



FILE JS

Menampilkan Elemen Checkbox pada Form:

Pada file Javascript form, ganti TEXTBOX.TEXTNORMAL menjadi CHECKBOX.NORMALCHECKBOX pada item yang ingin digunakan sebagai checkbox.

Sintaks untuk membuat checkbox field adalah :

```
CHECKBOX.NORMALCHECKBOX({ITEM_ID:"itemID", LABEL:"itemLabel"})
```

Isikan LABEL sesuai dengan kebutuhan, Contoh :

```
CHECKBOX.NORMALCHECKBOX({ITEM_ID: "otdnormalcheckbox", LABEL: "Normal  
Checkbox"})
```


Maka LABEL pada form akan berubah menjadi 'Normal Checkbox' seperti berikut.

Contoh :

```
19
20 let elementODENTABLEHEAD = [
21   TEXTBOX.TEXTPRIMARY({ITEM_ID: "othtextnormal", LABEL: "oth_textnormal", LENGTH: 10}),
22   CHECKBOX.NORMALCHECKBOX({ITEM_ID: "new_checkbox_id", LABEL: "Normal Checkbox"}),
23   CHECKBOX.NORMALCHECKBOX({ITEM_ID: "othnormalcheckboxinteger", LABEL: "oth_normalcheckboxinteger", CHECK_VALUE: 'T'}),
24 ]
25
```

Mengganti Check Value dari Elemen Checkbox :

Secara default, elemen checkbox pada form mengembalikan check value 'Y' atau uncheck value 'N' . Namun, check dan uncheck value juga dapat berupa 'T' atau 'F', maupun berupa 0 atau 1. Untuk mengganti check value, tentukan property CHECK_VALUE menjadi 1 atau 'T'.

****NOTE****

Elemen Checkbox tidak dapat mengembalikan nilai **NULL**

- **Mengubah Check Value menjadi 1 atau 0 :**

Untuk mengubah check value menjadi 0 atau 1, tambahkan attribute "CHECK_VALUE" pada parameter fungsi dengan value 1 seperti berikut :

```
CHECKBOX.NORMALCHECKBOX({ITEM_ID: "othnormalcheckboxinteger", LABEL:
"oth_normalcheckboxinteger", CHECK_VALUE: 1})
```

- **Mengubah Check Value menjadi 'T' atau 'F' :**

Untuk mengubah check value menjadi 'T' atau 'F', tambahkan attribute "CHECK_VALUE" pada parameter fungsi dengan value 'T' seperti berikut :

```
CHECKBOX.NORMALCHECKBOX({ITEM_ID: "othnormalcheckboxinteger", LABEL:
"oth_normalcheckboxinteger", CHECK_VALUE: 'T'})
```

Penjelasan Attribute Element Checkbox :

ITEM_ID	Property ITEM_ID harus ada dan terisi pada property element textbox. Pastikan itemId terdaftar pada variabel winConfigForm, serta sesuai dengan itemId yang terdapat pada item class di file form PY. Contoh : <pre>ITEM_ID: "itemId"</pre>
LABEL	Property LABEL dapat diubah sesuai kebutuhan tampilan (hanya berpengaruh terhadap tampilan). Contoh : <pre>LABEL: "Normal Checkbox"</pre>
CHECK_VALUE	Property CHECK_VALUE menentukan nilai ketika checkbox pada kondisi checked atau unchecked. Secara default, elemen checkbox pada form mengembalikan check value 'Y' atau uncheck value 'N'. Check value juga dapat berupa 'T' / 'N' atau 0 / 1. Contoh : <pre>CHECK_VALUE: 'T'</pre>

FILE PY

Lokasi Folder= "{**LOKASI_PROJECT**}\application"

Pada file Python , tampilan kode adalah sebagai berikut:

```
class Coth_normalcheckbox(CheckBox):
    lengthItem = 1
    itemId = "othnormalcheckbox"
    databaseColumn = "oth_normalcheckbox"
    databaseItem = True
    nullable = True
    primaryKey = False
    allowSpecialChar = []
    def validate(self):
        """
```

Fungsi ini digunakan untuk validasi column oth_normalcheckbox

```
@return: dictionary dengan key status = nilai boolean (True = validasi
berhasil, False = validasi gagal)
key msg = nilai string
"""
return {"status": True, "msg": ""}
```

Parameter untuk element Checkbox.

```
class Coth_normalcheckbox(CheckBox):
```

Penjelasan atribut pada element class Checkbox form py :

lengthItem	Atribut lengthItem berfungsi untuk membatasi panjang karakter yang diterima oleh backend (Python). Value dari atribut lengthItem disesuaikan dengan property LENGTH yang ada pada bagian JS. <pre>lengthItem = 1</pre>
itemId	Atribut itemId memiliki value yang sama dengan ITEM_ID pada form JS. <pre>itemId = "itemId"</pre>
databaseColumn	Atribut databaseColumn harus menyesuaikan nama kolom pada database. <pre>databaseColumn = "oth_normalcheckbox"</pre>
databaseItem	Atribut databaseItem dengan value True menyatakan bahwa item tersebut merupakan item yang berada dalam database. <pre>databaseItem = True</pre>
nullable	Atribut nullable dengan value True menyatakan bahwa field dari class tersebut dapat di-save / di-update dengan keadaan kosong. Sebaliknya, value False menyatakan bahwa field dari class tersebut harus diisi dan akan memberikan validasi jika usser melakukan Save / Update. <pre>nullable = False</pre>
primaryKey	Atribut primaryKey dengan value True menyatakan bahwa item tersebut adalah primary key dalam tabel. <pre>primaryKey = True</pre>
allowSpecialChar	Atribut allowSpecialChar memungkinkan user untuk mengisi field dengan special character yang di listkan pada atribut ini. Contoh : <pre>allowSpecialChar = ['.', '(', '+']</pre>

[KEMBALI KE TABLE LIST OF CONTENT](#)

2.7 How To Create Date in Oden Webix

Dokumentasi pembuatan element Date pada Oden Webix

Date-Month-Year

16-02-2023



Month-Year

02-2023



Year

2023



Clock

15:20



Timestamp

16-02-2023 15:36:00



FILE JS

Menampilkan Elemen Date pada Form:

Sintaks untuk membuat **date** field dengan format **DD-MM-YYYY** :

```
DATE.DDMMYY({ITEM_ID: "othddmmyy", LABEL: "oth_ddmmyy"})
```

Ketika menggunakan format date DD-MM-YYYY, maka ketika user melakukan input baik melalui keyboard atau ikon kalender, format penulisan yang diterima akan berbentuk DD-MM-YYYY. **Contoh:** 10-10-2010

Sintaks untuk membuat **date** field dengan format **MM-YYYY** :

```
DATE.DDMMYY({ITEM_ID: "othmmyy", LABEL: "oth_mmyy"})
```

Ketika menggunakan format date MM-YYYY, maka ketika user melakukan input baik melalui keyboard atau ikon kalender, format penulisan yang diterima akan berbentuk MM-YYYY.

Contoh: 10-2010

Sintaks untuk membuat **date** field dengan format **YYYY** :

```
DATE.YY({ITEM_ID: "othyy", LABEL: "oth_yy"})
```

Ketika menggunakan format date DD-MM-YYYY, maka ketika user melakukan input baik melalui keyboard atau ikon kalender, format penulisan yang diterima akan berbentuk YYYY.

Contoh: 2010

Sintaks untuk membuat **date (time)** field dengan format **CLOCK** :

```
DATE.CLOCK({ITEM_ID: "othclock", LABEL: "Clock", ADD_SECOND:false})
```

Ketika menggunakan format date CLOCK, maka ketika user melakukan input baik melalui keyboard atau ikon jam, format penulisan yang diterima akan berbentuk HH:mm. Atribut ADD_SECOND:false akan menampilkan format jam HH:mm sedangkan untuk value true, akan menampilkan format jam HH:mm:ss

Contoh: 20:10 . Dengan ADD_SECOND:true, 20:10:00

Sintaks untuk membuat **datetime** field dengan format **TIMESTAMP** :

```
DATE.TIMESTAMP({ITEM_ID: "othtimestamp", LABEL: "Timestamp"})
```

Ketika menggunakan format date TIMESTAMP, maka ketika user melakukan input baik melalui keyboard atau ikon jam, format penulisan yang diterima akan berbentuk DD-MM-YYYY HH:mm:ss.

Contoh: 06-02-2023 00:00:00

Isikan LABEL sesuai dengan kebutuhan, Contoh :

```
DATE.DDMYY({ITEM_ID: "othddmmy", LABEL: "Date-Month-Year"})
```

Maka LABEL pada form akan berubah menjadi 'Date-Month-Year' seperti berikut.

Contoh :

```
19
20 let elementODENTABLEHEAD = [
21   TEXTBOX.TEXTPRIMARY({ITEM_ID: "othtextnormal", LABEL: "oth_textnormal", LENGTH: 10}),
22   DATE.DDMYY({ITEM_ID: "othddmmy", LABEL: "Date-Month-Year"}),
23   DATE.MMY({ITEM_ID: "othmmy", LABEL: "oth_mmy"}),
24   DATE.YY({ITEM_ID: "othyy", LABEL: "oth_yy"}),
25 ]
```

Penjelasan Property Element Date :

ITEM_ID	Property ITEM_ID harus terisi pada element textbox. Pastikan itemId terdaftar pada variabel winConfigForm, serta sesuai dengan itemId yang terdapat pada item class di file form PY. Contoh : <pre>ITEM_ID: "othddmmyy"</pre>
LABEL	Property LABEL dapat diubah sesuai kebutuhan tampilan (hanya berpengaruh terhadap tampilan). Contoh : <pre>LABEL: "Date-Month-Year"</pre>
LENGTH	Property LENGTH membatasi panjang karakter input user. Contoh : <pre>LENGTH : 20</pre>
IS_PRIMARY_KEY	Property IS_PRIMARY_KEY dengan value true ditambahkan jika element tersebut adalah primary key.
DISPLAY_ITEM	Property DISPLAY_ITEM dengan value true menjadikan element terdisable

Penjelasan Property Element Date.CLOCK :

ITEM_ID	Property ITEM_ID harus terisi pada element textbox. Pastikan itemId terdaftar pada variabel winConfigForm, serta sesuai dengan itemId yang terdapat pada item class di file form PY. Contoh : <pre>ITEM_ID: "othddmmyy"</pre>
LABEL	Property LABEL dapat diubah sesuai kebutuhan tampilan (hanya berpengaruh terhadap tampilan). Contoh : <pre>LABEL: "Date-Month-Year"</pre>
LENGTH	Property LENGTH membatasi panjang karakter input user. Contoh : <pre>LENGTH : 4</pre>
ADD_SECOND	Property ADD_SECOND dengan value true ditambahkan jika element jam ingin ditampilkan hingga value detik.
DATE_ID	Property DATE_ID:"itemid" dengan value itemId yang adalah tanggal digunakan untuk menghubungkan Tanggal dan jam dari 2 item yang berbeda. note: atribut date_id hanya untuk form. Datagrid belum ada.
DISPLAY_ITEM	Property DISPLAY_ITEM dengan value true menjadikan element terdisable

FILE PY

Lokasi Folder= "{**LOKASI_PROJECT**}\application\form"

Pada file Python , tampilan kode adalah sebagai berikut:

- **Format DD-MM-YYYY**

```
class Coth_ddmmyy(Calendar):
lengthItem = 20
```



```

databaseColumn = "oth_ddmmyy"
databaseItem = True
nullable = True
primaryKey = False
allowSpecialChar = []
def validate(self):
    """
    Fungsi ini digunakan untuk validasi column oth_ddmmyy
    @return: dictionary dengan key status = nilai boolean (True = validasi
    berhasil, False = validasi gagal)
    key msg = nilai string
    """
    return {"status": True, "msg": ""}

```

- **Format MM-YYYY**

```

class Coth_mmyy(Calendar):
    lengthItem = 20
    itemId = "othmmyy"
    databaseColumn = "oth_mmyy"
    databaseItem = True
    nullable = True
    primaryKey = False
    allowSpecialChar = []
    def validate(self):
        """
        Fungsi ini digunakan untuk validasi column oth_mmyy
        @return: dictionary dengan key status = nilai boolean (True = validasi
        berhasil, False = validasi gagal)
        key msg = nilai string
        """
        return {"status": True, "msg": ""}

```

- **Format YYYY**

```

class Coth_yy(Calendar):
    lengthItem = 20
    itemId = "othy"
    databaseColumn = "oth_yy"
    databaseItem = True

```

```

nullable = True
primaryKey = False
allowSpecialChar = []
def validate(self):
    """
    Fungsi ini digunakan untuk validasi column oth_yy
    @return: dictionary dengan key status = nilai boolean (True = validasi
    berhasil, False = validasi gagal)
    key msg = nilai string
    """
    return {"status": True, "msg": ""}

```

- **Format CLOCK**

```

class Coth_clock(Time):
    lengthItem = 20
    itemId = "othclock"
    databaseColumn = "oth_clock"
    databaseItem = True
    nullable = True
    primaryKey = False
    allowSpecialChar = []
    def validate(self):
        """
        Fungsi ini digunakan untuk validasi column oth_yy
        @return: dictionary dengan key status = nilai boolean (True = validasi
        berhasil, False = validasi gagal)
        key msg = nilai string
        """
        return {"status": True, "msg": ""}

```

- **Format Timestamp**

```

class Coth_timestamp(Time):
    lengthItem = 20
    itemId = "othtimestamp"
    databaseColumn = "oth_timestamp"
    databaseItem = True
    nullable = True
    primaryKey = False

```

```

allowSpecialChar = []
def validate(self):
    """
    Fungsi ini digunakan untuk validasi column oth_yy
    @return: dictionary dengan key status = nilai boolean (True = validasi
    berhasil, False = validasi gagal)
    key msg = nilai string
    """
    return {"status": True, "msg": ""}

```

Penjelasan properti pada element class Date :

Parameter untuk element **Date (DD-MM-YYYY, MM-YYYY, YYYY)** .

```
class Coth_ddmmy(Calendar):
```

Parameter untuk element **CLOCK dan Timestamp**

```
class Coth_clock(Time):
```

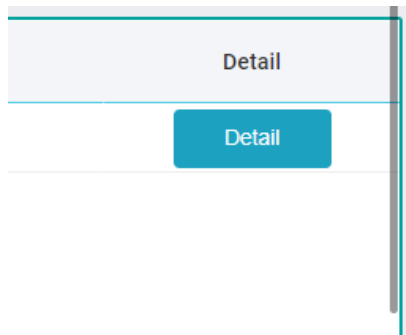
Penjelasan atribut pada element class Date form py :

lengthItem	Atribut lengthItem berfungsi untuk membatasi panjang karakter yang diterima oleh backend (Python). Value dari atribut lengthItem disesuaikan dengan property LENGTH yang ada pada bagian JS. <pre>lengthItem = 20</pre>
itemId	Atribut itemId memiliki value yang sama dengan ITEM_ID pada form JS. <pre>itemId = "othddmmyy"</pre>
databaseColumn	Atribut databaseColumn harus menyesuaikan nama kolom pada tabel database. <pre>databaseColumn = "oth_ddmmyy"</pre>
databaseItem	Atribut databaseItem dengan value True menyatakan bahwa item tersebut merupakan item yang berada dalam database. <pre>databaseItem = True</pre>
nullable	Atribut nullable dengan value True menyatakan bahwa field dari class tersebut dapat di-save / di-update dengan keadaan kosong. Sebaliknya, value False menyatakan bahwa field dari class tersebut harus diisi dan akan memberikan validasi jika usser melakukan Save / Update. <pre>nullable = False</pre>
primaryKey	Atribut primaryKey dengan value True menyatakan bahwa item tersebut adalah primary key dalam tabel. <pre>primaryKey = True</pre>
allowSpecialChar	Atribut allowSpecialChar memungkinkan user untuk mengisi field dengan special character yang di listkan pada atribut ini. Contoh : <pre>allowSpecialChar = ['.', '(', '+']</pre>

[KEMBALI KE TABLE LIST OF CONTENT](#)

2.8 How To Create Button in Oden Webix

Dokumentasi pembuatan element Button di Oden Webix



FILE JS (Folder path : static/application/)

Pada element di file JS, buat sintaks berikut:

```
{cols: [{}, BUTTON.BUTTONFORM({ ITEM_ID: "itemId", LABEL: "Label"},  
{CUSTOM_CLICK:customBtnTrigger})]}
```

Jika berada pada blok datagrid :

```
BUTTON.BUTTONDATAGRID({ ITEM_ID: "itemId", LABEL: "Label" },  
{CUSTOM_CLICK:customBtnTrigger}),
```

Penjelasan Property Element Button:

ITEM_ID	Property ITEM_ID, adalah itemId untuk element button, dapat diisi sesuai kebutuhan. ITEM_ID: "btnDetail"
LABEL	Property LABEL dapat diubah sesuai kebutuhan tampilan, akan berpengaruh pada tampilan button. Contoh : LABEL: "Detail"
CUSTOM_CLICK	Property CUSTOM_CLICK berisi function custom yang dibuat user

Pada element button, tambahkan parameter dictionary dengan key **CUSTOM_CLICK** untuk mengarahkan button ke fungsi trigger yang dibutuhkan. Contoh :

```
BUTTON.BUTTONFORM({ ITEM_ID: "itemId", LABEL: "Label" }, { CUSTOM_CLICK: customTrigger } ),
```

Selanjutnya, pada file js, diatas variabel element block, tambahkan variabel berisi fungsi sesuai value **CUSTOM_CLICK** diatas:

```
let customTrigger = function () {  
  //Put Your Button Logic Here  
}
```

Fungsi tersebut dapat diisi sesuai fungsi button yang diinginkan oleh user.

Contoh Function CUSTOM_CLICK:

```
let customBtnTriggerDtg = function(){  
  let data = getBlockValue("ODENTABLEDET");  
  restapi.restApiCustom({ ITEM_ID:"custom123123", DATA:data})  
}
```

getBlockValue	Fungsi ini digunakan untuk mengambil value pada block
restapi.restApiCustom	Fungsi ini digunakan untuk hit api custom yang ada pada python, return dari api nya harus berupa dictionary dengan 3 key yaitu {'status':, 'msg':, 'data':} Parameter fungsi ini ialah ITEM_ID (id button) , DATA(data yang digunakan di api custom)

Pembuatan API pada python:

```
def ITEMID_MENUID():
    ajaxData = json.loads(request.args.get("data"))

    return {'status':True,'data':{},'msg':''}
```

2.9 How to Create Uploader File in Oden Webix

File Uploader adalah elemen yang memungkinkan user untuk mengupload file dari local komputer ke dalam sistem Oden Webix

Gambaran FORM

FORM

prev

next

search

add

update

delete

reset

oth_normalcheckboxinteger

☐

oth_ddmmyy

oth_ddmmyy

* Date only (DD-MM-YYYY)

oth_mmyy

oth_mmyy

* Date only (DD-MM-YYYY)

oth_yy

oth_yy

* Date only (DD-MM-YYYY)

UPLOADER DEFAULT

GUI Validasi Oden_25 Sept 2023.pdf

✓

×

UPLOAD FILE

DATAGRID

SAMPLE 2

prev

next

search

save

add row

update

delete

reset

othcombolistapi

Multiview 1

Multiview 2

otdtextupper

UPLOADED FILE

Delete Row

0 files -

FILES

otdtextnormal

DET001

UPLOADER DEFAULT

UPLOAD FILE

Page 1 from 1

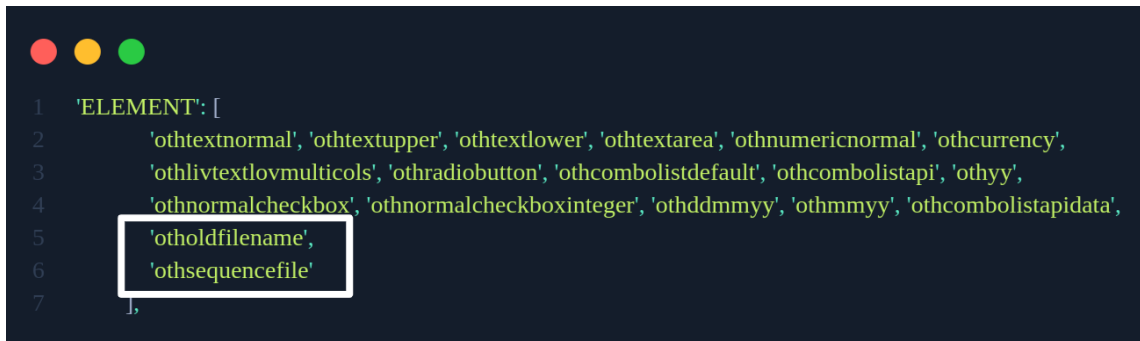
- Showing 1 from 1

Langkah Pembuatan BUCKET

- Untuk struktur folder pada bucket, contohnya seperti dibawah ini



FILE JS



1. Tambahkan 2 Element diatas untuk menjadi element dari tombol dan sequence

otholdfilename	Element yang nantinya akan menjadi oldFileName dari File yang akan diupload nantinya *nama elemen ini disesuaikan dengan nama tabel pada database
othsequencefile	Element yang akan menjadi nama baru dari file yang diupload, nama baru nantinya akan disesuaikan dengan sequence dari file file yang sudah ada. *nama elemen ini disesuaikan dengan nama tabel pada database

2. Kemudian Tambahkan Element Button di list element pada block tersebut

- Pada FORM gunakan code dibawah ini

```
BUTTON.BUTTONUPLOADERFORM({  
  ITEM_ID: "btnUploader",  
  LABEL: "UPLOAD FILE",  
  SECTION: "Uploader Default",  
  COLS_FILE: {  
    "oldFileName": "otholdfilename",  
    "newFileName": "othsequencefile"  
  },  
  MULTIPLE_FILES: true,
```

```
ALLOW_FILE: ["pdf", "png", "php"],
MAX_SIZE: 1500,
REQUIRED: true
}},
```

- Pada DATAGRID gunakan code dibawah ini

```
BUTTON.BUTTONUPLOADERDATAGRID({
ITEM_ID: "btnUploader",
LABEL: "UPLOAD FILE",
SECTION: "Uploader Default",
COLS_FILE: {
"oldFileName": "otholdfilename",
"newFileName": "othsequencefile"
},
MULTIPLE_FILES: true,
ALLOW_FILE: ["pdf", "png", "php"],
MAX_SIZE: 1500,
REQUIRED: true
}},
]
```

ITEM_ID	Property ITEM_ID, adalah itemId untuk element button, dapat diisi sesuai kebutuhan. ITEM_ID: "btnUploader"
LABEL	Property LABEL dapat diubah sesuai kebutuhan tampilan, akan berpengaruh pada tampilan button. Contoh : LABEL: "Uploader File"
SECTION	Property SECTION dapat diubah sesuai kebutuhan tampilan, akan berpengaruh pada tampilan section untuk mengetahui batasan element uploader. SECTION: "Uploader Default"
COLS_FILE	• oldFileName Tempat tampungan untuk menyimpan nama file lama yang adalah otholdfilename dari nama file yang diinputkan user. • newFileName Tempat tampungan untuk menyimpan nama file baru yang akan di simpan nantinya, isi dari newFileName adalah othsequencefile yang nantinya akan diolah lagi untuk menentukan nama baru dari file yang diinputkan user serta sequence dari file tersebut. <i>Contoh : 15-GUI Validasi Oden.pdf,</i> <i>15 adalah hasil sequence</i>
MULTIPLE_FILES	Nilai dari MULTIPLE_FILES ini berisi boolean True atau False, dimana nantinya akan mempengaruhi untuk penginputan lebih dari 1 file
ALLOW_FILE	

ALLOW_FILE adalah jenis ekstensi file yang akan diterima sesuai kebutuhan, pada Oden Webix terdapat beberapa ekstensi yang dapat digunakan antara lain <pre>ALLOW_FILE: ["pdf", "xlsx", "docx", "pptx", "xls", "doc", "ppt", "jpg", "jpeg", "gif", "png", "rar", "zip", "3gp", "mkv"],</pre>	
MAX_SIZE	MAX_SIZE adalah ukuran maximum dari file yang akan diupload, MAX_SIZE disini dalam bentuk kb, <i>contohnya : 1500</i>
REQUIRED	Nilai dari REQUIRED adalah boolean True atau False, jika True maka harus ada file yang diupload saat mode Insert, sedangkan jika False, tidak perlu ada file yang di upload.

FILE PY

1. Pada folder globalVar tambahkan `DEFAULT_BUCKET = "example"`, disesuaikan dengan nama di bucket
2. Buat Objek Element pada python dengan isi TextArea, serta jangan lupa tambahkan di `execVcount`
Contohnya :

```
class Coth_item_id(TextArea):
```

```
lengthItem = 200
itemId = "item_id"
databaseColumn = "nama_database"
databaseItem = True
nullable = True
primaryKey = False
allowSpecialChar = [",", ".", "&", "-", "/", "(", ")", "|"]
```

2. Pada preInsert & preUpdate (jika Form tersebut bisa insert / update), tambahkan code dibawah ini :

```
result = preTransactionFileData(self, "mode",
{"columnIdSequence": "namaElemenSequence", "bucketName": "sat-oden-upload",
"tempFolderName": "tempFileOden/", "folderName": "menuId/",
"sqlSequence": sqlSequence})
```

Fungsi preTransacationFileData memiliki 3 parameter

Parameter 2	Berisi mode apa yang akan digunakan, contohnya : "insert", "update"
Parameter 3	Dictionary yang isinya sebagai berikut : - columnIdSequence : berisi nama elemen sequence, contohnya "othsequencefile" - bucketName : berisi tempat penampungan dari file yang di upload - tempFolderName : berisi tempat penampungan sementara file - folderName : berisi menuId/, contohnya FORM/ - sqlSequence : merupakan urutan file, berisi biasanya query sql untuk mengambil nilai dari database dan dijadikan sequence dari file tersebut, contohnya : <pre>sqlSequence = ""select nama_kolom as sequence from nama_tabel ORDER BY random() LIMIT 1;""</pre>

2.9.1 How to Create Uploader File in Oden Webix NonBase

File Uploader adalah elemen yang memungkinkan user untuk mengupload file dari local komputer ke dalam sistem Oden Webix

Gambaran

FORM

The screenshot displays the Oden Webix application interface. The top navigation bar shows the application name '[applicationName]' and the user '2209116 | Guest Development Oden'. The main content area is titled 'Copy Struktural Code' and contains the following elements:

- A dropdown menu for 'othcode' with the value 'othcode' selected.
- A text input field for 'lovcodeacuan' with the value 'lovcodeacuan' entered.
- A text input field for 'Deskripsi Acuan'.
- A button labeled 'Choose File' highlighted with a red rectangle.
- A button labeled 'Posting' highlighted with a red rectangle.
- A button labeled 'Detail Upload'.
- A section titled 'Format Upload File' with a sub-label '1. FORMAT FILE (*.XLSX)' and a button labeled 'Contoh file upload copy so'.

Langkah Pembuatan

FILE JS

```
var winConfigForm = {  
  'COPYSTRUKTURALCODE': {  
    'ELEMENT': [  
      'kodeacuan',  
      'kodeacuan',  
      'kodetujuan',  
      'oldFileName',  
      'newFileName',  
      'btnposting',
```

```

],
'PRIMARY_KEY': [],
'RELATION_KEY': [],
'WHEN_NEW_FORM': {},
'BLOCK_TYPE': ['FORM', 'NONE'],
'BASE_TABLE': false,
}
}

```

oldfilename	Element yang nantinya akan menjadi oldFileName dari File yang akan diupload nantinya <i>*nama elemen ini disesuaikan dengan nama tabel pada database</i>
newfilename	Element yang akan menjadi nama baru dari file yang diupload, nama baru nantinya akan disesuaikan dengan sequence dari file file yang sudah ada. <i>*nama elemen ini disesuaikan dengan nama tabel pada database</i>
btnPosting	Element button yang akan digunakan sebagai trigger

```

LOVFIELD.TEXTLOVMULTICOLS({ITEM_ID: ["kodeacuan", "kodeacuandesc"],
LABEL: ["lovcodeacuan", "Deskripsi Acuan"], LENGTH: 15,UPPER:true,
WINDOW_ID:"wdLovBM", TOTAL_COLS:2,},{PRE_ITEM: preItemLov}),
{
cols:[
BUTTON.BUTTONUPLOADERFORM({
ITEM_ID: "kodetujuan",
LABEL:"Choose File",
SECTION:"Code Tujuan",
MULTIPLE_FILES:false,
ALLOW_FILE:["xlsx"],
MAX_SIZE:100,

```

```
REQUIRED:true
)),{}
]],
{view:"template", template:"", type:"spacer"},
{
cols:[BUTTON.BUTTONFORM({ITEM_ID:"btnposting", LABEL:"Posting"},
{CUSTOM_CLICK:triggerBtn})),{},
BUTTON.BUTTONFORM({ITEM_ID:"callform", LABEL:"Detail Upload"},
{CUSTOM_CLICK:triggerCallForm})
]],
```


ITEM_ID	Property ITEM_ID, adalah itemId untuk element button, dapat diisi sesuai kebutuhan. <pre>ITEM_ID: "kodetujuan"</pre>
LABEL	Property LABEL dapat diubah sesuai kebutuhan tampilan, akan berpengaruh pada tampilan button. Contoh : <pre>LABEL: "Choose File"</pre>
SECTION	Property SECTION dapat diubah sesuai kebutuhan tampilan, akan berpengaruh pada tampilan section untuk mengetahui batasan element uploader. <pre>SECTION: "Code Tujuan"</pre>
MULTIPLE_FILES	Nilai dari MULTIPLE_FILES ini berisi boolean True atau False, dimana nantinya akan mempengaruhi untuk penginputan lebih dari 1 file
ALLOW_FILE	ALLOW_FILE adalah jenis ekstensi file yang akan diterima sesuai kebutuhan, pada Oden Webix terdapat beberapa ekstensi yang dapat digunakan antara lain <pre>ALLOW_FILE: ["pdf", "xlsx", "docx", "pptx", "xls", "doc", "ppt", "jpg", "jpeg", "gif", "png", "rar", "zip", "3gp", "mkv"],</pre>
MAX_SIZE	MAX_SIZE adalah ukuran maximum dari file yang akan diupload, MAX_SIZE disini dalam bentuk kb, <i>contohnya : 100</i>
REQUIRED	Nilai dari REQUIRED adalah boolean True atau False, jika True maka harus ada file yang diupload saat mode Insert, sedangkan jika False, tidak perlu ada file yang di upload.

```

let triggerBtn = function(){
if(Object.keys($$("#kodetujuan").files.data.pull).length === 0){
RMSGBOX("Please Choose File First")
return false
}
insertFileCopySc =
restapi.restApiCustom({"ITEM_ID":"insertFile","DATA":getBlockValue("COPY
STRUKTURALCODE")})
if(!insertFileCopySc.status){
RMSGBOX(insertFileCopySc.msg)
return false
}
MSGBOX("Berhasil memasukkan file, file akan diproses dalam waktu 30
menit.")
return true
}

```

Langkah Pembuatan

FILE PY

restApiCustom

```

def insertFile_+MenuId ():
ajaxData = json.loads(request.args.get("data")) #wajib
cekNullable = nonBaseBlockValidate("COPYSTRUKTURALCODE")
if(cekNullable["status"] == False):
return {"status":False,"data":{},"msg":cekNullable["msg"]}
sqlSeq = """select nextval('table')
"""

sequence = selectQuery(sqlSeq)
if(sequence["status"] == False or sequence["data"][0] == []):
return {"status":False,"data":{},"msg":"Error get sequence in database"}
session["vForm"].actionActive = "NON-BASE"
objBlock = session["vForm"].getBlock("COPYSTRUKTURALCODE")
objBlock.item("sequence").value = ajaxData["newFileName"]
cstfid = sequence["data"][0][0]
creator = session['loggedId']['nik']
ajaxData.update({"cstfid":cstfid,"creator":creator})

```

```

sqlInsert = """insert into table (column)
values %(column)s)
"""

executeInsert = executeDataQuery(sqlInsert, ajaxData)
if(executeInsert["status"] == False):
return {"status":False,"data":{},"msg":executeInsert["msg"]}
namaSequence = str(cstfid) + '-' + ajaxData["oldFileName"].split('.')[0]
result = preTransactionFileData(objBlock,"insert",
{"columnIdSequence":"sequence","bucketName":"sat-oden-upload",
"tempFolderName":"tempFileOden/","folderName":"UH_OD_01/",
"sequenceName":namaSequence})

if(result["status"] == False):

sqlDelete = """delete from table where column = %(column)s"""

executeDataQuery(sqlDelete,ajaxData)
return {"status":False,"data":{},"msg":"Upload File Gagal"}

return {"status":True,"data":{"cstfid":cstfid},"msg":""}

```

Query Sequence	Berisi sequence data
Query Insert	Digunakan untuk insert data ke column yang sesuai berdasarkan table. <pre> sqlInsert = """insert into table (column) values %(column)s) """ </pre>
preTransactionFileData	Digunakan untuk cek apakah status bernilai true, jika false maka data yang di insert bakal dihapus kembali

3. How to Create Validation in Oden Webix

Validasi pada Oden Framework terdiri dari 2 macam, yaitu validasi pada tampilan depan (**Javascript**) dan validasi pada program belakang (**Python**)

Validasi Javascript

Validasi Item pada Form

```
let validateothtextarea = function(){

let value = $$("othtextarea").getItemValue()

if(value!=""){
//berisi logic untuk melakukan pengolahan data
return {"status":true}
}else{
return{"status":false, "tipe":"error", "pesan":"validate gagal"}
}
}
```

User dapat membuat 1 variabel let yang sesuai dengan item yang akan ditambahkan validasi, sebagai contoh adalah **validateothtextarea**. Variabel tersebut berisi sebuah fungsi yang dapat disesuaikan dengan kebutuhan item yang dituju.

Fungsi Get Item Value pada Tampilan Form dan Datagrid

```
let value = $$("othtextarea").getItemValue()
```

User menggunakan cara seperti contoh diatas untuk mengambil value dari sebuah item yang berada di **Form**. **\$\$('othtextarea')** adalah cara untuk mengambil object dengan id **'othtextarea'** dan menggunakan fungsi **.getValue()** untuk mendapatkan value dari object yang dituju.

Validasi Item pada Datagrid

```
let validateotdtextarea = function(){

let value = getItemValue("otdtextarea")

if(value!=""){
//berisi logic untuk melakukan pengolahan data
return {"status":true}
}else{
return{"status":false, "tipe":"error", "pesan":"validate gagal"}
}
}
```

User dapat membuat 1 variabel let yang sesuai dengan item yang akan ditambahkan validasi, sebagai contoh adalah **validateotdtextarea**. Variabel tersebut berisi sebuah fungsi yang dapat disesuaikan dengan kebutuhan item yang dituju.

Basic Custom Logic Validasi

```
if(value!=""){
//berisi logic untuk melakukan pengolahan data

return {"status":true}
}else{
return{"status":false, "tipe":"error", "pesan":"validate gagal"}
}
```

User dapat menambahkan logic validasi yang sesuai dengan item yang dituju. Variabel **value** pada contoh diatas mengacu dengan pengambilan value pada id object yang sudah dilakukan.

Default Return True pada Fungsi Validasi

```
return {"status":true}
```

Apabila pada validasi memasuki logic yang **benar**, maka user **harus** memberikan return object *dictionary* yang berisi:

- "status" : **true**

Default Return False pada Fungsi Validasi

```
return {"status":false, "tipe":"error", "pesan":"validate gagal"}
```

Apabila pada validasi memasuki logic yang **salah**, maka user **harus** memberikan return object *dictionary* yang berisi:

- 'status' : **false**
- 'tipe' : 'error' / 'warning' / 'info' (**dapat memilih salah satu**)
- 'pesan' : " (**disesuaikan dengan validasi yang dibuat**)

Fungsi Get Item Value pada Semua Tampilan (Form/Datagrid)

```
let valueIdItem = getItemValue("idItem")
```

User dapat menggunakan fungsi **getItemValue('idParam')** untuk mengambil value **baik dari tipe tamplan form maupun datagrid**. Parameter **'idItem'** digunakan untuk mengambil spesifik value dari pada tampilan form maupun datagrid. User dapat mengambil data dari block manapun sesuai kebutuhan.

Fungsi Set Value pada Form

```
$$("itemId").setValue("value")
```

Fungsi Set Value pada Datagrid

```
setValueDt(["idKolom1", "idKolom2", "idKolom3"],  
["valueId1", "valueId2", "valueId3"])
```

Single Line Validation Not Null Value

```
let validateotdtextlower = function(){  
// let valueIdItem = getItemValue("idItem") => get Value Item pada tipe  
Form / tipe Datagrid
```

```

let validasiNotNull = notNullValidate("otdtextlower") => return
{"status":true/false, "tipe":"error"/"warning"/"info", pesan:"id tidak
boleh kosong"}
if(validasiNotNull.status){
//logic tambahan setelah validasi not null
return {"status":true}
}
return validasiNotNull
}

```

User dapat menggunakan single line validation untuk pengecekan not null memakai fungsi **notNullValidate('idItem')**. Parameter **'idItem'** digunakan untuk pengecekan secara spesifik pada id yang dituju. Apabila User **ingin menambahkan logic lain setelah pengecekan not null**, User dapat menambahkan pada area yang sudah disediakan pada **//logic tambahan setelah validasi not null**.

Single Line Validation LOV

****penamaan id deskripsi LOV menggunakan lower case semua**

User pada umumnya menggunakan fungsi ini untuk mengambil deskripsi dari suatu Code yang diinput.

```

let validateotdlivtextlovmulticols = function(){
// let valueIdItem = getItemValue("idItem") => get Value Item pada tipe
Form / tipe Datagrid
let validasiLOV =
validateLov({"LOV_CODE":"othlivtextlovmulticols","DISPLAY_ID":
["displayitemhead"],"API_NAME":"validate..." + menuId})
if(validasiLOV.status){
//logic tambahan setelah validasi not null
return {"status":true}
}
return validasiLOV
}

```

Penjelasan code diatas :

- parameter pertama **"LOV_CODE"** memiliki value **"otdlivtextlovmulticols"**, value tersebut adalah id dari element LOV code.
- parameter kedua **"DISPLAY_ID"** memiliki value **["displayitemhead",...]**, value tersebut berbentuk list yang bisa diisi lebih dari 1 key yang akan dijadikan display item.

- parameter ketiga **"API_NAME"** memiliki value **"validate..." + menuId**, value tersebut disamakan dengan nama fungsi yang akan digunakan sebagai validate pada file python.

Penempatan Validasi ke Element Item yang Dituju

Setelah pembuatan fungsi validate, User perlu menambahkan parameter untuk memasukkan fungsi pada item yang dituju.

```
TEXTBOX.TEXTAREA({ITEM_ID: "othtextarea", LABEL: "othtextarea", LENGTH: 10},{VALIDATE_ITEM:validateothtextarea})
```

User harus menambahkan parameter **VALIDATE_ITEM** pada dictionary **kedua** saat pembuatan element. Penempatan sesuai dengan contoh diatas

Validasi Record pada Form dan Datagrid :

```
let rulesODENTABLEHEAD = {
  "othtextnormal":webix.rules.isNotEmpty,
}
```

Pada hasil generate file, terdapat let **rules+namaBlock** yang digunakan untuk menempatkan fungsi validate record. Pada contoh diatas, user menambahkan fungsi

webix.rules.isNotEmpty pada element dengan id **"othtextnormal"**. Fungsi tersebut berarti bahwa element dengan id **"othtextnormal"** tidak boleh kosong.

Terdapat beberapa **fungsi validate Webix** yang bisa digunakan User, antara lain:

isChecked	"id_element" : webix.rules.isChecked	Fungsi yang akan memvalidasi bahwa tipe checkbox sudah ter-check
isEmail	"id_element" : webix.rules.isEmail	Fungsi yang akan memvalidasi tipe inputan string sudah berformat email
isEmpty	"id_element" : webix.rules.isEmpty	Fungsi yang akan memvalidasi inputan tidak boleh kosong
isNumber	"id_element" : webix.rules.isNumber	Fungsi yang akan memvalidasi inputan hanya terdiri dari angka

Cara penggunaan fungsi validate Webix

```
let rulesBlockA = {
  "text1":webix.rules.isNotEmpty,
```



```

"text2":webix.rules.isNumber,
"text3":webix.rules.isEmail,
"text4":webix.rules.isChecked
}

```

Custom Validate Record

User dapat menambahkan **custom validate** pada rules

```

let rulesBlockA = {
  "text1": function(value){
    //logic
    if(value=="B"){
      return false
    }
    return true
  }
}

```

Pada custom validate, User dapat memberikan logic function seperti contoh diatas. User **harus** memberi **return value** berupa **true / false** sehingga validasi dapat berjalan.

Validasi Python

Validasi Item

```

class Coth_textnormal(TextNormal):
    def validate(self):
        """
        Fungsi ini digunakan untuk validasi column oth_textnormal
        @return: dictionary dengan key status = nilai boolean (True = validasi
        berhasil, False = validasi gagal)
        key msg = nilai string
        """

        # othtextnormal = self.item("othtextnormal")
        # othtextdisplay = self.item("othtextdisplay")
        # othtextupper = self.item("othtextupper")

```

```
# othtextlower = self.item("othtextlower")
othtextarea = self.item("othtextarea")
valueArea = othtextarea.value
//logic validate
if valueArea == "kosong":
return rMsgBox("error text area")
return {"status":True, "msg": ""}
```

User dapat menambahkan fungsi validasi pada tempat yang sudah disediakan setelah generate file python. Pada **def validate(self)**, User dapat menambahkan logic validasi dan dapat mengambil object data lain sesuai dengan id yang dimau.

Validasi Record

```
class CRecordOdentablehead(Record):
def validate(self):
"""
Fungsi ini digunakan untuk validasi record
@return: dictionary dengan key status = nilai boolean (True = validasi
berhasil, False = validasi gagal)
key msg = nilai string
"""

//logic validate
# othtextnormal = self.item("othtextnormal")
# othtextdisplay = self.item("othtextdisplay")
# othtextupper = self.item("othtextupper")
# othtextlower = self.item("othtextlower")
othtextarea = self.item("othtextarea")
valueArea = othtextarea.value
if valueArea == "kosong":
return rMsgBox("error text area")
return {"status":True, "msg": ""}
```

User dapat menambahkan fungsi validasi pada tempat yang sudah disediakan setelah generate file python. Pada **def validate(self)**, User dapat menambahkan logic validasi dan dapat mengambil object data lain sesuai dengan id yang dimau.

Fungsi Select

Pada form dan datagrid yang menggunakan select, fungsi select pada return seperti berikut.
selectDataQueryRO

```
return selectDataQueryRO(sqlDataDepartemen2, ajaxData)
```

Fungsi Get Item Value

```
othtextarea = self.item("othtextarea")  
valueArea = othtextarea.value
```

User dapat menggunakan fungsi **self.item("id")** untuk mengambil object sesuai id pada 1 record yang sama. Apabila ingin menggunakan value id tersebut maka User perlu menggunakan tambahan **.value** untuk bisa mengolah/menggunakan data value tersebut pada logic validate.

Default Return False pada Validasi

```
return rMsgBox("error text area")
```

Apabila logic validasi bernilai false atau salah, maka User **harus** me-return **rMsgBox("pesan custom sesuai User")**. Fungsi ini akan menghentikan proses validasi dan akan mengembalikan nilai false. Parameter pada fungsi **rMsgBox** adalah pesan yang akan tampil pada alert UI.

Default Return True pada Validasi

```
return {"status":True, "msg": ""}
```

Apabila logic validasi bernilai true atau benar, maka User **harus** me-return dictionary sesuai dengan contoh diatas.

3.1 How To Create Custom Block Validation in Oden Webix

Dokumentasi untuk membuat validasi block custom pada Oden Webix.

Validasi block custom digunakan untuk memvalidasi item, record, dan memasukkan ajaxData ke dalam block object.

Pada penjelasan di bawah, kasus validasi custom block ini dikhususkan saat developer membutuhkan **set data insert** tanpa commit, dan melakukan validasi Item, Record dalam Object.

FILE JS

Pada file JS, buatlah sintaks dibawah ini :

```
let validateItem = function(){
  dataHead = getBlockValue("BLOCK_NAME_HEAD")
  dataDet = getBlockValue("BLOCK_NAME_DET")
  blockValidate = restapi.restApiCustom({"DATA":{"BLOCK_NAME_DET":
  {"blockType":"datagrid","data":dataDet},
  "BLOCK_NAME_HEAD":{"blockType":"form","data":
  [dataHead]}}, "ITEM_ID":"itemId"})
  if(blockValidate.status){
    //YOUR LOGIC HERE
  }else{
    return {"status":false, "tipe":"error", "pesan":blockValidate.msg}
  }
}
```

FILE PY

Pada file Py, buat sambungan dari API custom di atas:

```
def itemId_MENU_ID(**kwargs):
  ajaxData = json.loads(request.args.get('data'))
  #YOUR LOGIC HERE
  return {"status":True, "data":[{}], "msg":""}
```

Pada bagian #YOUR LOGIC HERE, developer dapat menggunakan fungsi - fungsi berikut ini untuk melakukan custom validasi ke dalam object base Python:

```
session["vForm"].setDataInsert(ajaxData)
```

Kode di atas digunakan untuk setDataInsert tanpa commit untuk block head dan det.

```
session["vForm"].actionActive = "INSERT"
```

Kode di atas digunakan untuk set perintah active dengan value atau tipe "INSERT".

```
session["vForm"].getBlock("NAMA_BLOCK").variabel = "value"
```

Kode di atas digunakan untuk membuat tempungan variabel global.

```
session["vForm"].getBlock("NAMA_BLOCK").pkActive =  
session["vForm"].getBlock("NAMA_BLOCK").recordsInsert[0].primaryKey
```

Kode di atas digunakan untuk set pkActive pada block.

```
session["vForm"].dataBlocks[0].validateBase("INSERT")  
session["vForm"].dataBlocks[1].validateBase("INSERT")
```

Kode di atas digunakan untuk memvalidasi block head dan detail

```
session["vForm"].getBlock("NAMA_BLOCK").recordsInsert
```

Kode di atas digunakan untuk mengambil object dataInsert yang telah di set sebelumnya.

```
session["vForm"].getBlock("NAMA_BLOCK").item("itemId").value
```

Kode di atas digunakan untuk mengambil value dari item pada block, jika ditambahkan = value akan set value dari item tersebut.

4.1 How To Create Pre Select in Oden Webix

Dokumentasi untuk membuat fungsi Pre Select pada Oden Webix.

Hanya dipergunakan untuk non-base-table-item database

Pada Oden Webix, terdapat fungsi Pre Select yang memungkinkan Oden Webix membatasi user sebelum select berjalan.

File PY (Folder path : application/)

Pada form Python, tambahkan fungsi preSelect dengan return True / False pada class block layout :

```
class COdentablehead(FormLayout):
    """
    Class ini digunakan untuk class block layout
    @return: -
    """
    def __init__(self, blockId):
        super().__init__(blockId)
        self.whereclause = ""
        self.databaseTable = "public.odentablehead"
        self.nonVisibleItems = [{"oth_create_user", "createUser"},
                                ["oth_update_user", "updateUser"]]
        self.noDbItems = []
        def preSelect(self, data):
            othtextupper = self.item("othtextupper")
            othtextupper.value
            othtextupper.value = ' '
            #YOUR LOGIC HERE
            return {'status': True, 'data': data, 'msg': ''}
```

Jika user menginginkan return false pada fungsi preSelect, return dapat diubah menjadi :

```
return rMessageBox(f"Data tidak ditemukan")
```

Comment '**YOUR LOGIC HERE**' di atas dapat diisi dengan logika pre Select dari user.

Penjelasan fungsi preSelect:

```
othtextupper = self.item("othtextupper")
```

Sintaks di atas digunakan untuk mendapatkan item 'othtextupper', disesuaikan dengan kebutuhan user

```
othtextupper.value
```

Sintaks diatas digunakan untuk mendapatkan value dari item tersebut.

```
othtextupper.value = ' '
```

Sintaks diatas digunakan untuk set value dari item tersebut.

Contoh Pre Select :

1. Tanpa Parameter

```
def preSelect(self,data):  
    sql= """select column_name as id , column_ as value from dmhr.ms_stores  
    ms where mst_status_aktif = 'Y' """  
    return selectHeaderQueryRO(sql)
```

Jika user menggunakan pre select berisi select statement tanpa parameter, bentuk returnnya adalah :

return **selectHeaderQueryRO(sql)**

2. Dengan Parameter

```
def preSelect(self,data):  
    Code1 = '1'  
    sql= """select mst_kd_store as id , mst_nama_store as value from  
    dmhr.ms_stores ms where mst_status_aktif = 'Y' and mst_kd_branch = %  
    (branchCode)s """  
    param = {'code':Code1}  
    return selectDataHeaderQueryRO(sql,param)
```

Jika user menggunakan pre select berisi select statement dengan parameter, bentuk returnnya adalah :

return **selectDataHeaderQueryRO(sql, param)**

[KEMBALI KE TABLE LIST OF CONTENT](#)

4.2 How To Create Post Select in ODEN Webix

Dokumentasi untuk membuat *Post Select* dalam ODEN Webix

Hanya dipergunakan untuk non-base-table-item database

Fungsi **postSelect** digunakan pada saat setelah melakukan *select* data yang memungkinkan Oden Webix melakukan sesuatu setelah melakukan *select* data.

FILE PY

Lokasi File = "{**LOKASI_PROJECT**}\application"

Contoh kode postSelect di dalam file **PY**.

```
class COdentablehead(FormLayout):
    """
    Class ini digunakan untuk class block layout
    @return: -
    """
    def __init__(self, blockId):
        super().__init__(blockId)
        self.whereclause = ""
        self.databaseTable = "public.odentablehead"
        self.nonVisibleItems = [{"oth_create_user", "createUser"},
                                ["oth_update_user", "updateUser"]]
        self.noDbItems = []
        def postSelect(self, data):
            sql = """select lov_text1 as branchdeshead ,lov_text2 as
            branchdeshead2 from lovodentable
            where lov_textnormal = %(othlivtextlovmulticols)s"""
            param = ["othlivtextlovmulticols"]
            setDesc = ["branchdeshead", "branchdeshead2"]
            data =
            getDataDisplayItem({"DATA":data, "BLOCK_TYPE":self.blockType, "SQL":sql, "P
            ARAM":param, "SET_DESC":setDesc})
            return {"status":data["status"], "data":data["data"]}
```

Fungsi postSelect

```
def postSelect(self, data):
    sql = """select lov_text1 as branchdeshead ,lov_text2 as
    branchdeshead2 from lovodentable
    where lov_textnormal = %(othlivtextlovmulticols)s"""
    param = ["othlivtextlovmulticols"]
    setDesc = ["branchdeshead", "branchdeshead2"]
    data =
    getDataDisplayItem({"DATA":data, "BLOCK_TYPE":self.blockType, "SQL":sql, "P
    ARAM":param, "SET_DESC":setDesc})
    return {"status":data["status"], "data":data["data"]}
```

Penjelasan properti dalam fungsi postSelect

- **sql**, berisi sintaks sql untuk mendapatkan data, *lov_text1*, *lov_text2* dan *lov_textnormal* merupakan nama kolom dan *lovodentable* merupakan nama tabel
- **param** berisi ITEM_ID dari input data
- **setDesc** berisi ITEM_ID untuk display item
- **data** berisi pemanggilan fungsi ke utilities

NOTE :

Jika param lebih dari 1, silahkan tambah dengan format list, contoh :

Terdapat sql dengan parameter lebih dari 1

```
sql = """select lov_text1 as branchdeshead ,lov_text2 as
branchdeshead2 from lovodentable where lov_textnormal = %
(othlivtextlovmulticols)s and lov_textupper = %(itemidparam)s"""
```

Maka pada variabel param ditambahkan satu itemid sesuai where clause, dengan format list.

```
param = ["othlivtextlovmulticols", "itemidparam"]
```

Jika membutuhkan validasi tanpa SQL, dapat mengambil dan set value dengan cara berikut:

Cara ambil value dari data pada postSelect :

```
variabel = data["data"][0]["itemId"] #ini ambil value
```

Untuk set value :

```
data["data"][0]["itemId"] = "value" #ini se value
```

Contoh lengkap fungsi **Post Select** dengan **Query** pada **FORM LOV dan DATA GRID**

```
def postSelect(self,data):
sql = """select lov_text1 as branchdeshead ,lov_text2 as
branchdeshead2 from lovodentable where lov_textnormal = %
(othlivtextlovmulticols)s"""
param = ["othlivtextlovmulticols"]
setDesc = ["branchdeshead", "branchdeshead2"]
data =
getDataDisplayItem({"DATA":data, "BLOCK_TYPE":self.blockType, "SQL":sql, "P
ARAM":param, "SET_DESC":setDesc})
return {"status":data["status"], "data":data["data"]}
```

[KEMBALI KE TABLE LIST OF CONTENT](#)

4.3 How To Create Pre Insert in Oden Webix

Dokumentasi untuk membuat fungsi Pre Insert pada Oden Webix.

Pada Oden Webix, terdapat fungsi Pre Insert yang memungkinkan Oden Webix membatasi user sebelum insert dijalankan.

File PY (Folder path : application/)

Pada form Python, tambahkan fungsi preInsert dengan return True / False pada **class block layout** :

```
class COdentablehead(FormLayout):
    """
    Class ini digunakan untuk class block layout
    @return: -
    """
    def __init__(self, blockId):
        super().__init__(blockId)
        self.whereclause = ""
        self.databaseTable = "public.odentablehead"
        self.nonVisibleItems = [{"oth_create_user", "createUser"},
                                ["oth_update_user", "updateUser"]]
        self.noDbItems = []
        def preInsert(self):
            othtextupper = self.item("othtextupper")
            othtextupper.value
            othtextupper.value = ' '
            #YOUR LOGIC HERE
            return {'status':True, 'msg':''}
```

Jika user menginginkan return false pada fungsi preInsert, return dapat diubah menjadi :

```
return rMsgBox(f"Data pada block tidak boleh di insert
{othtextupper.value}")
```

Comment 'YOUR LOGIC HERE' di atas dapat diisi dengan logika preInsert dari user.

Penjelasan fungsi preInsert:

```
othtextupper = self.item("othtextupper")
```

Sintaks di atas digunakan untuk mendapatkan item 'othtextupper', disesuaikan dengan kebutuhan user

```
othtextupper.value
```

Sintaks diatas digunakan untuk mendapatkan value dari item tersebut.

```
othtextupper.value = ' '
```

Sintaks diatas digunakan untuk set value dari item tersebut.

Contoh penggunaan preInsert pada form :

Data header tidak boleh di Insert

Jika ingin membatasi user dalam melakukan insert, cukup tambahkan fungsi preInsert berikut :

```
def preInsert(self):  
    return rMessageBox(f'Data pada block header tidak boleh di insert')
```

[KEMBALI KE TABLE LIST OF CONTENT](#)

4.4 How To Create Post Insert in Oden Webix

Hanya dipergunakan untuk non-base-table-item database

Fungsi postInsert memungkinkan user untuk menjalankan perintah tambahan setelah Insert data dilakukan

File PY (Folder path : application/)

Pada form Python, override fungsi **postInsert()** pada class item FormLayout. Berikut adalah contoh sederhana override fungsi postInsert :

```
def postInsert(self, data):  
    item = self.item("itemID")  
    item.value
```

```
item.value = ''
```

```
return {"status":True, "data":data["data"], "msg":""}
```

Contoh :

```
14
15 class COdentablehead(FormLayout):
16     """
17     Class ini digunakan untuk class block layout
18     @return: -
19     """
20     def __init__(self, blockId):
21         super().__init__(blockId)
22         self.whereclause = ""
23         self.databaseTable = "dmhr.odentablehead"
24         self.nonVisibleItems = [["oth_create_user", "createUser"], ["oth_update_user", "updateUser"]]
25         self.noDbItems = []
26
27     def postInsert(self, data):
28         othtextnormal = self.item("othtextnormal")
29         othddmmyy = self.item("othddmmyy")
30         print(othtextnormal.value, othddmmyy.value)
31         othtextnormal.value = ''
32         return {"status": 'True', 'msg': ''}
33
```

- Selain status True, fungsi postInsert juga dapat mengembalikan status False seperti berikut.

Contoh Post Insert TRUE	Contoh Post Insert FALSE
<pre>def postInsert(self, data): item = self.item("itemID") item.value item.value = '' return {"status":True, "data":data["data"], "msg":""}</pre>	<pre>def postInsert(self, data) item = self.item("itemID") item.value item.value = '' return rMsgBox(f'Data pada boleh di insert {item.valu</pre>

Penjelasan properti dalam fungsi **Post Insert**

- item** merupakan sebuah nama variable yang berisi **self.item("othtextupper")** yang digunakan untuk mendapatkan **block / inputan** yang memiliki ITEM_ID **othtextupper** . Samakan dengan ITEM_ID form input yang di inginkan
- item.value** digunakan untuk mendapatkan *value* dari item (Get Value)
- item.value = " "** digunakan untuk mengisi *value* dari item (Set Value)

4.5 How To Create Pre Update in ODEN Webix

Dokumentasi untuk membuat *Pre Update* dalam ODEN Webix

Hanya dipergunakan untuk non-base-table-item database

Fungsi **preUpdate** digunakan pada saat sebelum melakukan *update* / perubahan pada data yang memungkinkan Oden Webix membatasi user sebelum melakukan *update* / perubahan

FILE PY

Lokasi File = "{**LOKASI_PROJECT**}\application"

Tambahkan fungsi Pre Update di dalam file **PY** di dalam class **COdentablehead**.

```
class COdentablehead(FormLayout):
    """
    Class ini digunakan untuk class block layout
    @return: -
    """
    def __init__(self, blockId):
        super().__init__(blockId)
        self.whereclause = ""
        self.databaseTable = "public.odentablehead"
        self.nonVisibleItems = [{"oth_create_user", "createUser"},
                                ["oth_update_user", "updateUser"]]
        self.noDbItems = []

    def preUpdate(self):
        othtextupper = self.item("othtextupper")
        othtextupper.value
        othtextupper.value = ' '
        #YOUR LOGIC HERE#
        return rMsgBox(f'Data pada block tidak boleh di update
        {othtextupper.value}')
```

Contoh Pre Update TRUE	Contoh Pre Update FALSE
<pre>def preUpdate(self): othtextupper = self.item("othtextupper") othtextupper.value othtextupper.value = ' ' #YOUR LOGIC HERE# return {'status':True,'msg':''}</pre>	<pre>def preUpdate(self): othtextupper = self.item(' othtextupper.value othtextupper.value = ' ' #YOUR LOGIC HERE# return rMessageBox(f'Data pada boleh di update {othtextup</pre>

Penjelasan properti dalam fungsi **preUpdate**

- **othtextupper** merupakan sebuah nama variable yang berisi **self.item("othtextupper")** yang digunakan untuk mendapatkan **block / inputan** yang memiliki ITEM_ID **othtextupper** . Samakan dengan ITEM_ID form input yang di inginkan
- **othtextupper.value** digunakan untuk mendapatkan *value* dari item
- **othtextupper.value = " "** digunakan untuk mengisi *value* dari item
- **#YOUR LOGIC HERE#** digunakan untuk memberikan logic terhadap item yang di inisialisasikan sebelumnya.

Nilai kembalian dari fungsi preUpdate ada 2 yaitu kembalian True dan False.

True	<pre>return {'status':True,'msg':''}</pre>	Jika nilai kembaliannya true, maka akan mengembalikan dictionary dengan 2 key yaitu status dan msg. status berisi True dan msg kosong
False	<pre>return rMsgBox(f'Data pada block tidak boleh di update {othtextupper.value}')</pre>	Jika nilai kembaliannya false, maka akan memunculkan popup box dari fungsi rMsgBox dengan parameternya yaitu pesan gagalanya.

Contoh **Pre Update** dengan dan tanpa Parameter

- **Tanpa Parameter**

```
sql= """update tabel_oden set colom2 = 'bca' where colom1 = 'abc' """
return executeQuery(sql)
```

- **Dengan Parameter**

```
Code1 = '1'
sql= """update tabel_oden set colom2 = 'bca' where colom1 = %(code)s """
param = {'code':Code1}
return executeDataQuery(sql,param)
```

Perbedaan Dengan dan Tanpa Parameter adalah dari Returnnya, jika **Tanpa Parameter** nilai kembaliannya adalah **utilities.executeQuery(sql)** sedangkan **Dengan Parameter** nilai kembaliannya adalah **utilities.executeDataQuery(sql,param)**

Contoh kasus : Pre update hanya untuk 1 kolom (element)

```
def preUpdate(self):
othtext = self.item("othtext")
if othtext.changeStatus:
return {'status':False,'msg':'Tidak boleh update Othtext'}
return {'status':True,'msg':''}
```

Pertama ambil item dari element sesuai kebutuhan, lalu buat if untuk melihat apakah atribut `changeStatus` bernilai `True`,
return false dengan pesan sesuai kebutuhan user.

[KEMBALI KE TABLE LIST OF CONTENT](#)

4.6 How To Create Post Update in Oden Webix

Hanya dipergunakan untuk non-base-table-item database

Fungsi `postUpdate` memungkinkan user untuk menjalankan perintah tambahan setelah Update data dilakukan

File PY (Folder path : application/)

Pada form Python, override fungsi **`postUpdate()`** pada class item `FormLayout`. Berikut adalah contoh sederhana override fungsi `postUpdate` :

```
def postUpdate(self, data):  
    item = self.item("itemID")  
    item.value  
    item.value = ''  
  
    return {"status":True, "data":data["data"], "msg":""}
```

Contoh :


```

14
15 class COdentablehead(FormLayout):
16     """
17     Class ini digunakan untuk class block layout
18     @return: -
19     """
20     def __init__(self, blockId):
21         super().__init__(blockId)
22         self.whereclause = ""
23         self.databaseTable = "dmhr.odentablehead"
24         self.nonVisibleItems = [{"oth_create_user", "createUser"}, {"oth_update_user", "updateUser"}]
25         self.noDbItems = []
26
27     def postUpdate(self, data):
28         othtextnormal = self.item("othtextnormal")
29         othddmmyy = self.item("othddmmyy")
30         print(othtextnormal.value, othddmmyy.value)
31         othtextnormal.value = ''
32
33         sql = """update tabel_oden set kolom2 = 'bca' where kolom1 = %(code)s"""
34         param = {'code' : othtextnormal}
35         utilities.executeDataQuery(sql, param)
36
37         return {'status': 'True', 'msg': ''}
38

```

- Selain status True, fungsi postUpdate juga dapat mengembalikan status False seperti berikut.

Contoh Post Update TRUE	Contoh Post Update FALSE
<pre> def postUpdate(self, data): item = self.item("itemID") item.value item.value = '' return {"status":True, "data":data["data"], "msg":""} </pre>	<pre> def postUpdate(self, data) item = self.item("itemID") item.value item.value = '' return rMsgBox(f'Data pada boleh di Update {item.valu </pre>

Penjelasan properti dalam fungsi **Post Update**

- item** merupakan sebuah nama variable yang berisi **self.item("othtextupper")** yang digunakan untuk mendapatkan **block / inputan** yang memiliki ITEM_ID **othtextupper** . Samakan dengan ITEM_ID form input yang di inginkan
- item.value** digunakan untuk mendapatkan *value* dari item (Get Value)
- item.value = " "** digunakan untuk mengisi *value* dari item (Set Value)

Pada fungsi postUpdate, user dapat menambahkan logic tambahan seperti execute query pada tabel lain.

Contoh :

- execute tanpa parameter menggunakan fungsi executeQuery

```
def postUpdate(self, data):
    othtextnormal = self.item("othtextnormal")
    othddmmyy = self.item("othddmmyy")
    print(othtextnormal.value, othddmmyy.value)
    othtextnormal.value = ' '
    ## INSERT LOGIC HERE ##

    sql = """update tabel_oden set colom2 = 'bca' Where colom1 = 'kolom1'
    """
    executeQuery(sql)

    ##
    return {'status':True, 'msg':''}
```

- execute dengan parameter menggunakan fungsi executeDataQuery

```
def postUpdate(self, data):
    othtextnormal = self.item("othtextnormal")
    othddmmyy = self.item("othddmmyy")
    print(othtextnormal.value, othddmmyy.value)
    othtextnormal.value = ' '
    ## INSERT LOGIC HERE ##

    sql = """update tabel_oden set colom2 = 'bca' Where colom1 = %(code)s"""
    param = {'code': othtextnormal}
    executeDataQuery(sql, param)

    ##
    return {'status':True, 'msg':''}
```

KEMBALI KE TABLE LIST OF CONTENT

4.7 How To Create Pre Delete in ODEN Webix

Dokumentasi untuk membuat *Pre Delete* dalam ODEN Webix

Hanya dipergunakan untuk non-base-table-item database

Fungsi **preDelete** digunakan pada saat sebelum melakukan *delete* / menghapus data yang memungkinkan Oden Webix membatasi user sebelum melakukan *delete* / menghapus data.

FILE PY

Lokasi File = "{**LOKASI_PROJECT**}\application\"

Tambahkan kode ini di dalam file **PY**.

```
class COdentablehead(FormLayout):
    """
    Class ini digunakan untuk class block layout
    @return: -
    """
    def __init__(self, blockId):
        super().__init__(blockId)
        self.whereclause = ""
        self.databaseTable = "public.odentablehead"
        self.nonVisibleItems = [{"oth_create_user", "createUser"},
                                ["oth_update_user", "updateUser"]]
        self.noDbItems = []
        def preDelete(self, data):
            othtextupper = self.item("othtextupper")
            othtextupper.value
            othtextupper.value = ' '
            #YOUR LOGIC HERE#
            return rMsgBox(f'Data pada block tidak dapat di delete
                            {othtextupper.value}')
```

Fungsi Pre Delete

```
def preDelete(self, data):
```

```
othtextupper = self.item("othtextupper")
othtextupper.value
othtextupper.value = ' '
#YOUR LOGIC HERE#
return rMsgBox(f'Data pada block tidak dapat di delete
{othtextupper.value}')
```

Penjelasan properti dalam fungsi preUpdate

- **othtextupper** merupakan sebuah nama variable yang berisi **self.item("othtextupper")** yang digunakan untuk mendapatkan **block / inputan** yang memiliki ITEM_ID **othtextupper** . Samakan dengan ITEM_ID form input yang di inginkan
- **othtextupper.value** digunakan untuk mendapatkan *value* dari item
- **othtextupper.value = " "** digunakan untuk mengisi *value* dari item
- **#YOUR LOGIC HERE#** digunakan untuk memberikan logic terhadap item yang di inisialisasikan sebelumnya. Contoh : Memasukkan data kedalam riwayat .
- nilai kembalian dari fungsi preUpdate ada 2 yaitu kembalian True dan False.

True	<pre>return {'status':True,'msg':''}</pre>	Jika nilai kembaliannya true, maka akan mengembalikan dictionary dengan 2 key yaitu status dan msg. status berisi True dan msg kosong
False	<pre>return rMsgBox(f'Data pada block tidak boleh di update {othtextupper.value}')</pre>	Jika nilai kembaliannya false, maka akan memunculkan popup box dari fungsi rMsgBox dengan parameternya yaitu pesan galanya.

Contoh *Pre Update* dengan dan tanpa Parameter

- **Tanpa Parameter**

```
sql= """delete from table_oden where colom1 = '1' """
return executeQuery(sql)
```

- **Dengan Parameter**

```
Code1 = '1'
sql= """delete from tabel_oden where colom1 = %(code)s """
param = {'code':Code1}
return executeDataQuery(sql,param)
```

Perbedaan Dengan dan Tanpa Parameter adalah dari Returnnya, jika **Tanpa Parameter** nilai kembaliannya adalah **utilities.executeQuery(sql)** sedangkan **Dengan Parameter** nilai kembaliannya adalah **utilities.executeDataQuery(sql,param)**

KEMBALI KE TABLE LIST OF CONTENT

4.8 How To Create Post Delete in Oden Webix

Hanya dipergunakan untuk non-base-table-item database

Fungsi postDelete memungkinkan user untuk menjalankan perintah tambahan setelah Delete data dilakukan

File PY (Folder path : application/)

Pada form Python, override fungsi **postDelete()** pada class item FormLayout. Berikut adalah contoh sederhana override fungsi postDelete :

```
def postDelete(self, data):  
  
    item = self.item("itemID")  
    item.value  
    item.value = ''  
  
    return {'status': True, 'msg': ''}
```

Contoh :

```
14  
15 class COdentablehead(FormLayout):  
16     """  
17     Class ini digunakan untuk class block layout  
18     @return: -  
19     """  
20     def __init__(self, blockId):  
21         super().__init__(blockId)  
22         self.whereclause = ""  
23         self.databaseTable = "dmhr.odentablehead"  
24         self.nonVisibleItems = [["oth_create_user", "createUser"], ["oth_update_user", "updateUser"]]  
25         self.noDbItems = []  
26  
27     def postDelete(self, data):  
28         othtextnormal = self.item("othtextnormal")  
29         othddmmyy = self.item("othddmmyy")  
30         print(othtextnormal.value, othddmmyy.value)  
31         othtextnormal.value = ''  
32  
33         sql = """delete from tabel_oden where kolom1 = %(code)s"""  
34         param = {'code' : othtextnormal.value}  
35         utilities.executeDataQuery(sql, param)  
36  
37         return {'status': 'True', 'msg': ''}  
38
```

- Selain status True, fungsi postDelete juga dapat mengembalikan status False seperti berikut.

Contoh Post Delete TRUE	Contoh Post Delete FALSE
<pre>def postDelete(self, data): item = self.item("itemID") item.value item.value = '' return {'status':True, 'msg':''}</pre>	<pre>def postDelete(self, data) item = self.item("itemID") item.value item.value = '' return rMsgBox(f'Data pada boleh di Delete {item.valu</pre>

Penjelasan properti dalam fungsi **Post Delete**

- **item** merupakan sebuah nama variable yang berisi **self.item("othtextupper")** yang digunakan untuk mendapatkan **block / inputan** yang memiliki ITEM_ID **othtextupper** . Samakan dengan ITEM_ID form input yang di inginkan
- **item.value** digunakan untuk mendapatkan *value* dari item (Get Value)
- **item.value = " "** digunakan untuk mengisi *value* dari item (Set Value)

KEMBALI KE TABLE LIST OF CONTENT

5.1 How To Change Form Title in Oden Webix

Dokumentasi untuk mengubah nama form pada Oden Webix.

File JS (Folder path : static/application)

Untuk mengganti nama form dalam Oden Webix, user dapat mengganti winFormTitle (line 2) pada file JS

```
1  var menuId = 'MENU002'
2  var winFormTitle = 'samplegenerate'
3  var winConfigForm = {'ODENTABLEHEAD':
```

Contoh :

Judul form diubah menjadi "FORM MASTER"

```
var winFormTitle = 'FORM MASTER'
```

Sehingga tampilannya akan menjadi :

otd_textnormal	otd_oth_textnormal	otd_textdisplay	otd_textupper	o
----------------	--------------------	-----------------	---------------	---

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.10 Shortcut keyboard in Oden Webix

Fungsi shortcut keyboard pada Oden Webix

Shortcut	Fungsi shortcut
CTRL + L	Buka Pop-up LOV
CTRL + ← (arrow left)	Previous record
CTRL + → (arrow right)	Next record
CTRL + ↑ (arrow up)	Previous block
CTRL + ↓ (arrow down)	Next block
CTRL + F	Search mode
CTRL + ALT + N	Add value mode
CTRL + =	Add row data
CTRL + U	Update value
CTRL + R	Reset

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.11 How To Use Multi Tab View in Oden Webix

Dokumentasi penggunaan Tab View dalam Oden Webix

Contoh datagrid dengan 6 tab view dimana masing - masing tab datagrid memiliki relasi dengan master form / header

FILE JS

- Tambahkan block baru pada winConfigForm, dengan format :

```
"BLOCK_ID_BARU": {
  "ELEMENT": [ "ITEM_ID_ELEMEN_1", "ITEM_ID_ELEMEN_2", "ITEM_ID_ELEMEN_3" ],
  "PRIMARY_KEY": [ "ITEM_ID_ELEMEN_1" ],
  "RELATION_KEY": [ "ITEM_ID_ELEMEN_2" ],
  "WHEN_NEW_FORM": { },
  "BLOCK_TYPE": [ "DATAGRID", "CHILD" ]
},
```

Contoh:

```
3  var winConfigForm = {
4    "RC_GROUPING_POSISI": {
5      "ELEMENT": [ "rcgpcode", "rcgpdvcode", "rcgpdesc", "rcgpjobdesc", "rcgpjeniskelamin", "rcgpusiamin", "rcgpusiamax", "rcgppengalaman", "rcgpkualifikasi", "rcgprckpocode",
6        "PRIMARY_KEY": [ "rcgpcode" ],
7        "RELATION_KEY": [ ],
8        "WHEN_NEW_FORM": { },
9        "BLOCK_TYPE": [ "FORM", "PARENT" ]
10     },
11     "RC_GROUP_POSISI_STATUS_NIKAH": {
12       "ELEMENT": [ "rgpnrcgpcode", "rgpnkodestatus" ],
13       "PRIMARY_KEY": [ "rgpnkodestatus" ],
14       "RELATION_KEY": [ "rgpnrcgpcode" ],
15       "WHEN_NEW_FORM": { },
16       "BLOCK_TYPE": [ "DATAGRID", "CHILD" ]
17     },
18     "RC_GROUP_POSISI_SIM": {
19       "ELEMENT": [ "rgpsrcgpcode", "rgpskodesim" ],
20       "PRIMARY_KEY": [ "rgpskodesim" ],
21       "RELATION_KEY": [ "rgpsrcgpcode" ],
22       "WHEN_NEW_FORM": { },
23       "BLOCK_TYPE": [ "DATAGRID", "CHILD" ]
24     },
25   },
```

- Tambahkan restapi baru pada winRestApi function :

```
restapi.restApiParam({"BLOCK_NAME":"BLOCK_ID_BARU", "PARAM_ID":
["PARAM_ID"]});
```

Contoh :

```
66 var winRestApi = function(){
67     restapi.restApiParam({'BLOCK_NAME':'RC_GROUP_POSISI_STATUS_NIKAH', 'PARAM_ID':['rcgpcode']});
68     winConfigForm['RC_GROUP_POSISI_STATUS_NIKAH']['WHEN_NEW_FORM'] = {rgpnrcgpcode:$('#rcgpcode').getValue()}
69     restapi.restApiParam({'BLOCK_NAME':'RC_GROUP_POSISI_SIM', 'PARAM_ID':['rcgpcode']});
70 }
71
```

PARAM_ID : berisi list **ITEM_ID** primary key dari **FORM** yang memiliki relasi dengan **DATAGRID**

- Tambahkan variabel element, webix rules, dan variabel block baru :

```
let elementBLOCK_ID_BARU = [
TEXTBOX.TEXTPRIMARY({ITEM_ID:"ITEM_ID_ELEMEN_1", LABEL:"ITEM_ID_ELEMEN_1",
, LENGTH:15}),
TEXTBOX.TEXTNORMAL({ITEM_ID:"ITEM_ID_ELEMEN_2", LABEL:"ITEM_ID_ELEMEN_2",
LENGTH:30}),
TEXTBOX.TEXTNORMAL({ITEM_ID:"ITEM_ID_ELEMEN_3", LABEL:"ITEM_ID_ELEMEN_3",
LENGTH:30}),
]
```

```
let rulesBLOCK_ID_BARU = [
"ITEM_ID_ELEMEN_1" : webix.rules.isNotEmpty,
]
```

```
let blockBLOCK_ID_BARU = datatable.SimpleDatatable("BLOCK_ID_BARU",
elementBLOCK_ID_BARU, rulesBLOCK_ID_BARU, [0,0])
```

Fungsi **datatable.SimpleDatatable** menerima 4 parameter, yaitu: **Block_ID**, variabel element block, variabel rules block, dan **Freeze Column***

****NOTE****

* **Freeze Column** datagrid secara default memiliki value [0, 0] artinya tidak ada kolom yang dibekukan pada sisi kanan ataupun kiri, nilai pada list menandakan jumlah kolom yang akan dibekukan pada datagrid.

* Jumlah **Freeze Column** yang dapat diterapkan masih terbatas, apabila terdapat kasus khusus yang memerlukan beberapa **Freeze Column** sekaligus, dapat dikonsultasikan terlebih dahulu dengan Tim Oden.

- Tambahkan variabel **multiview.element** dan **multiview.view** :

```
let multi1 = multiview.element("LABEL_TAB_DATAGRID",blockDATAGRID)
let multi2 = multiview.element("LABEL_TAB_BARU",blockBLOCK_ID_BARU)

let multiviewGrouping = multiview.view([multi1,multi2])
```

Fungsi **multiview.element** menerima 2 parameter, yaitu: label tab, dan variabel block datagrid

- Label Tab : Berisi string yang akan menjadi label dari tab

Jumlah multiview element disamakan dengan banyaknya tabel detail / Tab yang diperlukan. *(Pada contoh dengan 6 Tab, jumlah multiview element adalah 6)*

Variabel **multiview.view** menerima parameter berupa list dari semua variabel multiview.element yang telah ditambahkan.

NOTE : UNTUK MULTIVIEW BERBENTUK VERTICAL, ganti variabel menjadi verticalMultiView

Contoh :

```
218
219 let multi1 = multiview.element("NIKAH",blockRC_GROUP_POSISI_STATUS_NIKAH)
220 let multi2 = multiview.element("SIM",blockRC_GROUP_POSISI_SIM)
221 let multi3 = multiview.element("KEMAMPUAN",blockRC_GROUP_POSISI_KEMAMPUAN)
222 let multi4 = multiview.element("BAHASA",blockRC_GROUP_POSISI_BAHASA)
223 let multi5 = multiview.element("PENDIDIKAN",blockRC_GROUP_POSISI_PENDIDIKAN)
224 let multi6 = multiview.element("JURUSAN",blockRC_GROUP_POSISI_JURUSAN)
225
226
227 let multiviewGroupingPosisi = multiview.view([multi1,multi2,multi3,multi4,multi5,multi6])
228
```

- Ganti variabel content dengan **baseLayout** baru :

```
let content = layout.baseLayout([blockMASTER_FORM, multiviewGrouping]);
```

variabel content berisi fungsi **layout.baseLayout** yang menerima parameter berupa list yang berisi variabel block Form dan variabel block multiview yang telah ditambahkan

Contoh :

```
241
242     let content = layout.baseLayout([blockRC_GROUPING_POSISI, multiviewGroupingPosisi]);
243
```

FILE PY

- Tambahkan Class Block Datagrid dan Class Record baru untuk setiap Tab yang ditambahkan:

```
class MenuId_CBlock_Baru(DataGridLayout):
    """
    Class ini digunakan untuk class block layout
    @return: -
    """
    def __init__(self, blockId):
        super().__init__(blockId)
        self.whereclause = ""
        self.databaseTable = ""
        self.nonVisibleItems = [{"create_user", "createUser"},
                                ["update_user", "updateUser"]] #ketik manual kolom yang mau ditempatkan
                                sebagai non visible items
        self.noDbItems = []
        self.orderBy = ""
        class MenuId_CRecord_Baru(Record):
            def validate(self):
                """
                Fungsi ini digunakan untuk validasi record
                @return: dictionary dengan key status = nilai boolean (True = validasi
                berhasil, False = validasi gagal)
                key msg = nilai string
                """
                return {"status": True, "msg": ""}
```

- Tambahkan Class item untuk setiap element pada datagrid :

```
class MenuId_CElement_1(TextPrimary):
```

```

lengthItem = 15
itemId = "ITEM_ID_ELEMEN_1"
databaseColumn = "Kolom_Elemen_1"
databaseItem = True
nullable = False
primaryKey = True
allowSpecialChar = []
def validate(self):
    """
    Fungsi ini digunakan untuk validasi column Kolom_Elemen_1
    @return: dictionary dengan key status = nilai boolean (True = validasi
    berhasil, False = validasi gagal)
    key msg = nilai string
    """
    return {"status": True, "msg": ""}

```

- Tambahkan variabel vBlock pada execVcount :

```

MenuId_vBlockBlockIdBaru = MenuId_CBlock_Baru("BLOCK_ID_BARU")
MenuId_vBlockBlockIdBaru.relationBlock = MSTREC18_vBlock_Master_Form
MenuId_vBlockBlockIdBaru.setListObj([MenuId_CElement_1,
MenuId_CElement_2, MenuId_CElement_3])
MenuId_vBlockBlockIdBaru.ObjRecord = MSTREC18_CRecordBaru

```

Nama variabel Block dituliskan dengan sintaks "MenuId_vBlock"+"Block_Id_Baru", di mana "Block_Id_Baru" disamakan dengan Block ID pada file JavaScript.

variabel vBlock object berisi nama class Block dengan parameter berupa Block Id dari object

atribut .relationBlock berisi nama variabel vBlock dari Block Form/ Master/ Header

atribut .setListObj menerima atribut berupa list dari setiap nama class item pada block

atribut .ObjRecord berisi nama Class record dari block

Contoh :

```

771
772 MSTREC18_vBlockRc_Group_Posisi_Sim = MSTREC18_CRC_Group_Posisi_Sim("RC_GROUP_POSISI_SIM")
773 MSTREC18_vBlockRc_Group_Posisi_Sim.relationBlock = MSTREC18_vBlockRc_Grouping_Posisi
774 MSTREC18_vBlockRc_Group_Posisi_Sim.setListObj([MSTREC18_Crgps_rcgp_code, MSTREC18_Crgps_kode_sim,MSTREC18_Crgps_descs_sim])
775 MSTREC18_vBlockRc_Group_Posisi_Sim.ObjRecord = MSTREC18_CRecordRc_Group_Posisi_Sim
776

```

- Tambahkan variabel Vblock pada vContainer :

```

MenuId_vContainer.getForm().dataBlocks = []
MenuId_vContainer.getForm().append(MenuId_vBlockBlockIdBaru)

```

Contoh :

```

796
797 MSTREC18_vContainer.getForm().dataBlocks = []
798 MSTREC18_vContainer.getForm().append(MSTREC18_vBlockRc_Grouping_Posisi)
799 MSTREC18_vContainer.getForm().append(MSTREC18_vBlockRc_Group_Posisi_Status_Nikah)
800 MSTREC18_vContainer.getForm().append(MSTREC18_vBlockRc_Group_Posisi_Sim)
801 MSTREC18_vContainer.getForm().append(MSTREC18_vBlockRc_Group_Posisi_Kemampuan)
802 MSTREC18_vContainer.getForm().append(MSTREC18_vBlockRc_Group_Posisi_Bahasa)
803 MSTREC18_vContainer.getForm().append(MSTREC18_vBlockRc_Group_Posisi_Pendidikan)
804 MSTREC18_vContainer.getForm().append(MSTREC18_vBlockRc_Group_Posisi_Jurusan)
805

```

- Tambahkan fungsi New Instance baru :

```

#NEW FORM INSTANCE
def newInstance_menuId_Block_Id_Baru(**kwargs):
newInstance = {"status":True,"data":{},"msg":""}
# newInstance = selectHeaderQueryRO(sql)
# newInstance = selectDataHeaderQueryRO(sql,param)
return newInstance

```

Contoh :

```

822
823 #NEW FORM INSTANCE
824 def newInstance_MST_REC_18_RC_GROUP_POSISI_SIM(**kwargs):
825     newInstance = {"status":True,"data":{},"msg":""}
826     # newInstance = selectHeaderQueryRO(sql)
827     # newInstance = selectDataHeaderQueryRO(sql,param)
828     return newInstance
829

```

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.12 How To Use checkAllDt, uncheckAllDt, and getAllChecklistData on Checkbox in Datagrid

Dokumentasi untuk penggunaan fungsi checkAllDt dan uncheckAllDt di checkbox pada datagrid

otd_textnormal	otd_textupper	otd_textlower	otd_currency	otd_normalcheckboxbox
DEVTPR03	UPPER TRII	lower five		☐
TXT9901	UPPERONE	awdawd		☐
TXT006A	UPPERTWO			☐
lastlast1a				☐
lastlast1b				☐
lastlast1c				☐
TESTIUS12				☐
asdadadad				☐
NEW0101				☐
1alwnd				☐

Total Currency: 123123 Rp 1.000,00

Page 1 from 15 - Showing 144 from 123456

Check All Uncheck All Custom Button

Contoh button check All, uncheck All, dan custom button untuk mendapatkan data checkbox pada datagrid

FILE JS

- Buatlah variabel berisi list seperti sintaks dibawah ini:

```
let buttonDatagrid = [
  BUTTON.CUSTOMBUTTON({ITEM_ID:"button1", LABEL:"Check All"},
  {CUSTOM_CLICK:function() {
    checkAllDt({"BLOCK_ID":"ODENTABLEDET", "COLUMN_ID":"otdnormalcheckbox"})
  }},
  {width:15},
  BUTTON.CUSTOMBUTTON({ITEM_ID:"button2", LABEL:"Uncheck All "},
  {CUSTOM_CLICK:function() {
    uncheckAllDt({"BLOCK_ID":"ODENTABLEDET",
    "COLUMN_ID":"otdnormalcheckbox"})
  }},
  BUTTON.CUSTOMBUTTON({ITEM_ID:"idButton", LABEL:"Custom Button"},
  {CUSTOM_CLICK:function() {
    getAllChecklistData({"BLOCK_ID":"ODENTABLEDET","PARAM_CHECK" :
    {"otdnormalcheckbox":"Y"}, "OUTPUT":"PK"})
  }},
  ]
```

Pada sintaks diatas, terdapat custom button dengan trigger berisi custom_click dengan value berupa function

yang memanggil fungsi checkAllDt / uncheckAllDt dengan property BLOCK_ID dan COLUMN_ID.

Selain itu, terdapat fungsi getAllChecklistData dengan property BLOCK_ID, PARAM_CHECK, dan OUTPUT untuk mendapatkan semua data yang terchecklist / sesuai value yang di masukkan.

- Jangan lupa tambahkan variabel tersebut pada variabel block<Nama Block>

```
let blockODENTABLEDET = datatable.SimpleDatatable("ODENTABLEDET",
elementODENTABLEDET, rulesODENTABLEDET,[1,0], buttonDatagrid)
```

Penjelasan fungsi checkAllDt dan uncheckAllDt

```
checkAllDt({"BLOCK_ID":"ODENTABLEDET", "COLUMN_ID":"otdnormalcheckbox"})
uncheckAllDt({"BLOCK_ID":"ODENTABLEDET",
"COLUMN_ID":"otdnormalcheckbox"})
```

Atribut :

Nama atribut	Penjelasan
BLOCK_ID	Block_id adalah nama blok dari datagrid
COLUMN_ID	Column_id adalah itemid dari checkbox [ada datagrid

Penjelasan fungsi getAllChecklistData

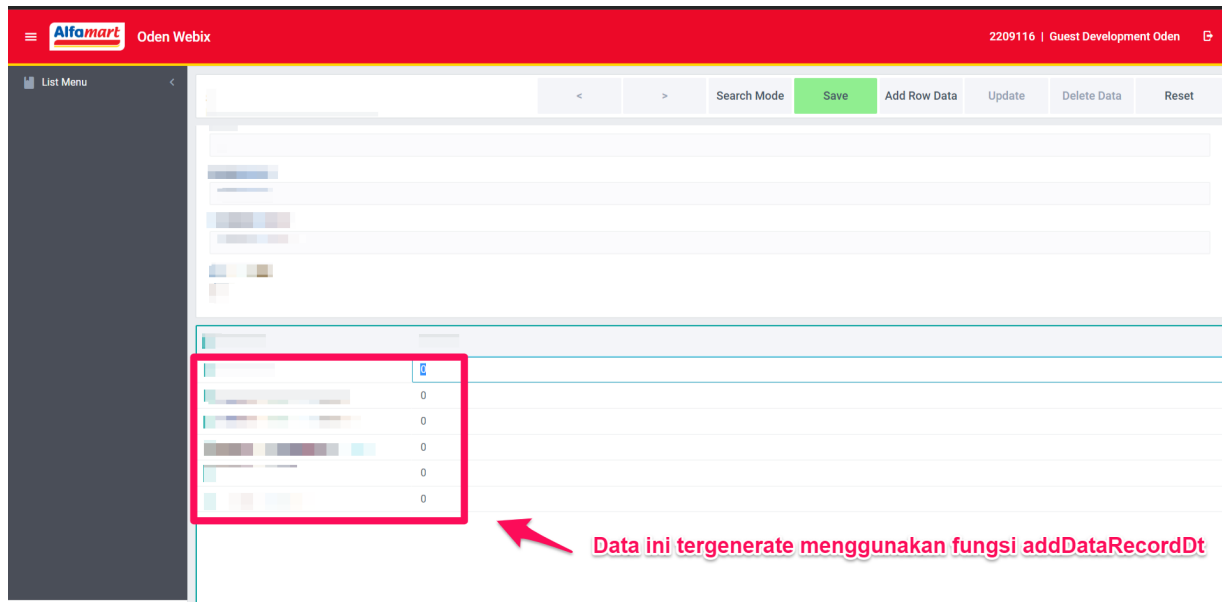
```
getAllChecklistData({"BLOCK_ID":"ODENTABLEDET","PARAM_CHECK" :
{"otdnormalcheckbox":"Y"}, "OUTPUT":"PK"})
getAllChecklistData({"BLOCK_ID":"ODENTABLEDET","PARAM_CHECK" :
{"otdnormalcheckbox":"Y"}, "OUTPUT":"RECORD"})
```

Nama atribut	Penjelasan
BLOCK_ID	Block_id adalah nama blok dari datagrid
PARAM_CHECK	PARAM_CHECK adalah dictionary berisi key itemid , dan value VALUE CHECKBOX
OUTPUT	Property OUTPUT dapat berisi string PK atau RECORD , value PK akan mengeluarkan output hanya PK, dan record akan mengeluarkan output satu record.

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.13 How To use addDataRecordDt in Oden Webix

Dokumentasi penggunaan fungsi addDataRecordDt pada Oden Webix.



FILE JS

Pada file JS bagian yang dibutuhkan developer, tambahkan line berikut :

```

$$("BLOCK_NAME_DATAGRID").clearAll() //harus ada untuk clear datagrid,
tidak tambah lagi
let allRecord = restapi.restApiCustom({"DATA":data,"ITEM_ID":"itemId"})
//opsional jika butuh data ke belakang ex:select
if(allRecord.status){
allRecord.data.forEach(function(record){
addDataRecordDt({"BLOCK":"NAMA_BLOCK","FOCUS":"first","DATA":record})
})
}

```

Variabel record di atas dapat diisi sesuai kebutuhan user, contoh di atas menunjukkan bagaimana bentuk dari value "DATA" pada fungsi addDataRecordDt, bentuknya adalah Object. Selain user dapat mendefinisikan sendiri record yang akan dibentuk, user juga dapat menentukannya melalui API restApiCustom dan mengambil data dari database untuk di insert sebagai record baru.

```

$$("BLOCK_NAME_DATAGRID").clearAll()
allRecord = [{"itemId1":"value1","itemId2":"value2"},
{"itemId1":"value1","itemId2":"value2"}]
allRecord.forEach(function(record){
addDataRecordDt({"BLOCK":"NAMA_BLOCK","FOCUS":"first","DATA":record})
})

```

Contoh diatas jika ingin mendefinisikan itemId dan value sesuai kebutuhan. Masing-masing record dipisahkan dengan ,
allRecord berbentuk list yang dalamnya adalah object/dictionary per record.

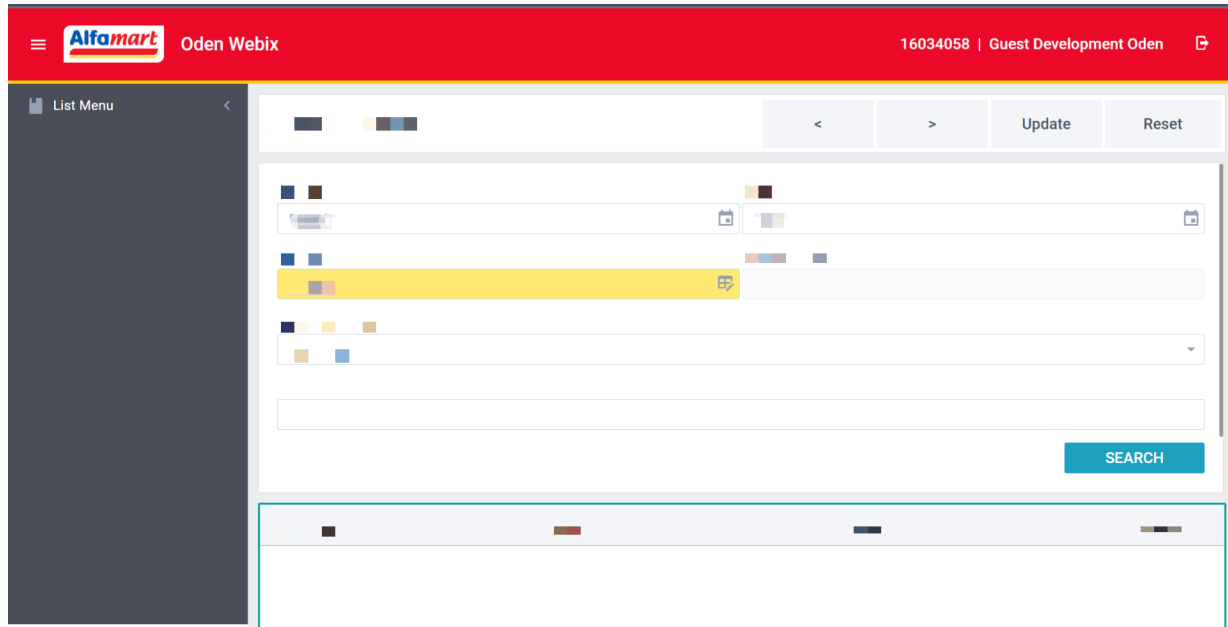
Penjelasan parameter dari fungsi addDataRecordDt :

Parameter	Penjelasan
"BLOCK"	Parameter key ini memiliki value nama block yang akan di addData, sesuai nama fungsi ini yaitu block Datagrid.
"FOCUS"	Parameter key FOCUS menentukan focus cell setelah addDataRecordDt, valuenya berisi " first " atau " last ". Value first memungkinkan focus cell berada pada record paling atas pada datagrid, sebaliknya value last memungkinkan focus cell berada pada record terakhir yang di insert pada datagrid.
"DATA"	Parameter key DATA membutuhkan value berupa Object (js) / Dictionary (dalam python) yang berisi pair key and value berupa itemId dan value yang akan di insert.

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.14 How To Create Non-Base Table Block Filter In Oden Webix

Dokumentasi pembuatan filter(search button) non-base table block dalam Oden Webix



Contoh filter(search button) non-base table block dalam Oden Webix

**CATATAN : HIDE BUTTON SEARCH JIKA FILTER ADALAH NON-BASE TABLE BLOCK.
GUNAKAN BUTTON CUSTOM**

FILE JS

- Buatlah konfigurasi form yang terdiri dari sebuah block pada tampilannya.

```
"NAMA_BLOCK1": {
  "ELEMENT": ["itemid1", "itemid2", "itemid3", "buttonSearch"],
  "PRIMARY_KEY": [],
  "RELATION_KEY": [],
  "WHEN_NEW_FORM": {},
  "BLOCK_TYPE": ["FORM", "NONE"],
  "BASE_TABLE": false
```

- Kemudian tambahkan potongan code dibawah untuk block utama hasil generate pada winConfigForm yang adalah BASE TABLE.

```
"BASE_TABLE": true
```

Buatlah juga variabel element, rules dan block pada javascript

- Pada element di file JS, tambahkan potongan code berikut

```
{cols:[{}],BUTTON.BUTTONFORM({ITEM_ID:"buttonsearch",LABEL:"SEARCH"},
{CUSTOM_CLICK:customBtnTrigger}})]}
```

- Buatlah juga sebuah fungsi yang berfungsi sebagai pengaturan tombol search setelah di klik

```
let customBtnTrigger = function(){
data =
restapi.restApiCustom({"DATA":getBlockValue("NAMA_BLOCK1"),"ITEM_ID":"buttonsearch"})
if(data.status){
let dataFilter = {"data":data["data"][0]};
return restapi.restApiSelect(dataFilter)
}else{
RMSGBOX(data.msg)
}
}
```

- Pada Bagian fungsi webix.ready ganti restapi.restApiSelect() dengan code dibawah ini

```
restapi.isWithoutApi("menuId")
```

FILE PY

- Buatlah Class block filter, record filter, dan item-item nya, seperti pada potongan code dibawah

```
class FORMID_Block_Filter(FormLayout):
"""
Class ini digunakan untuk class block layout
@return: -
"""
def __init__(self, blockId):
super().__init__(blockId)
self.whereclause = ""
self.databaseTable = ""
self.nonVisibleItems = [] #ketik manual kolom yang mau ditempatkan
sebagai non visible items
self.noDbItems = []
self.orderBy = ""
```

```

class FORMID_Record_Filter(Record):
def validate(self):
    """
    Fungsi ini digunakan untuk validasi record
    @return: dictionary dengan key status = nilai boolean (True = validasi
    berhasil, False = validasi gagal)
    key msg = nilai string
    """
    return {'status':True,'msg':''}
class FORMID_contoh(Calendar):
lengthItem = 10
itemId = "itemid1"
databaseColumn = ""
databaseItem = False
nullable = True
allowSpecialChar = []
def validate(self):
    """
    """
    return {"status": True, "msg": ""}

```

Note : Jumlah Class Item disesuaikan dengan banyaknya Item yang ada di winConfigForm.

- Perlu ditambahkan juga potongan code dibawah pada execVcount

```

FORMID_vBlock_Filter = FORMID_Block_Filter("NAMA_BLOCK1")
FORMID_vBlock_Filter.setListObj([FORMID_contoh])
FORMID_vBlock_Filter.ObjRecord = FORMID_Record_Filter

```

```

FORMID_vContainer.getForm().dataBlocks = []

```

- Jika menemui code di atas, maka klik enter, dan tambahkan potongan code seperti di bawah ini

```

FORMID_vContainer.getForm().append(FORMID_vBlock_Filter)

```

- Terakhir, buatlah sebuah fungsi yang berfungsi untuk me return "Block1" yang sudah tervalidasi setelah menekan tombol search.

```
def buttonsearch_FORMID():  
    return nonBaseBlockValidate("NAMA_BLOCK1")
```

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.15 How To Use GO_BLOCK & GO_ITEM Trigger in Oden Webix

Dokumentasi untuk menggunakan trigger element GO_BLOCK dan GO_ITEM pada Oden Webix.

Trigger GO_BLOCK digunakan pada trigger Button maupun Element untuk membuat fokus block berubah.

Trigger GO_ITEM digunakan pada trigger Button maupun Element untuk membuat fokus item berubah.

FILE JS

1. GO_BLOCK

Pada File JS, sesuai kebutuhan tambahkan **trigger "GO_BLOCK"** setelah validate ataupun custom_click.

```
DATE.DDMMYY({ITEM_ID: "itemId", LABEL: "Date", IS_PRIMARY_KEY:true},  
{VALIDATE_ITEM:validateDate, GO_BLOCK: "NAMA_BLOCK"}),
```

Contoh di atas adalah jika GO_BLOCK digunakan pada Element.

Value dari trigger GO_BLOCK adalah NAMA_BLOCK dari block yang ingin dijadikan focusNow.

2. GO_ITEM

Penggunaan GO_ITEM dibagi menjadi 2,

Jika GO_ITEM hanya menuju 1 itemId tanpa butuh kondisi / if-else untuk menentukan item yang dituju, maka tambahkan trigger berikut:

```
DATE.DDMMYY({ITEM_ID: "itemId", LABEL: "Date", IS_PRIMARY_KEY:true},  
{VALIDATE_ITEM:validateDate, GO_ITEM: "itemId"}),
```


NOTE : Jika itemId berada pada block lain, tidak perlu menambahkan GO_BLOCK lagi, karena trigger GO_ITEM akan otomatis mencari block yang memiliki item tersebut.

Jika GO_ITEM membutuhkan kondisi yang mengarah kepada beberapa item, maka tambahkan trigger berikut:

```
DATE.DDMMYY({ITEM_ID: "cekhtgltest", LABEL: "Efektif Date SK",  
IS_PRIMARY_KEY:true},{VALIDATE_ITEM:validateCekhTglTes, GO_ITEM:  
goItemFunction})),
```

Value item id diubah menjadi function, yang selanjutnya silahkan tambahkan kode berikut :

```
statusValidate = false //ini variabel global  
  
let goItemFunction = function(){  
  if(statusValidate){  
    return "itemId"  
  }else{  
    return "itemId2"  
  }  
  //PUT YOUR CONDITION HERE  
}
```

Di atas adalah contoh jika GO_ITEM membutuhkan kondisi dan mengarah ke beberapa item sesuai kondisi tersebut.

statusValidate adalah global variabel yang akan diubah sesuai kebutuhan pengguna dalam validasi maupun trigger button.

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.16 How To Custom Pagination Information in Datagrid

Dokumentasi untuk merubah informasi total record pada datagrid

Page 1 from 15 - Showing 145 from 123456

FILE JS

```
10 var winRestApi = function(){
11     // restapi.restApiParam("BLOCK_NAME:"ODENTABLEDETAIL",ID_PARAM:[ITEM_ID_FK]);
12     restapi.restApiTotalCount({BLOCK_NAME:"ODENTABLEDET"})
13 }
```

Pada file JS, tambahkan kode berikut pada variabel **winRestApi** :

```
restapi.restApiTotalCount({BLOCK_NAME:"BLOCK_NAME_DATAGRID"})
```

Parameter dari restApiTotalCount adalah nama block datagrid. Nama block ini digunakan untuk menghubungkan dengan fungsi total count pada bagian Python.

FILE PY

```
1036 def totalCount_NAMA_BLOCK_DATAGRID_MENU_ID(**kwargs):
1037     sql = """select count(*) from schema.table
1038     """
1039     data = selectQuery(sql)
1040     return {"status":True, "data":data["data"], "msg":""}
```

Pada file Py, tambahkan fungsi berikut :

```
def totalCount_NAMA_BLOCK_DATAGRID_MENU_ID(**kwargs):
    sql = """select count(*) from schema.table
    """
    data = selectQuery(sql)
    return {"status":True, "data":data["data"], "msg":""}
```

Kode diatas adalah contoh jika user ingin menampilkan informasi total data yang ada pada tabel database untuk ditampilkan pada informasi paginasi datagrid.

Nama fungsi terdiri dari totalCount_NAMA_BLOCK_DATAGRID_MENU_ID.

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.17 How To Format Ajax Data in Custom API using formatParamAjax Function Gate

Dokumentasi untuk membuat ajax data yang dikirim ke python terformat sesuai dengan tipe element dan siap untuk menjadi bind param pada sql Postgre.

FILE JS

```
1 data = formatParamAjax({"BLOCK_NAME":{"blockType":"form","data":[getBlockValue("BLOCK_NAME")]},"Search Data")
```

```
data = formatParamAjax({"BLOCK_NAME":{"blockType":"form","data":  
[getBlockValue("BLOCK_NAME")]}},"Search Data")
```

Fungsi `formatParamAjax` akan memformat pair key-value yang masuk sesuai dengan element dari item id yang ada pada key. Contoh kegunaannya ada pada saat user membutuhkan tanggal dengan format YYYY-MM-DD untuk dapat langsung digunakan sebagai bind-param pada `selectDataHeaderQueryRO` / `selectDataQueryRO`. Fungsi ini dapat digunakan untuk memformat data yang akan menjadi data dari `restApiCustom`.

Penjelasan parameter `formatParamAjax`:

Nama Parameter	Penjelasan
BLOCK_NAME	Key BLOCK_NAME adalah nama block dari element yang sedang ditambahkan validasi. User harus menggantinya dengan nama block yang sesuai.
blockType	Key blockType bersifat statis dan tidak perlu diganti user.
form	Value form disesuaikan dengan tipe blok dari element yang sedang ditambahkan validasi
data	Parameter data berisi dictionary / array dalam list yang dapat diganti dengan fungsi <code>getBlockValue("block_name")</code> , sesuai dengan kebutuhan param ajax pengguna.
"Search Data"	Search data bersifat statis, tidak perlu diubah. Search data menandakan fungsi <code>formatParamAjax</code> digunakan untuk keperluan search data / melakukan validasi.

Selanjutnya data yang berupa `formatParamAjax` dapat digunakan untuk menjadi parameter DATA pada `restApiCustom` sesuai dengan kebutuhan pengguna.

NOTE : JIKA VALUE DARI SEBUAH ITEM KOSONG, MAKA KEY DARI ITEM TERSEBUT AKAN DIHAPUS (TIDAK ADA PADA RETURN FUNGSI INI)

Dokumentasi `restapi.restApiCustom`

5.18 How to Display SUM, AVERAGE, MIN, MAX in Datagrid

Dokumentasi untuk menampilkan perhitungan sum (total), average(rata-rata), min, max di datagrid

otd_textnormal	otd_numericnormal	otd_currency	otd_jivtextiovmulticols	otd_rad
1	10	Rp 1.000,00		
1alwnd	10	Rp 2.000,00		
3	10	Rp 3.000,00		
5	10	Rp 4.000,00		
6	10	Rp 10.000,00		
8	10	Rp 25.000,00		
9	10	Rp 50.000,00		
aaaaaaaa1	10	Rp 100.000,00		
aaaaaaaa2	10	Rp 125.000,00		
aaaaaaaa3	20	Rp 150.000,00		
Total Currency	11.0000	Rp 470.000,00		

Page 1 from 1 - Showing 10 from 10

FILE JS

Untuk menambahkan display / tampilan perhitungan SUM, AVERAGE, MIN, MAX pada datagrid, tambahkan kode berikut:

```
let customOperatorDatagrid = {  
  ID_DATAGRID:"ODENTABLEDET",  
  ID_COLUMN_AND_TYPE:{"otdnumericnormal":"AVG","otdcurrency":"SUM",},  
  CUSTOM_TEXT:"Total Currency",  
  IS_CURRENCY:["otdcurrency"],  
}
```

Penjelasan dari variabel di atas:

Key	Value	Penjelasan
ID_DATAGRID	BLOCK_NAME	Value dari ID_DATAGRID adalah blockname dari datagrid yang ingin dilakukan operasi aritmatika
ID_COLUMN_AND_TYPE	{"itemId":"jenis"}	Value dari ID_COLUMN_AND_TYPE adalah dictionary berisi itemId dan jenis operasi yang akan ditampilkan
CUSTOM_TEXT	String bebas	Value dari CUSTOM_TEXT sesuai nama key nya yaitu string bebas yang akan ditampilkan sebagai keterangan untuk nilai yang ditampilkan
IS_CURRENCY	List itemId	Value dari IS_CURRENCY adalah list dari itemId yang berjenis currency, supaya dapat menjadi format Rupiah.

Setelah menambahkan variabel diatas, tambahkan juga variabel tersebut pada variable blockNAMABLOCK:

```
let blockODENTABLEDET = datatable.SimpleDatatable("ODENTABLEDET",
elementODENTABLEDET, rulesODENTABLEDET, [0,0], [], customOperatorDatagrid)
```

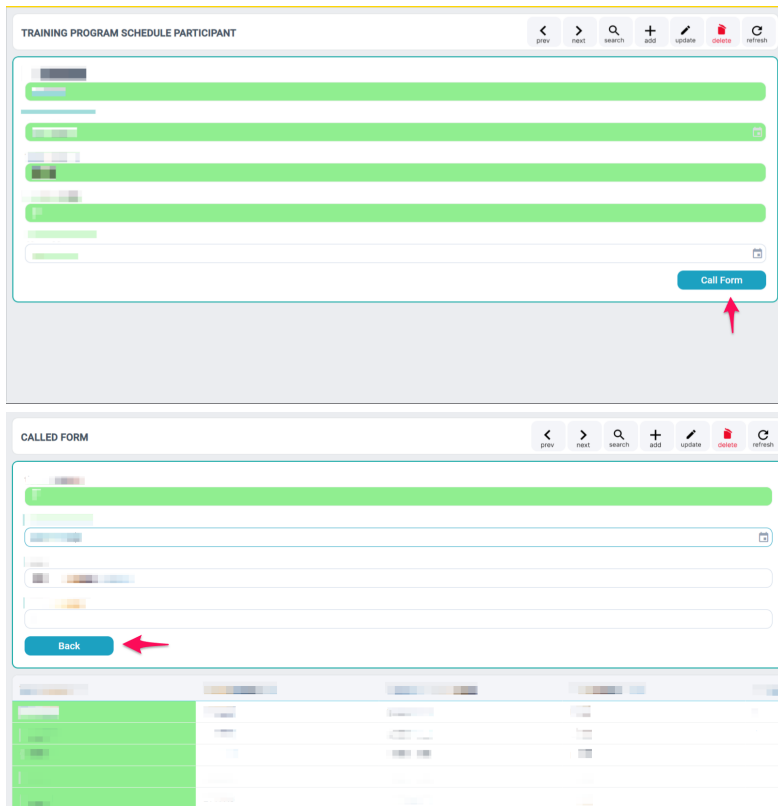
NOTE : PERLU DITAMBAHKAN LIST KOSONG SEBELUM VARIABEL JIKA TIDAK ADA BUTTON DATAGRID, seperti pada Line 2 pada kode di atas.

Untuk mengambil data display tersebut, gunakan tampungan berikut pada Javascript (OPTIONAL) :

```
winOperatorDt["NAMA_BLOCK_DATAGRID"]["ITEM_ID"]
```

5.19 How to Call Form and Call Back in Oden Webix

Dokumentasi untuk fitur call form dan call back pada Oden Webix



Form 1. Training Program Schedule Participant
dengan PK berelasi Form 1

Form 2. Form yang dituju,

**NOTE : Form 2 atau form yang dipanggil harus memiliki suffix / NAMA BELAKANG _ND.
Contohnya : ASS_99_ND,
PASTIKAN FORM 1 DAN 2 MEMILIKI PREFIX CLASS YANG BERBEDA. PREFIX =
SHORTDESC saat Generate / membuat manual**

FILE JS

Form 1 (Call Form) :

Form 1 adalah form yang memanggil form lain dengan fitur call form. Untuk menggunakan fitur call form, langkah pembuatannya adalah :

1. Membuat button dengan custom trigger

```
{cols:[{}],BUTTON.BUTTONFORM({ITEM_ID:"itemId",LABEL:"Call  
Form",CALL_FORM:true},{CUSTOM_CLICK:triggerBtn})}]}
```

Note : sesuaikan button menurut jenis block, misal : buttonform, buttondatagrid, atau custombutton

Atribut CALL_FORM:true wajib ditambahkan untuk membuat button menjadi disable saat Search maupun Insert.

2. Menambahkan variabel trigger button

```
let triggerBtn = function () {  
  callForm("MENU_ID_FORM_YANG_INGIN_DIPANGGIL")  
}
```

Pada trigger button, tambahkan fungsi callForm dengan parameter string menuId Form 2, atau form yang akan dipanggil.

FILE PY

Form 1 :

Pada file python form 1, tambahkan fungsi API callform :

```
def callForm_MENU_ID_TUJUAN_MENU_ID_SEKARANG():  
  pass
```

Fungsi tersebut berisi pass statement.

Form 2 :

Pada file python form 2 (form yang dipanggil menggunakan Call Form), tambahkan whereclause pada bagian block header sesuai dengan PK yang berhubungan dengan form 1, contohnya :

```
self.whereclause = " kolom_db1 = %(bindParam1)s and kolom_db2 = %  
(bindParam2)s and kolom_db3= %(bindParam3)s and kolom_db4 = %  
(bindParam4)s"
```

NOTE : Pastikan whereclause yang diisi adalah semua PK (primaryKey) dari form 1, karena parameter yang dikirim adalah PK dari form 1. Bind paramnya juga sesuaikan dengan itemId dari form pertama (form pemanggil)

Pada variabel execVcount tambahkan kode berikut setelah deklarasi vForm (SHORTDESC_vForm = Form()), LINE ke 3 setelah execVcount :

```
SHORTDESC_vForm.callFormLevel = 1
```

Inisialisasi callFormLevel wajib ditambahkan pada form yang akan dipanggil dengan value disesuaikan dengan urutan callForm.

Jika Form 1 memanggil Form 2, maka pada Form 2 tambahkan code di atas pada execVcount baris ke 4,

Jika Form 1 memanggil Form 2, dan Form 2 memanggil Form 3, maka pada Form 2 tambahkan code diatas **(FORM 2 ADALAH LEVEL 1)**, dan pada Form 3 tambahkan code diatas dengan value 2 **(FORM 3 ADALAH LEVEL 2)**.

Level menyesuaikan banyaknya call form dan urutan call form.

FILE JS

Form 2 (Call Back) :

Pada form terpanggil, tambahkan button untuk kembali ke form sebelumnya:

```
{cols:[BUTTON.BUTTONFORM({ITEM_ID:"callback",LABEL:"Back"},
{CUSTOM_CLICK:customBtnTrigger}),{}}]
```

Note : sesuaikan button menurut jenis block, misal : buttonform, buttondatagrid, atau custombutton

Tambahkan juga custom trigger berikut:

```
let customBtnTrigger = function(){
return callBack("MENU_ID_FORM_YANG_MEMANGGIL")
}
```

Custom trigger mereturn fungsi callBack dengan parameter menuId dari form sebelumnya.

CATATAN PENTING CALL FORM :

JIKA TERJADI DATA TIDAK VALID, PASTIKAN VALUE PK PADA PYTHON DAN JS SUDAH SAMA,

Jika masih terjadi, pindahkan restApiLov ke winRestApi

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.2 How To Hide Button in Oden Webix

Dokumentasi untuk menghilangkan button action pada Oden Webix.

Pada file JS hasil generate form, terdapat variabel hideButton seperti di bawah ini :

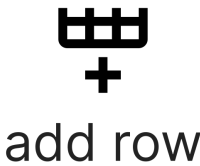

```
9  ✓ var hideButton = {  
10  ✓    // true = hidden button  
11      // false = show button  
12      "next" : false,  
13      "prev" : false,  
14      "searchValue" : false,  
15      "addValue" : false,  
16      "updateValue" : false,  
17      "deleteValue" : false,  
18      "clearBtn" : false  
19  }
```

Variabel `hideButton` berfungsi untuk menentukan button apa saja yang akan ditampilkan pada tampilan form Oden Webix.

Value **false** berarti button tersebut **ditampilkan**.

Sebaliknya value **true** akan membuat button tersebut menghilang dari tampilan Oden Webix.

No	Key	Tampilan	Keterangan
1.	next		Value true untuk key " next " menghilangkan button next pada tampilan Oden Webix
2.	prev		Value true untuk key " prev " menghilangkan button prev pada tampilan Oden Webix
3.	searchValue		Value true untuk key " searchValue " menghilangkan button Search Mode pada tampilan Oden Webix
4.	addValue		Value true untuk key " addValue " menghilangkan button Add New Data pada tampilan Oden Webix
5.	updateValue		Value true untuk key " updateValue " menghilangkan button Update pada tampilan Oden Webix

6.	deleteValue		Value true untuk key " deleteValue " menghilangkan button Delete Data pada tampilan Oden Webix
7.	clearBtn		Value true untuk key " clearBtn " menghilangkan button Reset pada tampilan Oden Webix
8.	addRowDb		Value true untuk key " addRowDb " menghilangkan button add row pada tampilan Oden Webix

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.20 How to Configure Load Data at Start for LOV

Dokumentasi untuk konfigurasi Load data at start (untuk efisiensi loading data saat form dibuka untuk LOV)

FILE JS

1. Perhatikan kode di bawah ini :

```
var winConfigForm = {
```

```

"BLOCK_NAME": {
  "ELEMENT": [
    "element1",
    "element2",
  ],
  "PRIMARY_KEY": ["pk1parent"],
  "RELATION_KEY": [],
  "WHEN_NEW_FORM": {},
  "BLOCK_TYPE": ["FORM", "PARENT"],
  "LOAD_DATA_AT_START" : true,
  "LOV_API":{
    "itemIdCodeLov": {"ITEM_ID":"itemIdCodeLov", "TABLE_ID":"dtNamaLov"}
  },
}

```

2. Pada block child, developer dapat menambahkan 2 key baru :

```

"LOAD_DATA_AT_START" : false,

```

Key ini akan menentukan block tersebut akan diproses atau tidak pada saat form dibuka.

Jika LOAD_DATA_AT_START : true, maka Lov yang didaftarkan pada winconfigform seperti biasa di load saat form terbuka.

Sebaliknya jika false, maka akan dijalankan / diproses saat button LOV di klik.

```

"LOV_API": { "itemIdCodeLov":
{"ITEM_ID":"itemIdCodeLov", "TABLE_ID":"dtNamaLOV"} }

```

Key LOV_API menggantikan restApiLov pada webix.ready ataupun pada winRestApi, formatnya berupa key code lov dan valuenya sama seperti parameter restApiLov. Dapat menggunakan parameter PARAM_ID jika dibutuhkan.

- Fungsi **winRestApi** pada file JS

```

var winRestApi = function(){
  loadDataBlockAtStart()
}

```

pada winRestApi, tambahkan kode di atas untuk menggantikan restApiParam / restApiLov. Fungsi tersebut akan menjalankan restApi secara otomatis berdasarkan winConfigForm.

NOTE : Pada block header juga dapat menambahkan LOAD_DATA_AT_START : false, dan LOV_API untuk LOV

5.21 How to Configure Load Data at Start for Multiview

Dokumentasi untuk konfigurasi Load data at start (untuk efisiensi loading data saat form dibuka untuk MULTIVIEW)

FILE JS

1. Perhatikan kode di bawah ini :

```
var winConfigForm = {
  "BLOCK_PARENT": {
    "ELEMENT": [
      "element1",
      "element2",
    ],
    "PRIMARY_KEY": ["pk1parent"],
    "RELATION_KEY": [],
    "WHEN_NEW_FORM": {},
    "BLOCK_TYPE": ["FORM", "PARENT"]
  },
  "BLOCK_CHILD_1_MULTIVIEW": {
    "ELEMENT": [
      "elementchild1",
      "elementchild2"
    ],
    "PRIMARY_KEY": ["pk1child"],
    "RELATION_KEY": ["pk1parent"],
    "WHEN_NEW_FORM": {},
    "BLOCK_TYPE": ["DATAGRID", "CHILD"],
    "LOAD_DATA_AT_START" : true,
    "REST_API": {"BLOCK_NAME": "BLOCK_CHILD_1_MULTIVIEW", "PARAM_ID":
      ["pk1parent"]},
  },
  "BLOCK_CHILD_2_MULTIVIEW": {
```

```

"ELEMENT": [
  "elementmultichild1",
  "elementmultichild2"
],
"PRIMARY_KEY": ["pk1child2"],
"RELATION_KEY": ["pk1parent"],
"WHEN_NEW_FORM": {},
"BLOCK_TYPE": ["DATAGRID", "CHILD"],
"LOAD_DATA_AT_START" : false,
"REST_API": {"BLOCK_NAME": "BLOCK_CHILD_2_MULTIVIEW", "PARAM_ID":
["pk1parent"]},
}
}

```

2. Pada block child, developer dapat menambahkan 3 key baru :

```

"LOAD_DATA_AT_START" : false,

```

Key ini akan menentukan block tersebut akan diproses atau tidak pada saat form dibuka. Key selanjutnya yang akan dijelaskan akan berjalan sesuai dengan key ini.

Jika `LOAD_DATA_AT_START : true`, maka tab view yang didaftarkan pada restapi dibawah akan seperti biasa di load saat form terbuka. Sebaliknya jika `false`, maka akan dijalankan / diproses saat Multiview di klik.

```

"REST_API": {"BLOCK_NAME": "BLOCK_CHILD_1_MULTIVIEW", "PARAM_ID":
["pk1parent"]},

```

Key `REST_API` menggantikan `restApiParam` pada `winRestApi`, dictionarynya sama dengan parameter `restApiParam`

- Fungsi **winRestApi** pada file JS

```

var winRestApi = function(){
  loadDataBlockAtStart()
}

```

pada `winRestApi`, tambahkan kode di atas untuk menggantikan `restApiParam`. Fungsi tersebut akan menjalankan `restApi` secara otomatis berdasarkan `winConfigForm`.

5.22 How To Make additional Custom Row in CustomOperatorDatagrid

Dokumentasi untuk membuat row tambahan pada fitur custom operator datagrid

```
let customOperatorDatagrid = {
  ID_DATAGRID:"BLOCK_ID_DATAGRID",
  ID_COLUMN_AND_TYPE:
  {"itemId1":"SUM","itemId2":"SUM","itemId3":"AVG","itemId4":"MAX"},
  //custom operator default (row 1)
  CUSTOM_TEXT: {"itemId1":"Total"},
  IS_CURRENCY:[],
  ADD_CUSTOM_ROW:[{"text":"Score", sum_itemId1:"", sum_itemId2:"",
  avg_itemId3:"valueStatic", max_itemId4:""}, //code disamping untuk row 2
  {"text":"Average", avg_itemId3:"123123"}] //code disamping untuk row 3
}
```

Tambahkan satu key pada customOperatorDatagrid yaitu "ADD_CUSTOM_ROW". Tipe data dari key tersebut ialah obj dalam list

- Obj --> berisi key yang akan digunakan dalam row tsb
 - Untuk key pada obj tersebut harus berhubungan dengan key dan operator pada row sebelumnya.
format = operator_itemId
contoh = sum_msjcode
- List --> berisi jumlah Obj(row) yang ingin ditambahkan (bisa lebih dari satu row yang ditambahkan)

Pada masing - masing row dapat ditambahkan key "text" yang berfungsi sebagai header yang akan diletakkan disamping data custom dengan index / urutan paling kiri

Untuk mengisi custom row dengan data dari database (API CUSTOM), tambahkan kode berikut:

```

var winConfigForm = {"BLOCK_ID_DATAGRID": {
"ELEMENT": ["itemId1","itemId2","itemId3","itemId4","itemId5"],
"PRIMARY_KEY": ["itemId1"],
"RELATION_KEY": [], "WHEN_NEW_FORM": {}, "BLOCK_TYPE": ["DATAGRID",
"PARENT"], "BASE_TABLE":true,
"CUSTOM_OPERATOR_DT": [{
"API" : "customText", //sesuai dengan nama API di python
"COLS_ID": ["sum_itemId1","max_itemId4"], //akan diisi berurutan dengan
hasil query di PY
"PARAM_ID": []}, //untuk menambahkan parameter bagi query di PY
],
},
}

```

Tambahkan satu key baru pada BLOCK di winConfigForm :

```

"CUSTOM_OPERATOR_DT": [{ "API" : "customText", //sesuai dengan nama API
di python
"COLS_ID": ["sum_itemId1","max_itemId4"],//akan diisi berurutan dengan
hasil query di PY
"PARAM_ID": []}, //untuk menambahkan parameter bagi query di PY
],

```

Key ini berbentuk dictionary dalam list, dengan urutan berdasarkan row (index 0 untuk row kedua, index 1 untuk row ketiga)

- Key "API" adalah ID yang menghubungkan kepada class restApiCustom pada PY
- Key "COLS_ID" adalah id kolom COLS_ID yang berada pada customOperator, urutan cols_id harus bersesuaian dengan hasil urutan query database di class python
- Key "PARAM_ID" adalah list berisi parameter yang dibutuhkan untuk query di API CUSTOM DI PY.

Custom class pada python dapat dibuat sebagai berikut, dengan memperhatikan nama API dan MenuID

```

def customText_menu_ID(**kwargs):
sql=f"""

```



```

select count(column_name) from schema.table where column_name= %
(itemIdParam)s
"""
ajaxData = json.loads(request.args.get("data"))
return selectDataQueryRO(sql, ajaxData)

```

5.23 How To Create Subparent Multiview Form (1 Table)

Dokumentasi membuat subparent multiview form yang merupakan 1 tabel dengan form parentnya

Subparent Multiview merupakan sebuah fitur dimana sebuah form parent memiliki atribut dalam satu tabel database yang letaknya tidak pada form header melainkan pada Multiview dibawahnya.

Gambaran

The screenshot displays a web application interface for 'SAMPLE 2'. At the top, there is a navigation bar with buttons for 'prev', 'next', 'search', 'add', 'update', 'delete', and 'reset'. The main form area is divided into two sections. The top section, labeled 'Multiview 1', contains three input fields: 'othtextnormal' (with the value 'new00001'), 'othtextdisplay', and 'othcombolistapi'. The bottom section, labeled 'Multiview 2', contains a table with two columns: 'Display Code' and 'Display Desc'. The first row of the table has 'Display Code' highlighted in yellow. Below the table, there is a field for 'othtextupper'.

Form diatas dan form pada mutliview dibawah berasal dari 1 tabel database yang sama akan tetapi dipisah menjadi 2 tampilan.

Langkah Pembuatan

FILE JS

**perubahan hanya dilakukan pada file JS saja*

```
1  'ODENTABLEHEAD':  
2  {  
3    'ELEMENT': ['othtextnormal', 'othtextdisplay', 'othcombolistapi', 'othlivtextlovmulticols', 'othtextupper'],  
4    'PRIMARY_KEY': ['othtextnormal'],  
5    'RELATION_KEY': [],  
6    'WHEN_NEW_FORM': {},  
7    'BLOCK_TYPE': ['FORM', 'PARENT'],  
8    "LOV_API": {  
9      "othlivtextlovmulticols": { "ITEM_ID": "othlivtextlovmulticols", "TABLE_ID": "dtDisplayHeaderView" }  
10   },  
11  },
```

1. Pada Codingan diatas terdapat block *ODENTABLEHEAD* yang memiliki 5 element dan *othtextnormal* merupakan primary keynya, kemudian kita ingin memisahkan element *othlivtextlovmulticols* dan *othtextupper* kedalam block baru yang bernama *ODENTABLEHEAD_SUBPARENT*

```

1  'ODENTABLEHEAD':
2  {
3    'ELEMENT': ['othtextnormal', 'othtextdisplay', 'othcombolistapi'],
4    'PRIMARY_KEY': ['othtextnormal'],
5    'RELATION_KEY': [],
6    'WHEN_NEW_FORM': {},
7    'BLOCK_TYPE': ['FORM', 'PARENT'],
8  },
9
10
11 'ODENTABLEHEAD_SUBPARENT':
12 {
13   'ELEMENT': ['othlivtextlovmulticols', 'othtextupper'],
14   'PRIMARY_KEY': [],
15   'RELATION_KEY': ['othtextnormal'],
16   'WHEN_NEW_FORM': {},
17   'BLOCK_TYPE': ['FORM', 'SUB_PARENT'],
18   "LOV_API": {
19     "othlivtextlovmulticols": { "ITEM_ID": "othlivtextlovmulticols", "TABLE_ID": "dtDisplayHeaderView" }
20   },
21 },

```

2. Pada Codingan diatas sudah kita sudah memisahkan *othlivtextlovmulticols* dan *othtextupper* ke dalam block baru yang bernama *ODENTABLEHEAD_SUBPARENT*, kemudian pada block tersebut, primary key-nya dikosongkan dan pada relation key kita tambahkan primary key dari *ODENTABLEHEAD* yang merupakan parent-nya, jangan lupa juga karena *othlivtextlovmulticols* merupakan sebuah LOV, maka kita harus mengikutsertakan LOV_API-nya

* *nama dari block dapat disesuaikan tidak perlu menggunakan _SUBPARENT, contoh: ODENTABLEHEAD_DESC*

```

1  let elementODENTABLEHEAD = [
2    TEXTBOX.TEXTPRIMARY({ ITEM_ID: 'othtextnormal', LABEL: 'othtextnormal', LENGTH: 10 }),
3    TEXTBOX.TEXTDISPLAY({ ITEM_ID: 'othtextdisplay', LABEL: 'othtextdisplay' }),
4    COMBO.COMBOLISTAPI({ ITEM_ID: 'othcombolistapi', LABEL: 'othcombolistapi' }),
5  ]
6  let elementODENTABLEHEAD_SUBPARENT = [
7    LOVFIELD.TEXTLOVMULTICOLS({
8      ITEM_ID: ["othlivtextlovmulticols", "displayItemHead"],
9      LABEL: ["Display Code", "Display Desc"],
10     WINDOW_ID: "wdDisplayHeaderView", TOTAL_COLS: 2, LENGTH: 10
11   }),
12   { VALIDATE_ITEM: validateothlivtextlovmulticols },
13   TEXTBOX.TEXTUPPER({ ITEM_ID: 'othtextupper', LABEL: 'othtextupper', LENGTH: 10 }),
14 ]

```

```

1  let rulesODENTABLEHEAD = {
2    "othtextnormal": webix.rules.isNotEmpty,
3  }
4  let rulesODENTABLEHEAD_SUBPARENT = {}

```

3. Kemudian kita pisahkan juga 2 item sub-parent tadi kedalam element baru serta membuat rules sendiri untuk block sub-parent tersebut

```

1  let blockODENTABLEHEAD = form.SimpleForm("ODENTABLEHEAD", elementODENTABLEHEAD, rulesODENTABLEHEAD)
2  let blockODENTABLEHEAD_SUBPARENT = form.SimpleForm("ODENTABLEHEAD_SUBPARENT", elementODENTABLEHEAD_SUBPARENT, rulesODENTABLEHEAD_SUBPARENT)
3  let blockODENTABLEDET = datatable.SimpleDatatable("ODENTABLEDET", elementODENTABLEDET, rulesODENTABLEDET, [0, 0])
4
5
6  let multi1 = verticalMultiView.element("Multiview 1", blockODENTABLEDET)
7  let multi2 = verticalMultiView.element("Multiview 2", blockODENTABLEHEAD_SUBPARENT)
8  let multiViewGrouping = verticalMultiView.view([multi1, multi2])

```

4. Setelah itu buat sebuah SimpleForm baru yaitu *blockODENTABLEHEAD_SUBPARENT* dengan parameter menyesuaikan dari yang telah dibuat diatas. Kemudian kita buat sebuah *multiViewGrouping* dengan isinya yaitu *multi1* dan *multi2*

```
1 let content = layout.baseLayout([blockODENTABLEHEAD, multiViewGrouping]);
```

5. Jangan lupa kita tambahkan *multiViewGrouping* kedalam *content*.

FILE PY

Pada file python, tidak terjadi perubahan yang artinya pada python element - element tersebut terbaca menjadi 1 block yang sama

```
1 execVcount = ""
2 session["loggedId"] = kwargs
3 vContainer = Container()
4 vForm = Form()
5 vContainer.update(vForm)
6 vBlockOdentablehead = Codentablehead("ODENTABLEHEAD")
7 vBlockOdentablehead.setListObj([Coth_textnormal,Coth_textdisplay,Coth_livtextlovmulticols,Coth_textupper,Coth_combolistapi])
8 vBlockOdentablehead.ObjRecord = CRecord
9
10 vBlockOdentabledet = Codentabledet("ODENTABLEDET")
11 vBlockOdentabledet.relationBlock = vBlockOdentablehead
12 vBlockOdentabledet.setListObj([Cotd_textnormal,Cotd_oth_textnormal,Cotd_textdisplay,Cotd_livtextlovmulticols,Cotd_textupper])
13 vBlockOdentabledet.ObjRecord = CRecord
14
15 vContainer.getForm().dataBlocks = []
16 vContainer.getForm().append(vBlockOdentablehead)
17 vContainer.getForm().append(vBlockOdentabledet)
18 ""
```

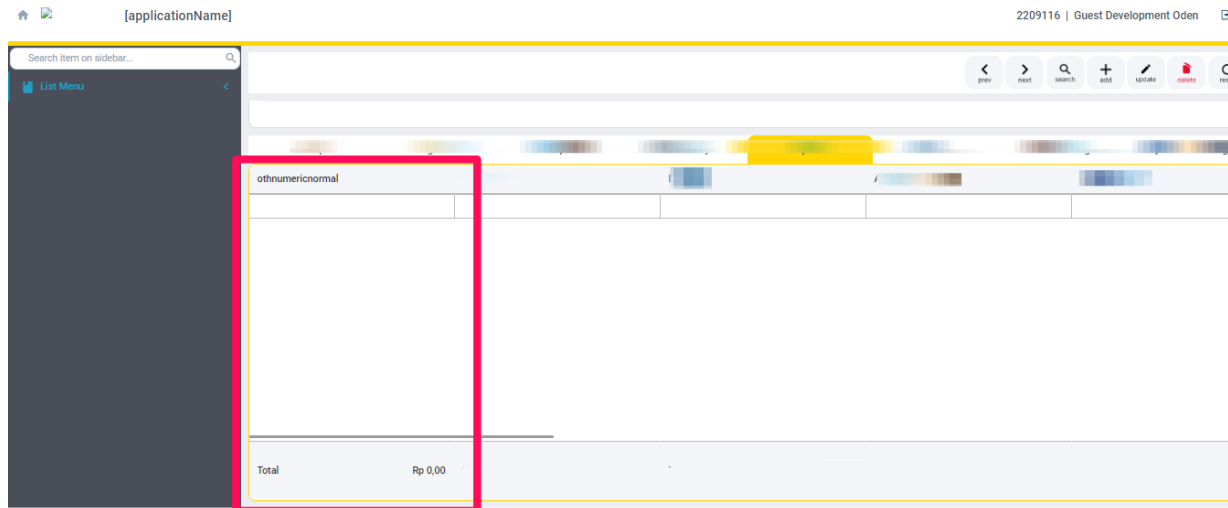
Seperti pada code diatas, tidak ada block ODENTABLEHEAD_SUBPARENT, block tersebut hanya berlaku di Javascript saja.

5.24 How To Create Custom Footer

Dokumentasi untuk membuat custom footer pada Datagrid

Custom Footer merupakan fitur untuk menggunakan function seperti (SUM,MIN,MAX, etc) disuatu column Datagrid.

Gambaran



untuk menggunakan custom footer buatlah ikutilah langkah berikut
masukan code seperti gambar berikut di element Js

Langkah Pembuatan

FILE JS

```
let elementMB_View_Payment= [
  NUMERIC.NUMERICNORMAL({ITEM_ID: "othnumericnormal", LABEL: "othnumericnormal", LENGTH: 7,
    UPPER: true,
    CUSTOM_FOOTER: [
      {
        content: "MULTI_COLUMNS", columns: [
          "Total", { content: "SUM", isCurrency: true }
        ],
      }
    ]
  })
]
```

Di dalam element js terdapat tambahan dictionary CUSTOM_FOOTER yang akan digunakan untuk membuat CUSTOM FOOTER di DATAGRID.

content: content untuk membuat jarak antara label dan value ditampilkan.

content : "SUM" merupakan tempat untuk memasukan fungsi seperti SUM,MAX,MIN,AVG.

isCurrency : true untuk merubah tampilan dalam bentuk currency.

Langkah Pembuatan custom_footer dengan customFunction

FILE JS

```
TEXTBOX.TEXTNORMAL({ITEM_ID: "othnumericcurrency", LABEL: "othnumericcurrency", LENGTH: 13, UPPER: true, CUSTOM_FOOTER: [
    {
        content: "MULTI_COLUMNS", columns: [
            "TOT_SEMUA", { content: "CUSTOM_FUNCTION", customFunction: totalBayarlain ,isCurrency: true }
        ],
    },
])
```

content : isi dengan CUSTOM_FUCTION

customFuction = buatlah 1 function.

untuk mengakses CUSTOM_FOOTER dari colomn lain bisa buat seperti contoh ini.

FILE JS

```

let totalBayarlain = function() {
  let win1 = winOperatorDt["NAMABLOCK"]["ITEM_ID.SUM"]
  let win2 = winOperatorDt["NAMABLOCK"]["ITEM_ID.SUM"]
  let win3 = winOperatorDt["NAMABLOCK"]["ITEM_ID.SUM"]
  let win4 = winOperatorDt["NAMABLOCK"]["ITEM_ID.SUM"]
  let win5 = winOperatorDt["NAMABLOCK"]["ITEM_ID.SUM"]
  let win6 = winOperatorDt["NAMABLOCK"]["ITEM_ID.SUM"]
  let total = win1+win2+win3+win4+win5+win6

  return total
}

```

winOperatorDt: untuk menampung total dari colomn tersebut.

namablock: nama dari block tersebut.

item_id: nama item_id ditambah dengan fungsi yang dipakai contoh SUM.

5.25 How To Add Hak Branch in Oden Webix

Hak Branch adalah daftar branch branch yang memiliki akses pada query sql tertentu dan biasanya digunakan pada where clause suatu query sql.

FILE JS

Pada file Javascript tidak terjadi perubahan karena hak branch dilakukan pada file python

FILE PY

- Kita dapat menggunakan code dibawah ini untuk mengakses hak branch

```

hak_branch = session["loggedId"]["access_menu"]["branch"]
if len(hak_branch) == 1:

```



```

branch = hak_branch[0]
p_branch = "and NAMA_KOLOM = '"+branch+"'"
else:
branch = tuple(hak_branch)
p_branch = f"and NAMA_KOLOM in {branch}"

```

- berdasarkan code diatas hak_branch tersebut disimpan pada session["loggedId"] ["access_menu"]["branch"], jika pada session hanya didapat 1 hak branch maka hanya akan diambil 1 saja dengan mengakses array ke 0 nya, sedangkan jika lebih dari 1 maka hak branch tersebut akan dimasukkan kedalam tuple.
- Kemudian hak branch tersebut digunakan pada variabel p_branch yang terdapat kata NAMA_KOLOM yang diganti dengan nama kolom yang ada di database.
- Jika ingin menggunakan hak branch tersebut, cukup menambahkan pada where clausenya menggunakan format f"""" """" seperti pada code dibawah ini

```

sql = f""""SELECT nama_kolom
FROM nama_tabel
WHERE
{p_branch}
""""

```

5.26 How To Make Date Format in LOV

dokumentasi pembuatan view lov untuk menampilkan date.

file JS

```

LOVFIELD.TEXTLOVMULTICOLS({ITEM_ID: ["othlivetextmulticols"], LABEL :["
othlivetextmulticols"], WINDOW_ID:"wdLovdate", TOTAL_COLS:1, LENGTH:50,
UPPER:true, DISPLAY_ITEM:false, ID_DATE_TYPE:["tgltransfer"]}),

```

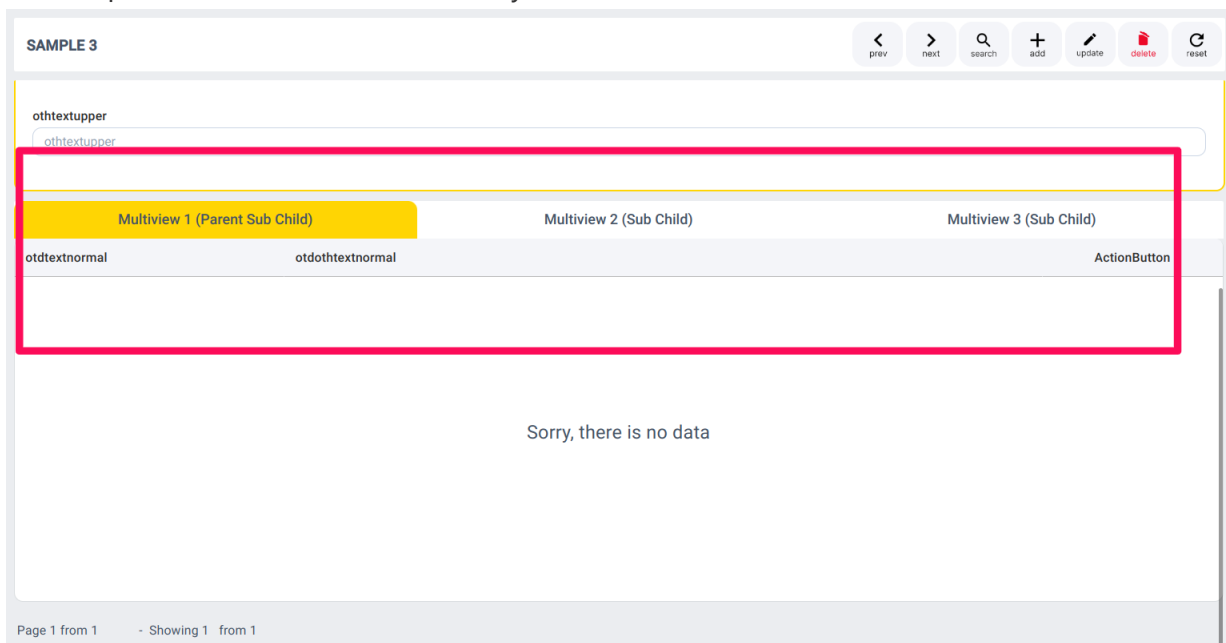
tambahkan ID_DATE_TYPE: dan berisi list item yang mengandung date

*Pastikan pada Item python diubah menjadi Calendar

5.27 How To Create Subchild Multiview Datagrid (1 Table)

Dokumentasi membuat sub-child multiview datagrid yang merupakan pecahan dari parent datagrid pada multiview yang sama

Multiview Subchild Datagrid merupakan konsep dimana sebuah view datagrid yang memiliki 1 table, dipecah ke block multiview lainnya.



FILE JS

1. Buat Parent dari subchild

```
'ODENTABLEDET':
{
  'ELEMENT': ['otdtextnormal', 'otdothtextnormal'],
  'PRIMARY_KEY': ['otdtextnormal'],
  'RELATION_KEY': ['othtextnormal'],
  'WHEN_NEW_FORM': {},
  'BLOCK_TYPE': ['DATAGRID', 'CHILD'],
  "LOAD_DATA_AT_START": true,
```

```
"REST_API": { "BLOCK_NAME": "ODENTABLEDET", "PARAM_ID":
["othtextnormal"] },
},
```

- BLOCK_TYPE pada parentnya, tidak ada perubahan karena dia juga masih merupakan child dari form diatas jadi pada BLOCK_TYPE terdapat 'DATAGRID' & 'CHILD'
- jika ingin menambahkan LIMIT_DATA, maka harus di tambahkan ke parent child dan sub childnya.

2. Kemudian buat sub child dari *ODENTABLEDET*.

```
'ODENTABLEDET_SUBCHILD1':
{
'ELEMENT': ['otdtextdisplay', 'otdlivtextlovmulticols'],
'PRIMARY_KEY': ['otdtextnormal'],
'RELATION_KEY': ['othtextnormal'],
'WHEN_NEW_FORM': {},
'BLOCK_TYPE': ['DATAGRID', 'SUB_CHILD', 'ODENTABLEDET'],
'LOAD_DATA_AT_START': false,
'REST_API': { "BLOCK_NAME": "ODENTABLEDET_SUBCHILD1", "PARAM_ID":
["othtextnormal"] },
'LOV_API': {
"otdlivtextlovmulticols": { "ITEM_ID": "otdlivtextlovmulticols",
"TABLE_ID": "dtDisplayDetView" }
}
},
```

- BLOCK_TYPE : Tambahkan pada bagian BLOCK_TYPE seperti dibawah ini, Pada index kedua kita menggunakan 'SUB_CHILD' karena block ini merupakan sub child dari block *ODENTABLEDET_SUBCHILD1*

```
'BLOCK_TYPE': ['DATAGRID', 'SUB_CHILD', 'ODENTABLEDET'],
```

- REST_API : BLOCK_NAME disesuaikan dengan nama blocknya, kemudian PARAM_ID menggunakan param yang sama dengan REST_API parentnya
- PRIMARY_KEY & RELATION_KEY : samakan dengan *ODENTABLEDET* yang merupakan *parentnya*
- Nama block bisa disesuaikan, ex ODENTABLEDET_SUBCHILD1 (tidak perlu menggunakan SUBCHILD1)

3. Membuat subchild ke 2

```
{
  'ELEMENT': ['otdtextupper'],
  'PRIMARY_KEY': ['otdtextnormal'],
  'RELATION_KEY': ['othtextnormal'],
  'WHEN_NEW_FORM': {},
  'BLOCK_TYPE': ['DATAGRID', 'SUB_CHILD', 'ODENTABLEDET'],
  'LOAD_DATA_AT_START': false,
  'REST_API': { "BLOCK_NAME": "ODENTABLEDET_SUBCHILD2", "PARAM_ID":
  ["othtextnormal"] },
}
```

- BLOCK_TYPE : Tambahkan pada bagian BLOCK_TYPE seperti dibawah ini, Pada index kedua kita menggunakan 'SUB_CHILD' karena block ini merupakan sub child dari block *ODENTABLEDET_SUBCHILD1*

```
'BLOCK_TYPE': ['DATAGRID', 'SUB_CHILD', 'ODENTABLEDET'],
```

- REST_API : BLOCK_NAME disesuaikan dengan nama blocknya, kemudian PARAM_ID menggunakan param yang sama dengan REST_API parentnya
- PRIMARY_KEY & RELATION_KEY : samakan dengan *ODENTABLEDET yang merupakan parentnya*
- Nama block bisa disesuaikan, ex ODENTABLEDET_SUBCHILD1 (tidak perlu menggunakan SUBCHILD1)

4. Membuat element element untuk block block tersebut

```
let elementODENTABLEDET = [
  TEXTBOX.TEXTPRIMARY({ ITEM_ID: 'otdtextnormal', LABEL: 'otdtextnormal',
  LENGTH: 10 }),
  TEXTBOX.TEXTNORMAL({ ITEM_ID: 'otdothtextnormal', LABEL:
  'otdothtextnormal', LENGTH: 10 }),
]
let elementODENTABLEDET_SUBCHILD1 = [
  TEXTBOX.TEXTNORMAL({ ITEM_ID: 'otdtextdisplay', LABEL: 'otdtextdisplay',
  LENGTH: 10 }),
  LOVFIELD.TEXTLOVMULTICOLS({ ITEM_ID: ["otdlivtextlovmulticols",
  "displayitemdet"], LABEL: ["Display Code", "Display Desc"], WINDOW_ID:
  "wdDisplayDetView", TOTAL_COLS: 2, LENGTH: 10 }, { VALIDATE_ITEM:
  validateotdlivtextlovmulticols }),
]
let elementODENTABLEDET_SUBCHILD2 = [
```

```
TEXTBOX.TEXTUPPER({ ITEM_ID: 'otdttextupper', LABEL: 'otdttextupper',
LENGTH: 10 })),
]
```

5. Kemudian ditambahkan kedalam SimpleDatatable dan nantinya di masukkan ke dalam multiview (CARA MEMBUAT MULTIVIEW)

FILE PY

*Pada file python, tidak terjadi perubahan yang artinya pada python element - element tersebut terbaca menjadi 1 block yang sama

```
1  execVcount = ""
2  session["loggedId"] = kwargs
3  vContainer = Container()
4  vForm = Form()
5  vContainer.update(vForm)
6  vBlockOdentablehead = Codentablehead("ODENTABLEHEAD")
7  vBlockOdentablehead.setListObj([Coth_textnormal,Coth_textdisplay,Coth_livtextlovmulticols,Coth_textupper,Coth_combolistapi])
8  vBlockOdentablehead.ObjRecord = CRecord
9
10 vBlockOdentabledet = Codentabledet("ODENTABLEDET")
11 vBlockOdentabledet.relationBlock = vBlockOdentablehead
12 vBlockOdentabledet.setListObj([Cotd_textnormal,Cotd_oth_textnormal,Cotd_textdisplay,Cotd_livtextlovmulticols,Cotd_textupper,Cotd_sequence_file,Cotd_old_filename])
13 vBlockOdentabledet.ObjRecord = CRecord
14
15 vContainer.getForm().dataBlocks = []
16 vContainer.getForm().append(vBlockOdentablehead)
17 vContainer.getForm().append(vBlockOdentabledet)
18 ""
```

Seperti pada code diatas, tidak ada block ODENTABLEDET_SUBCHILD1 dan ODENTABLEDET_SUBCHILD2, block tersebut hanya berlaku di Javascript saja.

5.3 How To Use Special Character in Oden Webix

Dokumentasi untuk menggunakan special character dalam class item pada Oden Webix.

File PY (Folder path : application/)

Dalam setiap class item yang tergenerate, terdapat 1 baris kode allowSpecialChar berupa list (LINE 143).

```
112 class Coth_textarea(TextArea):
113     lengthItem = 10
114     itemId = "othtextarea"
115     databaseColumn = "oth_textarea"
116     databaseItem = True
117     nullable = True
118     primaryKey = False
119     allowSpecialChar = []
120     def validate(self):
121         """
122         Fungsi ini digunakan untuk validasi col
123         @return: dictionary dengan key status
124         key msg = nilai string
125         """
126         return {"status": True, "msg": ""}
```

Pada line 119, terdapat allowSpecialChar (list kosong). List ini berfungsi untuk memperbolehkan sebuah item menggunakan karakter spesial. Jika user memerlukan special character pada sebuah item, user perlu menambahkan isi dari list allowSpecialChar.

Catatan : Untuk tipe elemen DATE / Calendar, Currency, tidak perlu user tambahkan karena sudah ada penambahan secara otomatis.

Contoh :

User memerlukan special character + (plus) dan special character () (tanda kurung)
Maka user perlu merubah list allowSpecialChar menjadi :

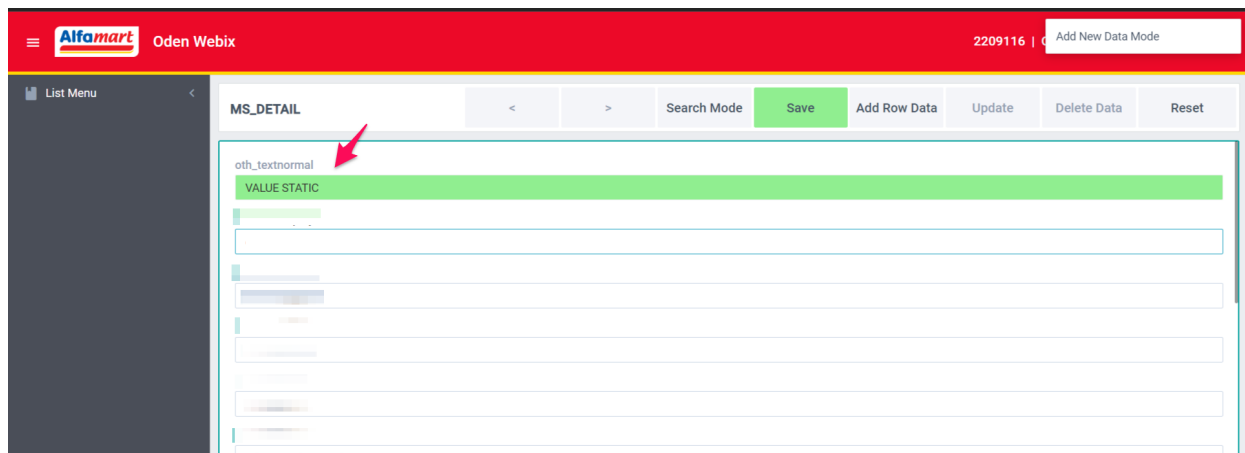
```
allowSpecialChar = ["+", "(", ")"]
```

Dengan menambahkan isi list seperti di atas, user dapat melakukan insert / update dengan special character tersebut.

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.4.1 How To Create New Form Instance JS IN ODEN WEBIX

Dokumentasi untuk membuat New Form Instance pada file JS (Javascript) dalam Oden Webix



New Form Instance digunakan untuk mengisi form secara otomatis pada saat ingin melakukan **Add New Data** sesuai dengan ID dan Value yang ditetapkan pada **WHEN_NEW_FORM**.

FILE JS

Lokasi File = "{LOKASI_PROJECT}\static\application"

Kode **Create New Form Instance**. Masukkan dalam variabel **winConfigForm**

```
NAMAELEMENTHEAD: {  
  ELEMENT: [  
    "itemID1",  
    "itemID2",  
    "itemID3",  
  ],  
  PRIMARY_KEY: ["itemID1"],  
  RELATION_KEY: [],  
  WHEN_NEW_FORM: {"itemID1" : "VALUE STATIC"},  
  BLOCK_TYPE: ["FORM", "PARENT"],  
},
```

Penjelasan Code.

- **NAMAELEMENTHEAD**, berisi Nama dari Head Element , sebagai contoh yaitu **ODENTABLEHEAD**
- **itemID**, berisi **ITEM_ID** pada variable **elementNAMAELEMENTHEAD**, sebagai contoh yaitu **othtextnormal**
- **WHEN_NEW_FORM**, berisi dictionary dengan key yaitu **ITEM_ID** dan valuenya yaitu **DEFAULT VALUE FORM** yang akan ditampilkan pada saat mau menambahkan data (**INSERT**)

Jika ingin membuat auto number sequence (running number) atau dari preInsert, tambahkan kode berikut:

```
WHEN_NEW_FORM: {"itemId":autoNumberSeq},
```

KEMBALI KE TABLE LIST OF CONTENT

5.4.2 How To Create New Form Instance PY In Oden Webix

Dokumentasi untuk membuat New Form Instance pada file PY (Python) dalam Oden Webix

New Form Instance digunakan untuk mengisi form secara otomatis pada saat ingin melakukan **Add New Data** sesuai dengan ID dan Valuenya pada **WHEN_NEW_FORM**.

FILE PY

Lokasi File = "{**LOKASI_PROJECT**}\application\form"

Kode **Create New Instance** pada file PY. **Tambahkan jika belum ada**

```
def newInstance_MENUID_NAMABLOCK(**kwargs):
    newInstance = {'status':True, 'data':{'othtextupper':"NEW
    INSTANCE"}, 'msg':''}
    return newInstance
```

Penjelasan Kode Create New Instance

```
def newInstance_MENUID_NAMABLOCK(**kwargs):
```

- **MENU_ID**, Sesuaikan dengan variabel **menuId**
- **NAMABLOCK**, berisi Nama dari Head Element , sebagai contoh yaitu **ODENTABLEHEAD**

```
def newInstance_MENUID_NAMABLOCK(**kwargs):
    newInstance = {'status':True, 'data':{'othtextupper':"NEW
    INSTANCE"}, 'msg':''}
    return newInstance
```

- **Status** berisi nilai **True**

- **data** berisi object yang terdiri dari key **ITEM_ID** dan value berisi value yang akan ditampilkan pada saat mau menambahkan data (**INSERT**)

Contoh penggunaan kode New Instance

```
379 #NEW FORM INSTANCE
380 def newInstance_FORM_ODENTABLEHEAD(**kwargs):
381     newInstance = {'status':True,'data':{'othtextnormal':"NEW INSTANCE"},"msg':''}
382     # newInstance = selectHeaderQueryRO(sql)
383     # newInstance = selectDataHeaderQueryRO(sql,param)
384     return newInstance
385
```

Bila ingin menambahkan new Instance dari API, dapat menambahkan langsung dalam fungsi seperti berikut ini

Tanpa Parameter

```
selectHeaderQueryRO(sql)
```

```
def newInstance_FORM_ODENTABLEHEAD(**kwargs):
sql = "SELECT kolom1 as item_id FROM ODENTABLE WHERE status = '1'"
newInstance = selectHeaderQueryRO(sql)
return newInstance
```

Dengan Parameter

```
selectDataHeaderQueryRO(sql,param)
```

```
def newInstance_FORM_ODENTABLEHEAD(**kwargs):
param = {'kode': '1'}
sql = "SELECT kolom1 as item_id FROM ODENTABLE WHERE status = %(kode)s"
newInstance = selectDataHeaderQueryRO(sql,param)
return newInstance
```

Penjelasan Kode **Tanpa Parameter** dan **Dengan Parameter**

- **sql**, berisi query SQL untuk mendapatkan data

- **param**, berisi dictionary dengan key dan value. sesuaikan key dengan variabel yang ada di dalam query SQL
- **newInstance** berisi fungsi selectDataHeaderQueryRO dengan parameter **sql** dan **param**

KEMBALI KE TABLE LIST OF CONTENT

5.5 How To Disable Item in Oden Webix

Dokumentasi untuk disable item (display only) pada Oden Webix.

Pada Oden Webix terdapat fitur untuk mendisable element. Untuk menggunakan fitur ini, user harus menambahkan property **DISPLAY_ITEM** di dalam sebuah element.

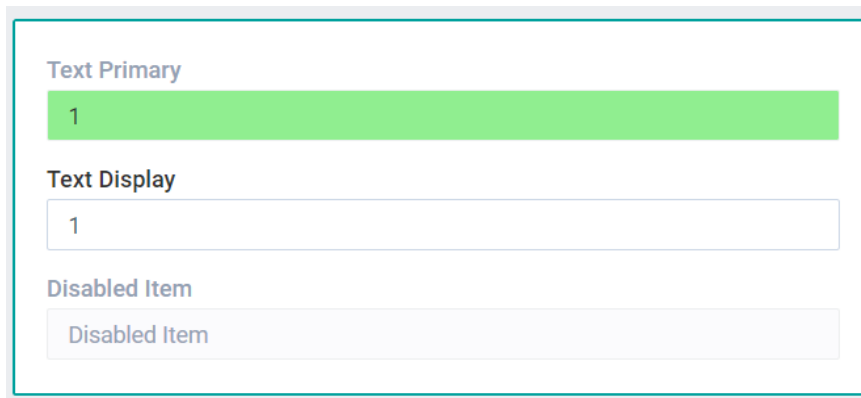
Contoh :

Pada element textlower

```
TEXTBOX.TEXTLOWER({ITEM_ID: "itemId", LABEL: "Item Label", DISPLAY_ITEM: true}),
```

Property **DISPLAY_ITEM : true** membuat element tersebut menjadi display only atau terdisable.

Tampilan pada Oden Webix :



List element yang dapat diberikan property DISPLAY_ITEM antara lain :

TEXTBOX.TEXTUPPER
TEXTBOX.TEXTLOWER
TEXTBOX.TEXTAREA
NUMERIC.NUMERICNORMAL
NUMERIC.CURRENCY
DATE.DDMMYY
DATE.MMY
DATE.YY

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.6 How To Use PRE ITEM in Oden Webix

Dokumentasi penggunaan PRE ITEM di Oden Webix

FILE JS

Lokasi File = "{LOKASI_PROJECT}\static\custom\js\form"

DEFINISI PREITEM

PreItem merupakan fungsi yang digunakan men-trigger **ITEM** agar menampilkan data sesuai dengan value item lain.

Pre Item dapat di gunakan di dalam 3 Element yaitu **TEXTLOVMULTICOLS**, **COMBOLISTAPI**, dan **COMBOLISTAPI DATA**

Kode Pre Item

```
COMBO.COMBOLISTAPI({  
  ITEM_ID: "othcombolistapi",  
  LABEL: "othcombolistapi",}),  
COMBO.COMBOLISTAPI DATA(  
  { ITEM_ID: "othcombolistapidata", LABEL: "Combo List API Data" },  
  { PRE_ITEM: preItemCombo }  
)
```

Penjelasan :

- **PRE_ITEM** , berisi nama fungsi untuk mentrigger value.

Kode PreItemCombo

```
let preItemCombo = function(){
let code = getItemValue('ITEM_ID');
if(code == ""){
RMSGBOX("Pilih Branch Terlebih Dahulu")
let goItemFunc = function(){
GO_ITEM("othcombolistapi")
}
}else{
restapi.restApiDataCombo({"ITEM_ID":"othcombolistapi","API_NAME":"combolistdata_"+menuId, 'PARAM_ID':[code]})
# restapi.restApiLov({"ITEM_ID":"othlov","TABLE_ID":"dtDisplayHeaderView", 'PARAM_ID':["PARAM1","PARAM2"]})
}
}
```

Penjelasan Fungsi:

- let **preItemCombo** merupakan nama fungsi dari preItem, sesuaikan dengan value yang ada di **PRE_ITEM**
- let **code** , berisi value dari ITEM_ID yang diinginkan, contoh **othcombolistapi**
- **restApiDataCombo**, merupakan fungsi dari modul restapi dengan 1 parameter yang berisi 3 data yaitu:
 - **Data pertama** dengan key **ITEM_ID** berisi **ITEM_ID Element** yang ingin di isi valuenya
 - **Data kedua** dengan key **API_NAME** berisi **url api**. sesuaikan dengan nama fungsi pada file Python.
 - **Data ketiga** dengan key **PARAM_ID** berisi list data yang ingin dikirimkan ke pada Api.
- Setelah itu dilakukan pengecekan, jika var code masih kosong maka akan muncul popup dari fungsi RMSGBOX., Jika ada isinya maka akan hit request ke rest api.

Kode PreItemLov

```
let preItemLov = function(){
restapi.restApiLov({"ITEM_ID":"othlov","TABLE_ID":"dtDisplayHeaderView", 'PARAM_ID':["PARAM1","PARAM2"]})
return true
}
```

5.7 RestApiCustom in Oden Webix

Dokumentasi REST API Custom di Oden Webix

RestApiCustom pada Oden digunakan untuk menjalankan suatu fungsi Python tertentu dari sebuah elemen yang belum tersedia restAPI spesifik seperti LOV atau Combolist.

File JS (Folder path : static/application/)

```
let fungsiElemen = function(){
return restapi.restApiCustom({"ITEM_ID":"button","DATA":data})
}

let elementPS_HEADER = [
BUTTON.BUTTONFORM({ITEM_ID:"button", LABEL:"Process"},
{CUSTOM_CLICK:fungsiElemen})
]
```

Pada contoh di block code di atas, **restApiCustom** digunakan sebagai return dari fungsi custom dari sebuah elemen button.

Penjelasan parameter dalam restApiCustom:

- **Parameter pertama** dengan key "**ITEM_ID**" yaitu ("**itemid**") merupakan **ITEM_ID** dari **restApiCustom**. samakan dengan **ITEM_ID** pada elemen.
- **Parameter Kedua** dengan key "**DATA**" yaitu **object / dictionary** berupa ajaxData yang diperlukan pada fungsi di file Python.

File PY (Folder path : application/form/)

API Custom

```
def itemID_menuId(**kwargs):

# ADD YOUR LOGIC HERE
return {"status":True, "data":[{}], "msg":""}
```

Pada file Python, tambahkan fungsi baru dengan format nama **ITEM_ID_menuID**.

Isi dari fungsi dapat berupa logic yang disesuaikan dengan kebutuhan developer, bisa berupa pengecekan kondisi / pengecekan value hingga select query ke Database.

[KEMBALI KE TABLE LIST OF CONTENT](#)

5.8 Session Logged ID in Oden Webix

Dokumentasi penjelasan session['loggedId'] pada Oden Webix.

session['loggedId'] memberikan balikan dictionary :

<pre>{ "access_menu": { "branch": ["MZ01", "1GZ1", "KZ01",], "msg": "success", "short_desc": "Y", "status_access": {"delete": "Y", "insert": "Y", "update": "Y"}, "success": True, }, "attributes": { "branch_code": "Z001", "company": "SAT", "company_code": "01", "name": "Guest Development Oden", "present_store_code": "Z001", "section": "C2271", "tittle_code": "C2271", }, "message": "Ada Session ID", "nik": "2209116", }</pre>	Penjelasan access_menu berisi : branch (dapat diatur melalui staticOdenMenu.py) status_access berisi hak / grant delete insert update attributes berisi : branch_code company company_code name present_store_code section tittle_code message = pesan apakah ada session ID nik = NIK dari loggedID user sekarang
--	---

Contoh :

1. User dapat menggunakan session['loggedId'] untuk mendapatkan **nik** dari user yang sedang *logged* dengan :

```
session['loggedId']['nik']
```

2. User dapat menggunakan session['loggedId'] untuk mendapatkan **branch** dari user yang sedang *logged* dengan :

```
session['loggedId']['access_menu']['branch']
```

5.9 How To Create Automatic Code in Oden Webix

Dokumentasi untuk melakukan automatic code di Oden Webix

Pertama, Buat Auto Create New Form Instance dengan menambahkan kode berikut pada **File JS**

```
'WHEN_NEW_FORM': {'othtextnormal':autoNumberSeq},
```

*Sesuaikan dengan block yang digunakan

Ada beberapa pilihan preInsert Automatic Code

1. Menggunakan Query Dinamis Database lain
2. Menggunakan Trigger Database

Menggunakan Trigger Database

Pada **File PY** tambahkan fungsi **preInsert** pada class block yang diinginkan

```
def preInsert(self):  
    sql = """select trigger_database()"""  
    result = selectHeaderQueryRO(sql)  
  
    namaElement = self.item("namaElement")  
    namaElement.value = result['data'][0]['namaElement']  
    return {'status': True, 'message': ''}
```

Penjelasan kode:

sql	Statement SQL untuk pengambilan data
result	Pemanggilan fungsi pengambilan selectHeaderQueryRO dengan parameter sql
namaElement	Nama Element pada Javascript

Returnnya defaultnya adalah ***return {'status': True, 'message': ''}***

NOTE :

Jika form berbentuk master detail, harap perhatikan catatan berikut:

Agar insert atas bawah secara bersamaan tidak error, harus menambahkan preInsert block datatable untuk set value PK yang berelasi dengan value atas.

```
def preInsert(self):
    self.item("itemId").value =
    self.parentObj.block("BLOCK_NAME").item("itemId").value
    return {"status":True, "msg":""}
```

[KEMBALI KE TABLE LIST OF CONTENT](#)

6.1 How To Use Query Select in Oden Webix

Dokumentasi penggunaan query select dalam Oden Webix.

Dalam Oden Webix terdapat 2 tipe penggunaan Query Select yaitu sebagai berikut

1. **Query Select tanpa parameter**
2. **Query Select dengan parameter**

Query select diletakkan pada file **PY**

QUERY SELECT TANPA PARAMETER

Pada query select tanpa parameter, mengembalikan nilai sebagai berikut

```
return selectHeaderQueryRO(sqlQuery)
```

Contoh penggunaan Query Select Tanpa Parameter

```

1 class Codentablehead(FormLayout):
2     """
3     Class ini digunakan untuk class block layout
4     @return: -
5     """
6     def __init__(self, blockId):
7         super().__init__(blockId)
8         self.whereclause = ""
9         self.databaseTable = "dmhr.odentablehead"
10        self.nonVisibleItems = [{"oth_create_user", "createUser"}, {"oth_update_user", "updateUser"}] #ketik manual kolom yang mau ditempatkan sebagai non visible
11        self.noDbItems = []
12        self.orderBy = ""
13
14    def preInsert(self):
15        self.parentObj.connActive
16        othtextupper = self.item("othtextupper")
17        othtextupper.value
18        othtextupper.value = ' '
19        #YOUR LOGIC HERE
20
21        sqlQuery = """select idTableItem as id, nameTableItem as value from odenTable ms where status='Y'"""
22        selectDataHeaderQueryRO(sqlQuery)
23        return {'status':True, 'message': ''}

```

QUERY SELECT DENGAN PARAMETER

Pada query select dengan parameter, mengembalikan nilai sebagai berikut

```
return selectDataHeaderQueryRO(sqlQuery, paramData)
```

Contoh penggunaan Query Select Dengan Parameter

```

1 class Codentablehead(FormLayout):
2     """
3     Class ini digunakan untuk class block layout
4     @return: -
5     """
6     def __init__(self, blockId):
7         super().__init__(blockId)
8         self.whereclause = ""
9         self.databaseTable = "dmhr.odentablehead"
10        self.nonVisibleItems = [{"oth_create_user", "createUser"}, {"oth_update_user", "updateUser"}] #ketik manual kolom yang mau ditempatkan sebagai non visible
11        self.noDbItems = []
12        self.orderBy = ""
13
14    def preInsert(self):
15        self.parentObj.connActive
16        othtextupper = self.item("othtextupper")
17        othtextupper.value
18        othtextupper.value = ' '
19        #YOUR LOGIC HERE
20
21        paramData = '1'
22        sqlQuery = """select idTableItem as id, nameTableItem as value from odenTable ms where status='Y' and kolomTableItem = %(kolomNo)s"""
23        param = {"kolomNo":paramData}
24        selectDataHeaderQueryRO(sqlQuery, param)
25
26        return {'status':True, 'message': ''}
27

```

KEMBALI KE TABLE LIST OF CONTENT

6.2 How To Use Query Execute in Oden Webix

Dokumentasi penggunaan queryExecute dalam Oden Webix.

Query Execute pada Oden digunakan untuk melakukan perintah INSERT / UPDATE / DELETE

Dalam Oden Webix terdapat 2 tipe penggunaan Query Execute yaitu sebagai berikut :

1. Query Execute tanpa parameter
2. Query Execute dengan parameter

- **QUERY EXECUTE TANPA PARAMETER**

```
sql= """insert into table_oden(colom1,colom2) values('1','abc') """
#Your Query Here

executeQuery(sql)
```

Contoh :

```
15 class COdentablehead(base.FormLayout):
16     """
17     Class ini digunakan untuk class block layout
18     @return: -
19     """
20     def __init__(self, blockId):
21         super().__init__(blockId)
22         self.whereclause = ""
23         self.databaseTable = "dmhr.odentablehead"
24         self.nonVisibleItems = [{"oth_create_user", "createUser"}, {"oth_update_user", "updateUser"}]
25         self.noDbItems = []
26
27     def postInsert(self, data):
28         othtextnormal = self.item("othtextnormal")
29         othddmmyy = self.item("othddmmyy")
30         print(othtextnormal.value, othddmmyy.value)
31         othtextnormal.value = ' '
32         ## INSERT LOGIC HERE ##
33
34         sql = """insert into table_oden(colom1,colom2) values('1','abc') """
35         utilities.executeQuery(sql)
36
37         ##
38         return {'status': True, 'msg': ''}
```

- **QUERY EXECUTE DENGAN PARAMETER**

```
sql = """update tabel_oden set colom2 = 'colname' Where colom1 = %
(code)s""" # Your Query Here
param = {'code': Item}

executeDataQuery(sql, param)
```

Contoh :

```

15 class COdentablehead(base.FormLayout):
16     """
17     Class ini digunakan untuk class block layout
18     @return: -
19     """
20     def __init__(self, blockId):
21         super().__init__(blockId)
22         self.wherclause = ""
23         self.databaseTable = "dmhr.odentablehead"
24         self.nonVisibleItems = [{"oth_create_user", "createUser"}, {"oth_update_user", "updateUser"}]
25         self.noDbItems = []
26
27     def postUpdate(self, data):
28         othtextnormal = self.item("othtextnormal")
29         othddmmyy = self.item("othddmmyy")
30         print(othtextnormal.value, othddmmyy.value)
31         othtextnormal.value = ' '
32         ## INSERT LOGIC HERE ##
33
34         sql = """update tabel_oden set colom2 = 'bca' Where colom1 = %(code)s"""
35         param = {'code': othtextnormal}
36         utilities.executeDataQuery(sql, param)
37
38         ##
39         return {'status': True, 'msg': ''}
40

```

KEMBALI KE TABLE LIST OF CONTENT

oden-dev-3 : UPDATE ODEN ASYNCHRONOUS

Penjelasan update Oden Asynchronous

Oden menggunakan fitur synchronous pada hampir setiap API. Fitur sync sudah mulai memunculkan pesan deprecated (tidak disupport / dilakukan pengembangan lanjutan) oleh engine browser. Perbedaan sync dan async dapat dilihat pada gambar berikut:

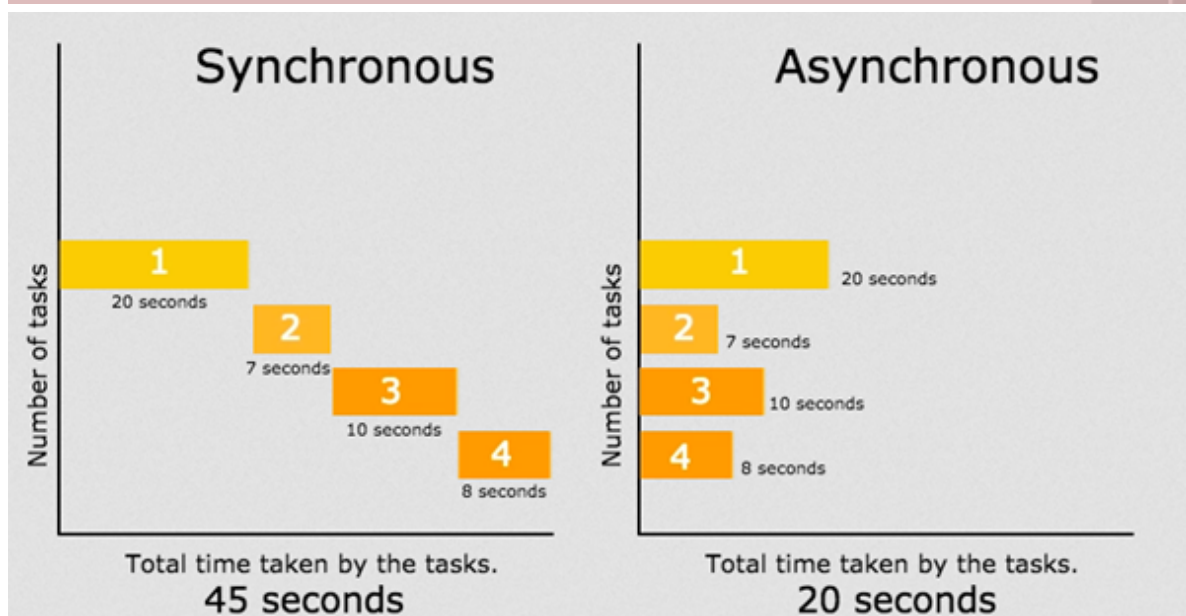
Synchronous Vs Asynchronous



- Blocking Architecture
- The execution of each operation is dependent on the completion of the one before it



- Non-blocking Architecture
- The execution of one task isn't dependent on another



Dengan perubahan api dari sync ke async, terdapat perubahan major dari sisi core, namun dari segi form telah diusahakan untuk seminimal mungkin terdapat perubahan. Dapat disimpulkan, untuk alur validasi terdapat perubahan dari sync ke async.

Semua validasi item pada form maupun datagrid, jika menggunakan restApiCustom, harus diubah menjadi bentuk asynchronous (proses selanjutnya harus dibuatkan then function supaya menunggu balikan dari restApiCustom)

Alur validasi Lov juga terdapat perubahan **jika** hasil validasi digunakan untuk melakukan validasi lainnya pada fungsi yang sama. Maka validasi Lov single line harus dibuat then function agar penerima respond dapat menunggu balikan terlebih dahulu.

Pada winRestApi, jika developer membutuhkan data pada awal form (child) di load, untuk digunakan untuk validasi dan lainnya, maka restApiParam perlu didefinisikan kembali pada winRestApi, ditambahkan then function, dan menghapus key REST_API pada winConfigForm.

1. Update Code

1.1 Update restApiCustom ASYNC

oden-dev-3 : Update restApiCustom ASYNC (VALIDATE ITEM)

Dokumentasi REST API Custom di Oden Webix ASYNC

RestApiCustom pada Oden digunakan untuk menjalankan suatu fungsi Python tertentu dari sebuah elemen yang belum tersedia restAPI spesifik selain LOV atau Combolist.

*Note : perubahan terletak pada bagian fungsi js dengan menambahkan makePromise, then function serta resolve

File JS (Folder path : static/application/)

```
let validateItem = makePromise(function(resolve) {
  let result =
  restapi.restApiCustom({"ITEM_ID":"othtextnoraml","DATA":data});
  result.then(function(respond) {
    if (!respond.status) {
      resolve({"status":false, "tipe":"error","pesan":""})
    }
    resolve({"status":true})
  })

})

let elementPS_HEADER = [
```

```

TEXTBOX.TEXTNORMAL({ITEM_ID:"othtextnormal", LABEL:"Text Normal",
LENGTH: 10}, {VALIDATE_ITEM:validateItem})
]

```

Pada contoh di block code di atas, **restApiCustom** digunakan sebagai return dari validate item dari sebuah elemen text normal.

Detail perubahan sync-async :

SYNC	ASYNC
<pre> let validateItem = function(){ } </pre>	<pre> let validateItem = makePromise(function(resolve){ }) </pre>
<pre> let result = restapi.restApiCustom({"ITEM_ID":"oth textnoraml","DATA":data}); if (!respond.status) { return {"status":false, "tipe":"error","pesan":""} } return {"status":true} }) </pre>	<pre> let result = restapi.restApiCustom({"ITEM_ID":"othte xtnoraml","DATA":data}); result.then(function(respond){ if (!respond.status) { resolve({"status":false, "tipe":"error","pesan":""}) }else{ resolve({"status":true}) } }) </pre>

Penjelasan :

1. Perubahan terjadi pada deklarasi fungsi
2. Pada async, perlu then function
3. Pada async, perlu resolve, dan resolve harus selalu ada pada kondisi, tidak boleh diletakkan paling akhir karena dapat menyebabkan validasi ter-resolve true padahal api async masih dieksekusi.

Penjelasan parameter dalam restApiCustom:

- **Parameter pertama** dengan key "**ITEM_ID**" yaitu ("**itemid**") merupakan **ITEM_ID** dari **restApiCustom**. samakan dengan **ITEM_ID** pada elemen.
- **Parameter Kedua** dengan key "**DATA**" yaitu **object / dictionary** berupa ajaxData yang diperlukan pada fungsi di file Python.

File PY (Folder path : application/form/)

API Custom

```
def itemID_menuId(**kwargs):  
  
    # ADD YOUR LOGIC HERE  
    return {"status":True, "data":[{}], "msg":""}
```

Pada file Python, tambahkan fungsi baru dengan format nama **ITEM_ID_menuID**.

Isi dari fungsi dapat berupa logic yang disesuaikan dengan kebutuhan developer, bisa berupa pengecekan kondisi / pengecekan value hingga select query ke Database.

Untitled